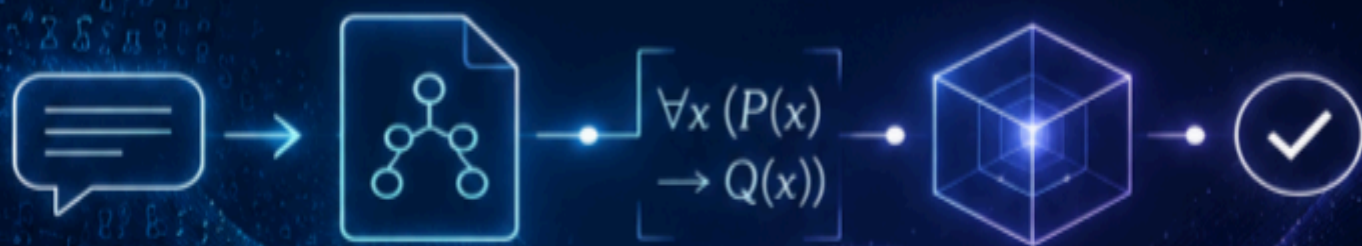


LENGUAJE NATURAL EJECUTABLE

Estudio crítico de la semántica formal, la interpretación abstracta, la ejecución simbólica y las arquitecturas de LLM fundamentadas simbólicamente



LENGUAJE NATURAL EJECUTABLE

Estudio crítico de la semántica formal, la interpretación abstracta,
la ejecución simbólica y las arquitecturas de LLM fundamentadas
simbólicamente

*Hacia un runtime neuro-simbólico para documentos extensos, reglas y sistemas
agénticos*



Derechos de autor © [2026] Axiologic Research

Todos los derechos reservados.

KDP derechos de publicación de Outfinity SRL (Iași, Rumania).

Disponible de forma gratuita en el sitio web de Axiologic (www.axiologic.net).

Nota del autor

Este trabajo fue creado bajo el paraguas de Axiologic Research como parte de dos programas de investigación interconectados, uno dedicado a la Inteligencia Artificial y el otro al Outfinitism, ambos liderados por Sînică Alboaie. Para explorar los detalles de cómo estos dos programas se cruzan y se construyen entre sí, invitamos a los lectores a visitar la página de Outfinity Venture Studio en www.outfinity.ch. [Mientras que los m](#)

odelos de lenguaje de Inteligencia Artificial se utilizaron para ayudar con la generación de texto, la filosofía fundamental, la estructura temática y el enfoque analítico se derivan exclusivamente de las décadas de estudio dedicado del autor de fenómenos sociales, tecnologías sociales y tecnologías duras genuinas resultantes de varios proyectos de investigación europeos y autofinanciados. Recientemente, dentro del proyecto de investigación de Aquiles, hemos estado estudiando profundamente la literatura generada AI y su verificación automatizada utilizando enfoques neuro-simbólicos; todos los libros gratuitos puestos a disposición del público son parte del conjunto de obras que utilizamos para probar nuestras últimas tecnologías.

Índice

Resumen.....	5
Prefacio: alcance, método y posición crítica.....	8
1. Del lenguaje fluido a la semántica ejecutable.....	11
1.1 Qué requiere la ejecución y dónde pertenece el límite de confianza.....	11
1.2 La brecha de formalización: por qué un solucionador correcto todavía puede producir la respuesta incorrecta.....	15
2. El linaje intelectual del lenguaje ejecutable.....	18
2.1 Lenguajes controlados, semántica formal, discurso y lógica natural.....	18
2.2 Análisis semántico, consultas ejecutables, inducción de programas y sistemas neurosimbólicos tempranos.....	22
3. Interpretación abstracta y ejecución simbólica del lenguaje.....	26
3.1 Interpretación abstracta como una semántica de incertidumbre limitada.....	26
3.2 Ejecución simbólica, resolución de restricciones, comprobación de modelos y refinamiento semántico.....	30
4. Lo que la era LLM ha logrado realmente.....	34
4.1 Autoformalización, razonamiento asistido por programas, lógica, especificaciones temporales y planificación.....	34
4.2 Análisis de programas, exploración simbólica, transformadores abstractos y LLM certificados....	38
5. Por qué fallan los sistemas de lenguaje ejecutable.....	42
5.1 Base, identidad, formalizaciones espurias, ontologías y conocimiento mundial.....	42
5.2 Pragmática, no monotonicidad, escala, seguridad y falla de evaluación.....	46
6. Familias de arquitectura para lenguaje natural ejecutable.....	50
6.1 Diseños de la competencia: Representaciones universales, carteras de formalismo, compilación controlada y sistemas de consulta primero.....	50
6.2 Un pipeline de referencia del texto a los símbolos, la ejecución y las respuestas restringidas en lenguaje natural.....	55
7. LongTextJS, CircuitJS y nllAgent: Un estudio de caso crítico.....	59
7.1 Lo que la arquitectura pública implementa y por qué sus límites importan.....	60
7.2 Evaluación independiente: fortalezas, problemas graves actuales y la evolución requerida.....	64
8. Construcción y validación de un runtime neuro-simbólico.....	68
8.1 Estrategia de implementación: un sistema de lenguaje ejecutable mínimo pero científicamente honesto.....	68
8.2 Preguntas de investigación, puntos de referencia y un programa de publicación.....	75
Conclusión: de la forma probabilística a la consecuencia simbólica.....	82
Referencias.....	86

Resumen

Los modelos de lenguaje grandes han hecho del lenguaje natural una interfaz efectiva para la computación, sin embargo, no han hecho que el lenguaje natural sea ejecutable en el sentido fuerte en el que se puede ejecutar un programa, una consulta de base de datos, un sistema de transición o una teoría lógica. Un modelo puede generar una explicación convincente mientras cambia el alcance de un cuantificador, omite una excepción, inventa una premisa o aplica la misma regla de manera diferente en dos lugares. Las herramientas externas mejoran la fiabilidad solo cuando el modelo ha traducido la solicitud correctamente. Un probador de teoremas puede probar la formalización incorrecta perfectamente; un intérprete de Python puede ejecutar un programa que accidentalmente responde al ejemplo mientras tergiversa la pregunta; un ejecutor simbólico puede exponer un camino real a través del software a pesar de que la regla del lenguaje natural utilizada para construir la condición de la ruta fue mal traducida. El problema difícil no es, por lo tanto, simplemente añadir un solucionador después de un LLM. Es construir una cadena auditable desde el lenguaje de origen hasta los símbolos fundamentados, desde los símbolos fundamentados hasta los objetos formales ejecutables, y desde los resultados simbólicos hasta el lenguaje natural sin permitir que el modelo de lenguaje final altere lo que se estableció.

Este libro examina las tradiciones relevantes como un problema de investigación relacionado. Los fundamentos anteriores incluyen lenguajes naturales controlados, semántica de modelo-teórica y discurso, lógica natural, análisis semántico, programación lógica, sistemas de texto a consulta, síntesis de programas a partir de ejemplos y lenguaje, motores de reglas, tablas de decisión, lógicas temporales, planificación automatizada y sistemas neuronales-simbólicos. Los fundamentos del programa-análisis incluyen interpretación abstracta, ejecución simbólica y cónclica, análisis de flujo de datos, comprobación de modelos, resolución de SAT/SMT/CSP, evaluación de Datalog y punto fijo, automatización y verificación de prueba. La literatura reciente de LLM agrega razonamiento asistido por programas, provocación de solucionadores declarativos, autoformalización, traducción de lógica de primer orden, traducción de lógica temporal, planificación formal, autorreparación guiada por solucionador, análisis de programas asistidos por LLM, exploración simbólica guiada por LLM y la síntesis de transformadores abstractos verificados. En estos cuerpos de trabajo, los resultados más fuertes aparecen consistentemente cuando el formalismo objetivo es lo suficientemente estrecho como para tener una semántica explícita y la salida aceptada es verificada por un componente independiente del LLM.

La encuesta presta especial atención a la distinción entre dos usos de la interpretación abstracta. El primero, ya establecido en NLP, verifica un modelo neuronal sobre una familia formalmente definida de perturbaciones textuales. Los dominios abstractos de intervalo y relacional pueden certificar que la predicción de un clasificador no cambia bajo cada sustitución permitida por un conjunto contradictorio especificado. Esta es una garantía formal genuina, pero se refiere a la estabilidad del modelo bajo una relación de perturbación construida; no certifica que las parafrasis humanas arbitrarias conserven el significado, ni que la respuesta original del modelo sea cierta. El segundo uso es más especulativo y más relevante para el lenguaje natural ejecutable: tratar el conjunto de interpretaciones admisibles de un documento como una colección de semántica y computación conservadora puede, no puede, y conclusiones desconocidas sobre ese conjunto. Esto requiere dominios explícitos para la

identidad, el tiempo, la modalidad, la cantidad, el estado de origen, la fuerza normativa y la cobertura. También requiere una garantía condicional honesta: el análisis es sólido solo en relación con el conjunto de interpretación, la ontología, la política de base y las reglas de transferencia que se le proporcionan.

La ejecución simbólica es complementaria. La interpretación abstracta puede mostrar a bajo costo que una violación es imposible bajo la abstracción actual o que una violación sigue siendo posible. La ejecución simbólica puede entonces intentar construir una interpretación concreta y un camino que se dé cuenta. Si no existe tal camino, el testigo fallido puede revelar qué abstracción o elección semántica era demasiado tosca. Esto sugiere un patrón de investigación llamado refinamiento semántico guiado por contraejemplo: el contraejemplo de un solucionador, el núcleo insatisfiable o la prueba fallida se mapean a los pasajes de origen exactos y los compromisos semánticos que lo generaron; se pide al LLM o a un humano que revise solo esos compromisos; se repite el análisis simbólico. El LLM se utiliza para buscar el espacio de formalizaciones, mientras que el núcleo simbólico determina lo que se sigue de cada formalización aceptada.

Los sistemas recientes muestran fragmentos importantes de esta visión. PAL delega computación de procedimiento en Python. SatLM genera restricciones declarativas para un motor de satisfacción. Logic-LM y LINC traducen tareas de razonamiento del lenguaje natural en lógica formal y se basan en solucionadores simbólicos o probadores de teoremas para la deducción. NL2TL y nl2spec traducen los requisitos en especificaciones temporales, con nl2spec soportando explícitamente la corrección interactiva a nivel de subfórmula. El trabajo de planificación formal ha mostrado grandes ganancias cuando LLMs construye instancias de planificación que luego son resueltas y comprobadas por herramientas de sonido. SymGPT traduce las reglas del lenguaje natural ERC a una gramática restringida y utiliza la ejecución simbólica para encontrar violaciones concretas de contratos inteligentes, reportando miles de violaciones de reglas y caminos de ataque explícitos. En la otra dirección, AutoBug descompone el análisis del programa en rutas simbólicas manejadas por LLM, pero sigue siendo aproximada; SAIL utiliza LLMs para buscar transformadores abstractos, mientras que las verificaciones semánticas y sintácticas duras preservan la solidez global. Estos resultados son evidencia de que LLMs son una propuesta efectiva y mecanismos de búsqueda en torno a los núcleos formales, no evidencia de que la generación probabilística en sí misma se haya convertido en un razonamiento formal.

Una revisión sistemática reciente de la neuro-simbólica NLP argumenta que las arquitecturas federativas, en las que los módulos neuronales y simbólicos permanecen autónomos y se comunican a través de interfaces estructuradas, a menudo muestran mayores ganancias que las arquitecturas que simplemente inyectan sesgos simbólicos en las redes neuronales. Sus autores también advierten que la ventaja aparente aún no es concluyente porque los sistemas federativos e inyectivos generalmente se evalúan en diferentes tipos de tareas. Este libro adopta la misma posición disciplinada. El caso arquitectónico de la separación es fuerte porque aclara el límite de confianza, permite la repetición independiente y mantiene el componente simbólico inspeccionable; la afirmación empírica de que una familia universalmente supera a otra sigue siendo no probada [Chatzikyriakidis and Lappin 2026].

El libro también proporciona un estudio de caso crítico del público NaturalLanguageLinterAgent, o nllAgent, arquitectura y sus componentes LongTextJS y CircuitJS. La implementación separa un programa inmutable del lado del documento de un

programa del lado de la teoría reutilizable. Conserva anclajes de origen exactos, tipos de observación versionados, identidades de productores, cobertura, brechas y estado epistémico. La compilación de circuitos deriva las observaciones realmente exigidas por una regla, la publicación comprueba si existen productores compatibles, pruebas de compatibilidad en tiempo de ejecución si el documento concreto produjo observaciones y cobertura suficientes, y un verificador independiente controla si se publica un hallazgo candidato. El aprendizaje se produce en un espacio de trabajo desechable y no puede publicar automáticamente. Estas son decisiones de gobernanza inusualmente serias para un sistema de análisis de documentos con asistencia de LLM. El repositorio público también registra las limitaciones no resueltas en lugar de ocultarlas. Su actual registro de temas graves identifica la incrementalidad del planificador limitado, la propagación incompleta de alternativas de interpretación a través de cada ruta de ejecución, el análisis de compatibilidad conservador para el comportamiento procesal opaco, la planificación mínima de consultas y la indexación temporal, la generación controlada deliberadamente estrecha y el soporte de mutaciones que sigue siendo específico del escenario. Estas son limitaciones de investigación creíbles: se refieren al sustrato semántico y de ejecución, no solo al pulido de la interfaz del usuario.

No se localizó ninguna evaluación independiente revisada por pares o revisión técnica de terceros de nllAgent en las búsquedas realizadas para esta edición. En consecuencia, la evaluación de caso-estudio es explícitamente la propia revisión arquitectónica del autor, basada en el repositorio público, la documentación, el alcance ejecutable y los problemas graves autodeclarados. El juicio es favorable sobre la separación de la evidencia, la teoría, la verificación y la publicación, pero cauteloso sobre la integridad semántica. Las afirmaciones de implementación actuales más fuertes se refieren a controles estructurales y controlados en inglés. La resolución general de identidad, la rica semántica de eventos, el conocimiento del mundo actual, la transferencia amplia de dominios y las reclamaciones de calidad con apoyo estadístico siguen siendo trabajo de investigación en lugar de capacidad establecida.

La propuesta central del libro es un tiempo de ejecución multi-formalismo ejecutable-natural-en el lenguaje. Un LLM propone en primer lugar un pequeño conjunto de estructuras semánticas candidatas, cada una vinculada a las extensiones de fuente exactas y tipificadas contra una ontología seleccionada. Un núcleo de base simbólica admite solo objetos cuya identidad, procedencia, alcance, tiempo, modalidad y evidencia satisfagan contratos explícitos. Un router selecciona uno o más objetivos formales de acuerdo con la pregunta: evaluación relacional o Datalog para consultas de cierre y documentos; tablas de decisión para políticas; SAT/SMT/CSP para restricciones; autómatas para patrones ordenados; verificación de modelos de comportamiento temporal; ejecución simbólica para testigos de ruta; y asistentes de prueba o proveedores de teoremas para reclamos deductivos. La ejecución formal produce un resultado canónico que contiene hechos aceptados, candidatos rechazados, incertidumbre, testigos, objetos de prueba, límites de cobertura y suposiciones. Un LLM final puede explicar ese resultado, pero cada frase en la explicación debe ser licenciada por la estructura canónica y rastreable a la evidencia o una derivación verificada.

Por lo tanto, el lenguaje natural ejecutable no debe publicitarse como la ejecución de una prosa sin restricciones. Debe definirse como un proceso controlado de compilación, conexión a la base, ejecución y realización sobre uno o más candidatos semánticos explícitos. La garantía es condicional pero útil: dada la revisión de la fuente establecida, la ontología, la base, el conjunto de interpretación, la semántica formal y los límites de ejecución, el resultado reportado sigue y se puede reproducir. El principal problema de investigación abierta no es el

solucionador. Es la construcción, comparación, refinamiento y validación de las formalizaciones sobre las que opera el solucionador.

Prefacio: alcance, método y posición crítica

La ambición del lenguaje natural ejecutable es fácil de expresar y fácil de exagerar. Un usuario escribe una regla, política, hipótesis científica, procedimiento o pregunta en el lenguaje ordinario; un sistema lo entiende; el cálculo sigue; el resultado se devuelve como lenguaje ordinario. Los modelos de lenguaje contemporáneo hacen que el principio y el final de esta tubería parezcan resueltos. Pueden leer prosa sin restricciones, generar código y fórmulas, invocar herramientas y explicar los resultados con fluidez. La peligrosa ilusión es que el medio ha desaparecido. De hecho, el medio contiene casi todos los problemas difíciles: decidir qué significado importa, identificar objetos a través de pasajes, distinguir la evidencia explícita de las suposiciones de fondo, preservar la modalidad y las excepciones, seleccionar un formalismo apropiado, demostrar que el artefacto generado está bien formado e impedir que la explicación final diga más que la ejecución establecida.

Un estudio útil no puede ser un catálogo de pequeños sistemas presentados en subsecciones aisladas cortas. El campo está fragmentado precisamente porque las mismas palabras se utilizan para diferentes operaciones. “Ejecución simbólica” puede significar exploración de ruta de estilo King sobre entradas de programas simbólicos, la ejecución de una forma lógica generada en una base de conocimiento, o simplemente el uso de un intérprete externo. La “verificación formal” puede referirse a la corrección de un resultado de solucionador, la corrección de un programa compilado, la fidelidad de la traducción del lenguaje natural o la robustez de una red neuronal bajo una familia de perturbaciones predefinida. “Enrollar” puede significar conectar palabras a columnas de base de datos, entidades a tramos de documentos, proposiciones al conocimiento del mundo o instrucciones de robot a objetos perceptuales. Un tratamiento serio debe mostrar cómo se relacionan estas capas y dónde se detienen las garantías.

Por lo tanto, el método de la encuesta sigue los mecanismos en lugar de las categorías de marketing. Para cada línea de trabajo, el análisis hace cinco preguntas. ¿Qué artefacto se produce a partir del lenguaje? ¿Qué semántica tiene ese artefacto independiente del LLM? ¿Cómo se basa el artefacto en la fuente o el entorno? ¿Qué componente realmente determina el resultado? ¿Qué es y qué no se garantiza cuando el sistema tiene éxito? Los documentos son tratados como evidencia de sus tareas reportadas, no como prueba de una arquitectura general. Se incluyen fuertes resultados numéricos cuando aclaran el mecanismo, pero no se transfieren a dominios con diferente estructura semántica. El horizonte de la encuesta es el 2 de agosto de 2026 e incluye literatura revisada por pares, procedimientos oficiales, documentos de investigación primarios y el repositorio y documentación público nllAgent.

El término “revisión” se utiliza con cuidado. Ahora existen varias encuestas amplias, pero cada una cubre solo una parte del problema de extremo a extremo. La encuesta de lenguaje natural controlado de Kuhn organiza idiomas de acuerdo con la precisión, la expresividad, la naturalidad y la simplicidad [Kuhn 2014]. Hamilton y sus colegas evalúan si la neuro-simbólica NLP ha entregado realmente el razonamiento, la transferencia, la interpretabilidad, la eficiencia de los datos y la generalización fuera de la distribución, mientras que las encuestas más recientes clasifican los sistemas neurosimbólicos orientados a tareas y basados en datos y conocimientos [Hamilton et al. 2024; Delvecchio et al. 2025; Wang et al. 2025b]. Las encuestas de ejecución simbólica y el análisis de programas asistidos por LLM describen los límites de ingeniería de la exploración de la ruta y el papel creciente de los componentes aprendidos

[Baldoni et al. 2018; Wang et al. 2025a]. Una revisión dedicada estudia LLMs como formalizadores para la planificación automatizada [Tantakoun et al. 2025], y la encuesta de autoformalización se concentra en la traducción de matemáticas informales en formas verificables por máquina [Weng et al. 2025]. Una hoja de ruta bidireccional para la ingeniería de requisitos examina tanto el uso de LLMs para hacer que los métodos formales sean accesibles y el uso de métodos formales para restringir los requisitos producidos por [Hamilton et al. 2024; Delvecchio et al. 2025; Wang et al. 2025b] [Ferrari and Spoletini 2025]. Un estudio focalizado de treinta y cinco documentos mapea la formalización asistida por LLM de los requisitos y especificaciones del software, incluido el trabajo dirigido a Dafny, C y Java; su alcance confirma que el soporte de traducción está creciendo más rápido que el aseguramiento semántico de extremo a extremo [Beg et al. 2025]. La revisión sistemática de Frontiers 2026 desarrolla una nueva taxonomía para el NLP neurosimbólico y aboga por una mayor atención a los sistemas federativos [Chatzikiyiakidis and Lappin 2026].

Ninguna revisión encuestada unifica el lenguaje controlado, la base semántica, la interpretación abstracta, la ejecución simbólica, la selección del formalismo, los resultados de prueba y la regeneración restringida como una arquitectura de confianza. Esa conclusión es una síntesis de la literatura más que una afirmación de inexistencia exhaustiva: la terminología difiere entre campos y los documentos relevantes se distribuyen a través de la lingüística computacional, lenguajes de programación, bases de datos, métodos formales, ingeniería de software, planificación y robótica. Cuando no existe una revisión independiente, especialmente para nllAgent, el texto etiqueta sus juicios como análisis original en lugar de implicar respaldo externo.

La posición del libro es obstinada pero conservadora. Rechaza la idea de que se debe confiar en un modelo grande como el razonador final simplemente porque su prosa se asemeja a una prueba. También rechaza la idea opuesta de que solo un analizador semántico completo escrito a mano puede apoyar la ejecución formal útil. En los dominios abiertos, un LLM es a menudo el mejor mecanismo disponible para proponer eventos, relaciones, esquemas formales y mapeos de candidatos. La cuestión arquitectónica es cómo restringir ese poder. La respuesta más defendible es un sistema federado con artefactos intermedios explícitos, interfaces mecanografiadas y versionadas, ejecución simbólica independiente, estados desconocidos conservadores y evidencia rejugable.

Esta posición tiene una importante consecuencia. El propósito del sistema no siempre es producir una formalización definitiva. A veces el artefacto correcto es un conjunto de alternativas. Un pronombre puede referirse a dos posibles actores; una frase legal puede admitir alcances contrapuestos; un requisito puede estar incompleto; dos fuentes pueden estar en desacuerdo; un documento puede no decir si se aplica una excepción. Un tiempo de ejecución simbólico que elige arbitrariamente una interpretación es menos riguroso que un LLM que afirma con franqueza la ambigüedad. La ejecución se vuelve valiosa cuando puede calcular lo que sigue bajo cada lectura admisible, lo que sigue bajo al menos una, qué conclusiones dependen de una elección en disputa y qué evidencia adicional resolvería la elección.

Por lo tanto, la arquitectura propuesta trata estructuralmente la incertidumbre. Los puntajes de confianza pueden clasificar a los candidatos, pero no transforman una interpretación propuesta en un hecho certificado. El estado de un ítem registra cómo se conoció: extraído directamente de un analizador determinista, propuesto por un modelo, asumido para un

escenario, derivado por una regla, comprobado por un verificador, o confirmado por una persona. La cobertura registra si un dominio fue examinado completamente, muestreado, recuperado o no procesado. Solo se permite una conclusión negativa cuando la búsqueda pertinente se cierra bajo una política establecida. Esta es la diferencia entre un sistema simbólico confiable y un clasificador fluido con una salida con forma de JSON.

La discusión sigue siendo accesible en lugar de una prueba pesada. La interpretación abstracta, la ejecución simbólica, la comprobación de modelos y la semántica lógica se explican a través de su esencia operativa y las obligaciones que crean. La notación formal aparece solo donde aclara una distinción de que la prosa podría desdibujarse. El objetivo no es presentar las matemáticas decorativas. Es para dar a los diseñadores de sistemas, revisores e investigadores la precisión conceptual suficiente para decidir dónde es válida una garantía propuesta, dónde es condicional y dónde es meramente aspiracional.

La tesis resultante puede indicarse claramente. LLMs debe descubrir formas formales plausibles; los componentes deterministas y formalmente restringidos deben controlar la conexión a tierra, la compilación, la ejecución y la aceptación; LLMs puede entonces realizar el resultado canónico como lenguaje, pero solo bajo un contrato que impida adiciones sin soporte. Esta no es una solución completa al significado. Es una ruta creíble desde el procesamiento probabilístico del lenguaje hasta el razonamiento limitado, ejecutable y auditable.

1. Del lenguaje fluido a la semántica ejecutable

1.1 Qué requiere la ejecución y dónde pertenece el límite de confianza

La palabra “ejecutar” conlleva un compromiso más fuerte que “interpretar”, “respuesta” o “razón”. Un programa es ejecutable porque sus operaciones tienen un efecto definido en un estado. Una consulta de base de datos es ejecutable porque los operadores relacionales determinan qué tuplas se seleccionan, se unen, agrupan o se excluyen. Un problema de planificación es ejecutable porque las acciones tienen condiciones previas y efectos, y un planificador busca una secuencia que alcance un objetivo. Una teoría lógica es ejecutable en un sentido más amplio porque un procedimiento de inferencia determina si sigue una conclusión, si existe un modelo o si las restricciones son satisfactibles. En cada caso, la operación decisiva es externa a la retórica utilizada para describir la tarea. Un resultado puede ser reproducido de un artefacto y semántica.

Una respuesta LLM es diferente. El modelo mapea un contexto a una distribución sobre las continuaciones. Puede internalizar patrones que se asemejan a algoritmos, y puede resolver muchas tareas correctamente, pero la regla que se aplica en un paso particular no se puede inspeccionar por separado o se garantiza que se use de manera consistente. Las indicaciones de cadena de pensamiento pueden mejorar el rendimiento, sin embargo, una cadena de oraciones no es un objeto de prueba simplemente porque está organizada como pasos. Muestras de autoconsistencia varias continuaciones, no varias ejecuciones formalmente definidas. El uso de la herramienta puede hacer que un valor sea exacto al dejar la interpretación que produjo la llamada de la herramienta no verificada. La distinción crucial es entre un modelo que propone un artefacto ejecutable y un modelo cuya propia generación se trata como ejecución.

El lenguaje natural ejecutable, por lo tanto, comienza con la materialización. El sistema debe crear un objeto con una gramática declarada, sistema de tipo, semántica y mapa de origen. Ese objeto puede ser una consulta relacional, una teoría lógica, una máquina de estado, un conjunto de restricciones temporales, una tabla de decisión, un programa Datalog, un autómata finito o un gráfico tipificado con operadores registrados. La elección particular depende de la tarea. Lo que importa es que el comportamiento del objeto ya no está determinado por la distribución del siguiente token del modelo de lenguaje. Se puede rechazar, ejecutar, probar, comparar y reproducir de forma independiente.

Este requisito es más fuerte que la producción estructurada ordinaria. Un modelo puede emitir JSON sintácticamente válido que contiene entidades inventadas o campos mutuamente inconsistentes. Un esquema valida la forma, no la verdad. Del mismo modo, un modelo puede generar un SQL legal que se une a las columnas equivocadas, una lógica válida de primer orden que invierte un cuantificador o una fórmula temporal que expresa la relación de precedencia opuesta. Por lo tanto, un límite de confianza útil no es simplemente “después de la generación JSON”. Se encuentra después de la conexión a tierra de la fuente, la comprobación de tipo, la compatibilidad semántica y la compilación formal. Cada símbolo que afecta la respuesta debe tener un origen admisible y un rol definido.

La arquitectura naturalmente separa cuatro autoridades. La autoridad de origen determina qué texto, datos u observaciones se suministraron realmente. La autoridad semántica determina qué tipos, relaciones, modalidades y políticas de identidad están permitidas. La

autoridad de la regla determina qué restricciones o transformaciones constituyen la teoría que se está ejecutando. La autoridad de ejecución determina si un resultado candidato sigue a esa teoría. Un LLM puede ayudar a los cuatro, pero no debe fusionarlos silenciosamente. Cuando el mismo modelo inventa una premisa, selecciona una ontología, aplica una regla y escribe el veredicto, no hay una separación significativa entre la evidencia y la conclusión.

La autoridad de la fuente comienza con bytes y lapsos estables. Un sistema de análisis de documentos debe preservar la revisión exacta, la estructura y la ubicación de cada fragmento citado. Debe distinguir una cita de un resumen y un resumen de una inferencia. Cuando un modelo propone que una oración expresa una obligación, la observación debe apuntar a la oración y retener el modelo, el rápido, el esquema y las alternativas que produjeron la clasificación. Si la cita no se puede encontrar en la fuente, la observación es rechazada en lugar de aceptada como una reconstrucción plausible. Esto suena como ingeniería de procedencia mundana, pero es una condición previa para cada garantía posterior.

La autoridad semántica generalmente se distribuye a través de una ontología superior y paquetes de dominio. Una ontología universal es atractiva porque promete interoperabilidad, pero tiende a volverse demasiado abstracta para decidir cuestiones útiles o demasiado grande y polémica para gobernar. Un mejor diseño define una pequeña capa común para los lapsos de origen, menciones, entidades, eventos, tiempo, mundos, estado, procedencia, alternativas, cobertura y brechas. Los paquetes de dominios luego definen conceptos como la intervención clínica, la obligación regulatoria, el evento de objetos narrativos, la llamada al software API o la transacción financiera. Cada paquete debe indicar no solo los nombres de campo, sino también los compromisos y pruebas semánticas. Un tipo llamado “aprobación” no es significativo a menos que el sistema especifique si denota un documento, un acto, un estado, una decisión de autoridad o alguna combinación.

La autoridad de la regla debe ser versionada por separado de la interpretación de los documentos. Una política, teoría de líneas o cambios en el modelo científico a un ritmo diferente de la fuente que analiza. Si las reglas se incrustan de manera invisible en las indicaciones, se hace difícil saber si dos ejecuciones aplicaron la misma teoría. Los artefactos de reglas versionadas permiten la publicación comparada, las diferencias semánticas y la reversión. También permiten múltiples teorías sobre la misma representación de documentos. Un manuscrito se puede verificar en cuanto a la consistencia terminológica, la coherencia causal y el soporte de citas sin volver a compilarlo en un mensaje opaco diferente para cada regla.

La autoridad de ejecución debe ser tan pequeña y mecánica como práctica. Para las consultas relacionales, puede ser un evaluador determinista. Para restricciones, un solucionador SMT o CSP. Para el comportamiento temporal, un corrector modelo. Para propiedades de programa específicas de ruta, un ejecutor simbólico. Para obligaciones de prueba, un probador de teoremas más un pequeño comprobador de pruebas. Para un lenguaje de circuito personalizado, puede ser un intérprete restringido con operadores registrados y verificadores independientes. El componente de aceptación no debe preguntar a un LLM si el resultado “parece correcto”. Debería comprobar a un testigo contra la fuente y la teoría.

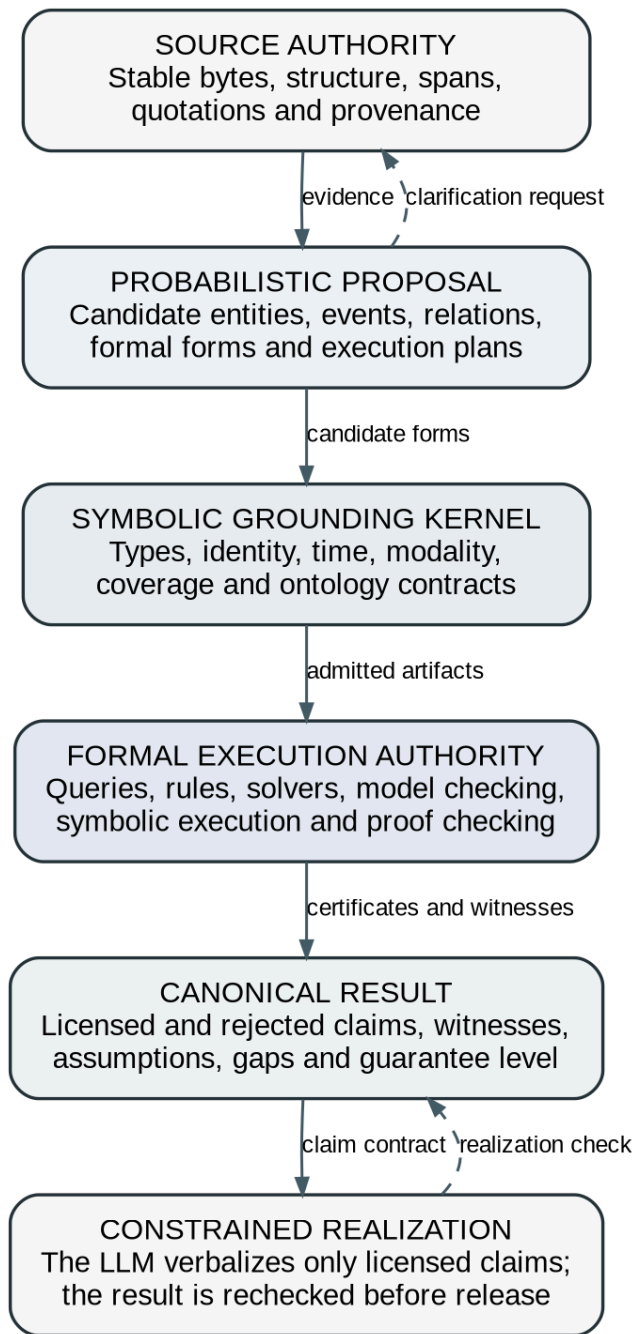


Figura 1. Fronteras de confianza para el lenguaje natural ejecutable. El LLM propone formularios semánticos, mientras que la base de la fuente, la ejecución formal, la aceptación y la licencia de reclamo final siguen siendo verificables de forma independiente.

Una manera útil de clasificar los sistemas es por la ubicación del oráculo final. En un sistema puramente neural, el LLM es el oráculo. En un sistema aumentado de herramientas, una herramienta calcula un valor, pero el LLM puede seguir siendo el oráculo para si la llamada de la herramienta representa la pregunta. En un sistema respaldado por un solucionador, el solucionador es el oráculo para las consecuencias de una formalización generada. En un sistema basado en la evidencia, el solucionador y el núcleo de conexión a tierra determinan conjuntamente el resultado aceptado. En un sistema de prueba, incluso la salida del solucionador puede ser comprobada por un núcleo de confianza más pequeño. Cada paso estrecha el espacio en el que el error probabilístico puede cambiar silenciosamente el veredicto.

Esto no implica que cada aplicación necesite garantía a nivel de proveedor de teoremas. Un asistente de escritura puede usar circuitos deterministas ligeros, mientras que un sistema regulador o de seguridad puede requerir artefactos de prueba formal. El principio importante es que la garantía debe coincidir con el mecanismo. Una comprobación dereminista de regex puede certificar la presencia de un patrón literal, no la ausencia de un concepto. Un extractor de eventos asistido por un modelo puede apoyar una advertencia de continuidad propuesta, no un relato mecánicamente seguro de la identidad narrativa. Un solucionador puede establecer la insatisfacibilidad de las limitaciones, no la integridad de la traducción del lenguaje natural. El rigor consiste en gran medida en negarse a transferir la confianza a través de estas fronteras.

La arquitectura más fuerte es, por lo tanto, federativa. El componente neural no se disuelve en una red simbólica, y el componente simbólico no se reduce a un regulador diferenciable. Siguen siendo sistemas separados con una interfaz mecanografiada. El LLM es responsable de la amplitud, la generación de hipótesis, la normalización semántica y la recuperación del lenguaje inusual. El núcleo formal es responsable de la admisibilidad, ejecución, invariantes, testigos y reproducibilidad. La revisión de 2026 de Chatzikyriakidis y Lappin llama a los sistemas relacionados federativos y observa que a menudo muestran fuertes ganancias en tareas donde la percepción puede separarse de la inferencia. Los autores también enfatizan que las comparaciones siguen siendo confundidas por las diferencias de tareas. El argumento arquitectónico para la federación es, sin embargo, independiente de la superioridad de referencia: los módulos separados hacen que las reclamaciones sean auditables y permiten que un componente rechace otro [Chatzikyriakidis and Lappin 2026].

La salida final del lenguaje natural requiere su propio límite. Una vez que un solucionador devuelve un resultado, es tentador proporcionar el rastro a un LLM y solicitar una respuesta pulida. Sin restricciones, el modelo puede agregar explicaciones de fondo, inferir motivos o simplificar la incertidumbre en certeza. Por lo tanto, un resultado canónico debe enumerar las afirmaciones que pueden ser verbalizadas, su estado, evidencia, suposiciones y alcance. El renderizador puede elegir la redacción y la organización, pero debe comprobarse contra la estructura canónica. Un método práctico es la realización a nivel de reclamo: el modelo genera oraciones asociadas con identificadores de resultados; un verificador asegura que cada oración esté implicada o parafrasee una reclamación permitida; el material no compatible se elimina o etiqueta como comentario. En configuraciones de alta garantía, el lenguaje controlado con plantilla puede ser preferible a la prosa sin restricciones.

La frontera de confianza propuesta en este libro se puede resumir sin código. LLMs puede proponer formas semánticas, mapeos de ontología, elecciones de formalismo, reparaciones y explicaciones. No establecen pruebas, cierran una búsqueda de mundo abierto, certifican identidad, deciden la satisfacción o autorizan una reclamación final. Estas funciones pertenecen a contratos simbólicos explícitos. Esta separación preserva el valor único de los modelos de lenguaje al tiempo que evita que su fluidez se confunda con pruebas.

1.2 La brecha de formalización: por qué un solucionador correcto todavía puede producir la respuesta incorrecta

La debilidad central de la mayoría de los sistemas de razonamiento neurosimbólico no es el motor de inferencia. Los solucionadores SAT, los solucionadores SMT, los motores de base de datos, los probadores de teoremas, los verificadores de modelos y los ejecutores simbólicos a menudo son mucho más confiables que el modelo de lenguaje que los precede. La debilidad radica en la brecha de formalización: la diferencia entre lo que significa la fuente en contexto y el objeto formal sometido a ejecución. Un sistema puede ser perfectamente sólido desde la entrada formal hasta la salida formal, mientras que no es sólido del lenguaje natural para responder.

Considere una simple sentencia de política: “Un revisor puede aprobar una liberación solo después de que se complete el informe de seguridad, a menos que el comandante del incidente autorice una excepción de emergencia”. Un formalizador debe preservar al menos el actor, la acción, el objeto, la condición temporal, el estado de finalización, la excepción, la autoridad y el alcance de “solo después”. Debe distinguir el permiso de la obligación. Debe decidir si la excepción elimina solo la condición temporal o autoriza la aprobación en general. Debe conectar “el informe de seguridad” al artefacto correcto y “el comandante del incidente” a una identidad o rol. Si alguna de estas opciones es incorrecta, un solucionador puede hacer cumplir correctamente la política incorrecta.

El alcance de la cuantificación es una fuente clásica de error. “Cada investigador revisó un artículo” puede significar que cada investigador revisó al menos un artículo, no necesariamente el mismo. “Un artículo fue revisado por cada investigador” tiene una estructura diferente. La negación crea problemas similares: “no todas las muestras pasadas” no significa que no haya pasado ninguna muestra. Los condicionales, comparativos, modalidades, relaciones temporales, anáfora y presuposición crean distinciones que la traducción a nivel de superficie puede borrar. Las tradiciones semánticas formales desarrollaron mecanismos sofisticados para estos fenómenos, pero el lenguaje de dominio abierto agrega conocimiento mundial, convenciones de género y objetivos implícitos que ninguna representación captura por completo.

La brecha también aparece en el análisis semántico de las denotaciones. Un sistema entrenado solo en una pregunta y respuesta final puede encontrar un programa que produzca la respuesta correcta en el entorno de entrenamiento. Tales programas espurios son un problema conocido en el análisis semántico débilmente supervisado [Pasupat and Liang 2016; Guu et al. 2017; Goldman et al. 2018]. La ejecución no distingue el programa previsto de uno accidental si ambos producen la misma denotación. Los sistemas de texto a SQL se enfrentan al mismo riesgo: una consulta puede devolver las filas correctas debido a los valores de datos actuales mientras que contiene la unión o el filtro incorrectos. La decodificación guiada por ejecución elimina los programas que se bloquean o contradicen las restricciones intermedias, pero no puede probar la fidelidad semántica [Wang et al. 2018].

La brecha de formalización se descompone mejor en obligaciones separadas. La base léxica pregunta qué predicados o campos corresponden a palabras y frases. La base de referencia pregunta qué menciones denotan la misma entidad. La base estructural pregunta qué argumentos, alcances y dependencias pertenecen juntas. La conexión a tierra temporal conecta las frases con los instantes, intervalos, calendarios y pedidos. La base modal distingue

los hechos, posibilidades, obligaciones, permisos, intenciones, reclamos reportados y contrafácticos. La base ontológica determina si un concepto pertenece al modelo de dominio seleccionado. La conexión a tierra evidente vincula cada premisa a un lapso de fuente, sensor, fila de base de datos o suposición declarada. Cada obligación puede fracasar de forma independiente.

El sistema también debe decidir cuánto significado formalizar. Una representación semántica total de la prosa sin restricciones no es práctica ni necesaria para la mayoría de las tareas. La regla que se está ejecutando crea demanda. Un circuito de terminología puede necesitar solo tramos exactos y términos normalizados. Una regla de cumplimiento temporal puede necesitar eventos, actores, fechas y excepciones. Un análisis causal puede necesitar afirmaciones, intervenciones, resultados e incertidumbre. La formalización impulsada por la demanda reduce los costos y limita el número de compromisos sin apoyo. También crea un problema de cobertura explícito: si una regla requiere un concepto que los productores disponibles no pueden materializar, la carrera debe detenerse o devolverse desconocida en lugar de aproximarse silenciosamente.

La ambigüedad debe ser representada como alternativas, no colapsada inmediatamente. Supongamos que “Alex le dijo a Jordan que deberían irse” aparece en un procedimiento. El pronombre puede referirse a Alex, Jordan o un grupo, dependiendo del contexto y las convenciones del lenguaje. Un modelo puede clasificar a los candidatos, pero una alta confianza no es una prueba. Si el veredicto de la regla descendente es el mismo bajo todas las vinculaciones plausibles, la ambigüedad es irrelevante y la ejecución puede continuar. Si el veredicto cambia, el resultado debe exponer la dependencia y solicitar aclaración o contexto adicional. Este es un lugar donde la interpretación abstracta se vuelve conceptualmente útil: puede calcular conclusiones sobre un conjunto de interpretaciones sin enumerar cada combinación en detalle.

Un mapeo completo es a menudo imposible porque el lenguaje natural es el mundo abierto. Un documento puede omitir hechos que son verdaderos, asumir conocimiento común o confiar en la práctica institucional. El razonamiento del mundo cerrado trata los hechos no registrados como falsos; el razonamiento de mundo abierto los trata como desconocidos. Ambas políticas son apropiadas en diferentes contextos. Un inventario de la base de datos puede cerrarse para una tabla y un tiempo especificados. Una colección de correos electrónicos recuperados generalmente no es evidencia completa de que no se envió ninguna notificación. Por lo tanto, la capa de conexión a tierra debe adjuntar cobertura a observaciones y búsquedas. Las conclusiones negativas requieren un token de cobertura que indique el dominio, la revisión de la fuente, el alcance, el método, las exclusiones y la política de integridad.

El conocimiento externo introduce complicaciones adicionales. Un LLM general contiene información extensa pero sin versiones y potencialmente obsoleta. Un sistema simbólico no puede tratar la memoria del modelo como un hecho atemporal. El conocimiento mundial debe entrar a través de paquetes de origen, fechados, específicos de la jurisdicción y con contenido. Un paquete puede afirmar que una regla legal fue efectiva en una jurisdicción particular durante un intervalo particular o que una constante científica fue extraída de una fuente nombrada. Los conflictos entre paquetes deben permanecer visibles. Los mundos ficticios o hipotéticos deben ser seleccionables sin contaminación por hechos del mundo actual. La documentación de nllAgent trata correctamente tales paquetes como un límite en lugar de fingir que la memoria de un modelo de base proporciona autoridad formal.

La fidelidad de la formalización puede ser probada, aunque no universalmente probada. Un método es la comprobación bidireccional: traducir el objeto formal de nuevo a lenguaje controlado y compararlo con la fuente. Otra es la prueba contrastiva: generar cambios textuales mínimos que deberían cambiar un elemento formal y preservar el resto. Un tercero es la comparación extensional: pregunte a las formalizaciones competitivas para generar ejemplos distintivos o resultados de consulta. Una cuarta es la estabilidad de la paráfrasa: las formulaciones semánticamente equivalentes deben compilarse a artefactos equivalentes o probablemente relacionados. Una quinta es una explicación contrainstalada: cuando el solucionador produce un testigo, mapea cada dependencia a la fuente y pregunta si el escenario es lingüísticamente admisible. Una sexta revisión humana se centra en el pequeño conjunto de opciones que afectan el resultado en lugar de todo el programa formal.

Los sistemas interactivos como nl2spec ilustran el valor de la reparación local. En lugar de pedir al usuario que entienda o reescriba una fórmula temporal completa, la interfaz asocia subfórmulas con fragmentos de origen, lo que permite corregir un componente equivocado [Cosler et al. 2023]. La retroalimentación de Solver puede desempeñar un papel similar. Logic-LM utiliza errores de sintaxis y ejecución para solicitar la reformalización [Pan et al. 2023]. Los sistemas más avanzados pueden usar núcleos insatisfechos o contraejemplos en lugar de mensajes de error genéricos. La idea central es que la retroalimentación formal debe reducir la cuestión lingüística.

La brecha de formalización también impone límites a las afirmaciones de “solidez” de extremo a extremo. Un sistema puede probar un teorema de la forma: si cada observación aceptada representa con precisión su fuente y la ontología y las reglas son correctas, entonces el veredicto sigue. Esto es valioso. No debe reescribirse ya que “el veredicto se ha demostrado en el documento” a menos que las obligaciones de observación y fundamentación también se hayan cumplido. La garantía debe nombrar sus supuestos y desconocidos. Un sobre de solidez es más honesto que una etiqueta de confianza binaria.

Por lo tanto, este libro trata la formalización como un proceso de compilación con múltiples artefactos intermedios en lugar de un solo aviso. La fuente está segmentada y anclada. Las observaciones de los candidatos se producen bajo esquemas. Se representan las alternativas de identidad y tiempo. Se comprueba la compatibilidad de la ontología. Se selecciona un formalismo objetivo. El compilador produce un objeto ejecutable y un mapa de origen. El ejecutor produce un testigo, prueba, modelo o fracaso. El resultado se repite y se representa. En cada etapa, un error tiene una categoría específica y una posible reparación. La arquitectura no elimina la incertidumbre semántica; evita que la incertidumbre se oculte dentro de la salida fluida.

La conclusión de investigación más importante sigue. Los sistemas LLM respaldados por solucionador deben evaluarse por separado en la fidelidad de formalización y la ejecución formal. Una puntuación de respuesta final alta puede ocultar programas espurios. Una puntuación baja puede ser causada por una buena formalización con un solucionador inadecuado, o por una mala formalización con un solucionador perfecto. Los puntos de referencia necesitan estructuras formales de oro o admisibles, alineaciones de evidencia y pares mínimos semánticos, no solo respuestas. Hasta que esta separación se convierta en estándar, muchas afirmaciones sobre el “razonamiento formal con LLMs” seguirán siendo difíciles de interpretar.

2. El linaje intelectual del lenguaje ejecutable

2.1 Lenguajes controlados, semántica formal, discurso y lógica natural

La idea de que el lenguaje legible por humanos puede soportar la inferencia mecánica es anterior a los grandes modelos de lenguaje en décadas. El trabajo anterior abordó el problema desde la dirección opuesta: en lugar de pedir un modelo estadístico para interpretar la prosa sin restricciones, los investigadores diseñaron lenguajes o teorías semánticas que limitaban la ambigüedad suficiente para apoyar la traducción a la lógica. Estas tradiciones son esenciales porque revelan lo que un sistema de lenguaje ejecutable debe preservar incluso cuando un LLM realiza el análisis front-end.

Los lenguajes naturales controlados son el precedente de ingeniería más claro. Restringen el vocabulario, la gramática o la interpretación mientras conservan una superficie que se asemeja al lenguaje ordinario. El intento de inglés controlado es el ejemplo más conocido. Las oraciones se traducen en estructuras de representación del discurso y luego en lógica formal, permitiendo consultas e inferencias mientras permanecen legibles para los no lógicos [Fuchs and Schwitter 1996; Fuchs et al. 2008]. Los lenguajes controlados se han utilizado para la representación del conocimiento, especificaciones técnicas, wikis semánticos, reglas de negocio y requisitos. La encuesta de Kuhn muestra que la familia es diversa: algunos lenguajes están diseñados para la claridad humana en lugar de la precisión formal, mientras que otros intercambian naturalidad por la semántica determinista [Kuhn 2014].

La lección perdurable no es que el lenguaje sin restricciones deba ser reemplazado por un dialecto controlado. Es que un sistema ejecutable necesita una capa de normalización cuya semántica es más estricta que la prosa ordinaria. Un LLM puede servir como traductor de lenguaje abierto a una forma semántica controlada, pero la forma controlada debe tener una gramática e interpretación independientes. Sin ese límite, el “lenguaje natural controlado” se convierte simplemente en un estilo de escritura preferido y no proporciona ninguna garantía computacional.

Los lenguajes controlados también exponen la importancia de la retroalimentación del autor. Si una oración está fuera del fragmento controlado, un sistema tradicional lo rechaza o produce una ambigüedad explícita. Las interfaces modernas LLM tienden a hacer lo contrario: seleccionan una lectura plausible y proceden. El rechazo a menudo se trata como un defecto de usabilidad, sin embargo, en un sistema de alta seguridad es una forma de integridad epistémica. Una interfaz de idioma ejecutable de próxima generación debería combinar la flexibilidad de la parafrasea LLM con la honestidad de un compilador de lenguaje controlado. Puede proponer varias lecturas normalizadas, explicar las distinciones y preguntar solo cuándo la distinción cambia el resultado.

La semántica formal proporciona un linaje teórico más profundo. La semántica teórica del modelo representa el significado en términos de condiciones bajo las cuales las expresiones son verdaderas en un modelo. La composición conecta la interpretación de una expresión compleja con los significados de sus partes y su combinación sintáctica. La semántica al estilo montage, los cálculos lambda mecanografiados, los cuantificadores generalizados y los marcos relacionados ofrecen relatos precisos de alcance, estructura de argumentos y operadores lógicos. Demuestran que el significado del lenguaje natural no es reducible a una

lista plana de triples. Los cuantificadores, la negación, los verbos intensacionales, la modalidad y el contexto pueden cambiar el comportamiento lógico de frases aparentemente simples.

Para el lenguaje natural ejecutable, la semántica formal contribuye con dos requisitos. En primer lugar, la representación intermedia debe preservar la estructura del operador en lugar de sólo las entidades y relaciones. “Todo”, “algunos”, “no”, “sólo”, “a menos que”, “puede”, “debe” y “antes” no son anotaciones en torno a una proposición; determinan el espacio de los modelos. En segundo lugar, la construcción semántica debe ser de tipo consciente. Un intervalo temporal, una persona, una organización, una proposición y una obligación no pueden ser sustituidos libremente simplemente porque sus etiquetas superficiales se parecen entre sí. La comprobación de tipo es una de las formas más baratas de rechazar las formalizaciones incoherentes antes de invocar a un solucionador.

La teoría de la representación del discurso y los enfoques dinámicos-semánticos relacionados abordan los fenómenos que la lógica de oración por oración maneja mal. Un discurso introduce referentes, los lleva a través de oraciones, actualiza el contexto y crea restricciones de accesibilidad para pronombres y descripciones. Los analizadores Boxer y DRS operacionalizaron estas ideas para el análisis semántico de cobertura amplia [Bos 2008; Liu et al. 2018]. El Banco de Significado Paralelo y las tareas de análisis compartido de DRS permitieron evaluar las representaciones de significado de alcance en todos los idiomas [Abzianidze et al. 2019]. Estos esfuerzos muestran tanto el progreso como la dificultad: la vinculación variable, la asignación de sentidos, el alcance y las relaciones de discurso son sustancialmente más difíciles que la extracción superficial.

Un sistema ejecutable a nivel de documento debe aprender de la semántica dinámica sin requerir un monolítico DRS para cada fuente. Necesita objetos de discurso persistentes, hipótesis de entidades de alcance, eventos, mundos y relaciones de actualización. Debe distinguir una mención de una entidad y un vínculo de identidad candidato de uno establecido. Debe preservar contextos citados, hipotéticos, negados y ficticios. Un gráfico de conocimiento plano que fusiona cadenas iguales en un nodo cometerá errores sistemáticos en documentos largos. Los nombres se repiten, se producen alias, los roles cambian y los documentos describen mundos mutuamente inconsistentes.

El costo práctico del análisis semántico formal completo explica por qué muchos sistemas se movieron hacia representaciones menos profundas. El etiquetado de roles semánticos identifica la estructura de predicado-argumento. La representación de significados representa conceptos y relaciones en un gráfico. La extracción de información identifica entidades, eventos y relaciones. Estas representaciones son útiles, pero su semántica a menudo está incompleta para la ejecución. Pueden omitir el alcance del cuantificador, la modalidad, el anclaje temporal y la diferencia entre el contenido afirmado y el reportado. Un tiempo de ejecución de lenguaje ejecutable debe tratar tales estructuras como observaciones que contienen evidencia en lugar de como una forma lógica completa. Un gráfico puede soportar muchas reglas locales mientras deja los aspectos no resueltos explícitos.

La lógica natural ofrece otro compromiso importante. En lugar de traducir oraciones en un lenguaje formal sin restricciones, los sistemas de lógica natural razonan a través de transformaciones estructuradas cerca de la forma superficial. MacCartney y Manning desarrollaron relaciones de inferencia basadas en la monotonicidad, la implicación léxica y la progetividad [MacCartney and Manning 2007, 2009]. NaturalLI utilizó posteriormente operadores de lógica natural para una inferencia eficiente sobre el texto [Angeli and Manning

2014]. LangPro combina una gramática formal, conocimiento léxico y pruebas de teoremas para la inferencia del lenguaje natural [Abzianidze 2017].

La lógica natural es atractiva porque muchas preguntas de documentos se refieren a relaciones de implicación que pueden ser capturadas sin construir un gran modelo mundial. La sustitución de “perro” por “animal” es válida en un contexto de levantamiento ascendente, pero no necesariamente bajo la negación o las restricciones universales. Estos cálculos son más transparentes que una puntuación de similitud de incrustación. Pueden apoyar los controles de contradicción, el razonamiento taxonómico y la inferencia controlada. Sin embargo, la lógica natural no es un ejecutor universal. Maneja bien cierta monotonicidad y relaciones léxicas, pero no expresa naturalmente procedimientos arbitrarios, aritméticos, de planificación o comportamiento temporal complejo. En una arquitectura multi-formalismo, debería ser un motor entre varios.

Las tradiciones semánticas también aclaran por qué la “base” tiene varios niveles. La semántica formal puede especificar cómo una expresión contribuye a las condiciones de la verdad, pero no determina automáticamente qué objeto del mundo real denota un nombre o si una fuente es creíble. DRS puede representar un referente del discurso, pero la resolución de identidad sigue siendo una tarea empírica. La lógica natural puede propagar la implicación léxica, pero la relación léxica puede ser dependiente del dominio. Por lo tanto, la ejecutabilidad requiere una asociación entre la estructura lingüística, la ontología del dominio y la gestión de la evidencia.

Una arquitectura útil puede distinguir al menos tres productos semánticos. El primero es un modelo fuente que contiene estructura exacta, tramos y citas. La segunda es una capa de interpretación que contiene menciones, eventos, relaciones, modalidades y alternativas mecanografiadas. La tercera es una teoría ejecutable que utiliza interpretaciones seleccionadas para responder a una pregunta en particular. Las técnicas semánticas formales informan principalmente a la segunda capa. Los lenguajes controlados y los probadores de teoremas conectan el segundo y el tercero. La procedencia y la cobertura conectan ambos de nuevo con el primero.

La literatura anterior también advierte contra asumir que una representación puede servir a cada tarea. DRS es expresivo para el discurso, pero costoso de producir y razonar. La lógica de primer orden es bien entendida pero incómoda para los fenómenos temporales, probabilísticos o desviados. Las lógicas de la descripción ofrecen un razonamiento ontológico decidible pero restringen la expresividad. La lógica natural se mantiene cerca del lenguaje, pero está especializada. El inglés controlado mejora la previsibilidad al restringir la entrada. La conclusión correcta no es elegir un ganador, sino preservar suficiente información semántica para reducir una tarea en el formalismo que se ajusta a ella.

LLMs cambiar la economía de este diseño. Pueden generar análisis sintácticos candidatos, reescribir lenguaje incontrolado en formas controladas, sugerir mapeos de ontología y explicar alternativas formales a los usuarios. Hacen que sea práctico intentar la interpretación semántica en documentos que las gramáticas escritas a mano rechazarían. No eliminan la necesidad de las distinciones semánticas desarrolladas por el trabajo anterior. Por el contrario, estas distinciones se convierten en los contratos por los que se juzgan las propuestas de LLM.

Un error recurrente en los sistemas contemporáneos es tratar una tienda triple generada por LLM como una representación semántica completa. Los triples son convenientes para la recuperación y las consultas de gráficos, pero por sí solos no codifican el alcance, la

procedencia, la modalidad, el tiempo, la negación, las alternativas de identidad o la cobertura. Una capa semántica ejecutable debe ser más rica pero finita y operativa. Puede almacenar una afirmación como un objeto tipificado con los participantes, el contexto temporal, la polaridad, la modalidad, el mundo, la evidencia, el productor y el estatus. Puede representar enlaces alternativos sin elegir uno. Puede declarar qué dimensiones están ausentes. Esto es menos elegante que una sola forma lógica, pero es más fiel a cómo los sistemas reales adquieren significado.

La contribución más profunda del linaje pre-LLM es, por lo tanto, la disciplina conceptual. La legibilidad humana no implica la determinación semántica. La determinación formal se logra mediante la restricción, la tipificación, el contexto explícito y el rechazo de construcciones sin soporte. El significado del documento es dinámico y de alcance. La inferencia cerca del lenguaje de la superficie es posible pero especializada. Múltiples representaciones formales son legítimas porque responden a diferentes preguntas. Estas lecciones definen la arquitectura que un compilador basado en LLM debe recuperar en lugar de omitir.

2.2 Análisis semántico, consultas ejecutables, inducción de programas y sistemas neurosimbólicos tempranos

El análisis sintáctico semántico convierte las expresiones del lenguaje natural en objetos formales que se pueden ejecutar contra una base de datos, una base de conocimientos, un entorno o un intérprete. Es el predecesor histórico más cercano al gasoducto de lengua ejecutable propuesto aquí. El campo incluye preguntas sobre tablas y gráficos de conocimiento, texto a SQL, seguimiento de instrucciones, síntesis de expresión regular, interpretación de comandos robóticos y generación de programas. Sus éxitos demuestran el poder de la ejecución externa; sus fracasos revelan por qué la ejecución por sí sola no establece la comprensión.

Los primeros analizadores semánticos se basaron en gramáticas, léxicos y formas lógicas supervisadas. Podrían producir representaciones transparentes, pero requerían anotaciones caras y a menudo se transferían mal entre los dominios. La débil supervisión redujo la carga de anotaciones mediante el entrenamiento de las denotaciones: una pregunta se combina con una respuesta, y el sistema busca un programa que lo produzca. Este enfoque permitió conjuntos de datos más amplios, pero creó el problema del programa espurio. Muchos programas pueden devolver accidentalmente la misma respuesta, especialmente en tablas pequeñas o bases de conocimiento. La señal de entrenamiento luego recompensa el comportamiento sin identificar la semántica [Pasupat and Liang 2016] prevista.

El problema es estructural, no meramente estadístico. Supongamos que la respuesta a “¿Qué ciudad fue sede del evento de 2012?” es Londres. Se pretende un programa que seleccione la fila con el año 2012. Un programa que seleccione la primera fila también puede devolver a Londres en la tabla de entrenamiento. Más datos pueden reducir las correlaciones accidentales, pero no pueden garantizar que el programa aprendido refleje el lenguaje. Se han desarrollado ejemplos abstractos, modelos de alineación, restricciones de tipo y filtrado basado en ejecución para suprimir los candidatos espurios [Goldman et al. 2018; Lee et al. 2023]. Estas técnicas también son importantes para la formalización LLM. Un resultado de solucionador debe ser tratado como una prueba de una formalización, no como su certificado semántico.

Las máquinas simbólicas neuronales combinan la generación de programas neuronales con un intérprete simbólico de Lisp [Liang et al. 2017]. El componente neural propone programas, mientras que el ejecutor rechaza candidatos inválidos y devuelve denotaciones. Mou y sus colegas acoplaron explícitamente la ejecución neuronal distribuida con la ejecución simbólica para consultas de lenguaje natural, reportando mejoras en la precisión, la eficiencia y la interpretabilidad [Mou et al. 2017]. Estos sistemas establecieron un patrón duradero: los modelos neuronales manejan la variabilidad del lenguaje; los intérpretes simbólicos imponen un significado operativo.

Text-to-SQL hizo que el patrón fuera comercialmente importante. Una pregunta de usuario se compila en una consulta cuya ejecución devuelve datos. La decodificación guiada por ejecución puede ejecutar candidatos parciales o completos de SQL y descartar aquellos que producen errores o resultados imposibles [Wang et al. 2018]. El esquema que vincula los nombres y las relaciones. La modernización LLMs mejora sustancialmente la generación, pero la generalización entre dominios, las preguntas ambiguas, las convenciones específicas de la base de datos y las solicitudes sin respuesta siguen siendo difíciles. La lección clave para el lenguaje natural ejecutable es que el entorno formal en sí mismo proporciona limitaciones

útiles. Los tipos de columna, las claves extranjeras y los errores de consulta proporcionan retroalimentación que carece el modelado de lenguaje puro.

Sin embargo, text-to-SQL también muestra los límites de una única métrica de precisión final. La coincidencia de cadena exacta puede rechazar consultas equivalentes. La precisión de la ejecución puede aceptar consultas semánticamente incorrectas que devuelven el mismo resultado. Una evaluación rigurosa necesita equivalencia bajo diversas instancias de base de datos o contraejemplos cuidadosamente contruidos. Esto es directamente análogo a la evaluación de la formalización de reglas: un programa formal generado debe ser probado en escenarios distintivos, no solo en una respuesta observada.

La síntesis de programas del lenguaje natural generaliza la idea. Los sistemas generan expresiones regulares, fragmentos de código, transformaciones de datos o pequeños programas específicos de dominio. Los ejemplos pueden restringir el espacio candidato, y los sistemas de tipo pueden eliminar los programas sin sentido. La generación neuronal de expresiones regulares mostró que el mapeo de lenguaje a programa puede funcionar incluso con un conocimiento limitado del dominio, al tiempo que ilustra la fragilidad cuando las descripciones están infraespecificadas [Locascio et al. 2018]. El LLMs evaluado por el programa más tarde trasladó el cálculo fuera del modelo, pero el problema fundamental de la síntesis se mantuvo: el modelo debe elegir un programa cuya semántica coincida con la solicitud.

Por lo tanto, un compilador robusto debe buscar múltiples formas de evidencia. Las descripciones en lenguaje natural limitan la intención. Los ejemplos de entrada-salida limitan el comportamiento de extensión. Los tipos limitan las operaciones admisibles. Los esquemas de dominio limitan el vocabulario. Pruebas de casos de borde de sonda. Un programa generado que satisface todas estas restricciones es más creíble que uno que simplemente ejecuta. Este principio de evidencia múltiple se puede aplicar a las reglas del documento: autoridad textual, ejemplos positivos y negativos, hallazgos esperados y mutaciones semánticas deben dar forma a un circuito antes de la publicación.

La programación lógica proporciona otro objetivo ejecutable. Las reglas Datalog calculan las consecuencias de un punto fijo y admiten una evaluación incremental. Los lenguajes de prolog y relacionados representan relaciones y búsqueda. La interpretación abstracta se ha utilizado durante mucho tiempo para analizar programas lógicos, mostrando un puente conceptual entre la semántica declarativa y el análisis de programas. Para aplicaciones en lenguaje natural, la programación lógica se adapta bien a los cierres monótonos: relaciones transitivas, taxonomías, propagación de dependencias y reglas de política sin incumplimientos complejos. Es menos adecuado cuando el razonamiento depende en gran medida de la incertidumbre, las fuentes inconsistentes o las excepciones no monotónicas a menos que se extiendan con mecanismos explícitos.

Las tablas de decisión y los sistemas de reglas de producción a menudo se pasan por alto en las encuestas académicas, pero son muy relevantes. Muchas políticas no son teorías lógicas profundas; son combinaciones finitas de condiciones y resultados. Una tabla de decisiones hace visible la cobertura, los conflictos y las combinaciones que faltan. Se puede compilar en predicados ejecutables y probar sistemáticamente. El lenguaje natural se puede utilizar para escribir o explicar la tabla, mientras que la tabla sigue siendo la semántica autorizada. Para el linting de documentos empresariales, las tablas de decisión pueden ser más seguras y fáciles de validar que la lógica arbitraria de primer orden.

Las instrucciones temporales y de procedimiento requieren objetivos más ricos. Las máquinas de estado finito, los árboles de comportamiento, los lenguajes de planificación y las lógicas temporales representan acciones y condiciones ordenadas. La robótica ha utilizado durante mucho tiempo analizadores de lenguaje que componen primitivas ejecutables. Los sistemas de manipulación neuro-simbólica recientes continúan este patrón: un analizador de lenguaje mapea una instrucción a un programa cuyos componentes pueden incluir la percepción neuronal y el recuento simbólico, el razonamiento espacial o el control de la acción. El éxito de tales sistemas depende de una biblioteca de habilidades restringida y un entorno fundamentado, no de una cobertura semántica no restringida [Tziafas and Kasaei 2026].

La programación lógica probabilística y los sistemas neurosimbólicos diferenciables intentan una integración más estrecha. DeepProbLog, Logic Tensor Networks y los sistemas relacionados combinan predicados aprendidos con la estructura lógica [Manhaeve et al. 2018; Badreddine et al. 2022]. Son valiosos cuando la incertidumbre y el aprendizaje son intrínsecos, y pueden inyectar restricciones en la capacitación. Sin embargo, la diferenciabilidad no equivale a la garantía formal. Los valores de verdad difusos y las sanciones suaves pueden guiar los modelos sin producir obligaciones de prueba discreta. Para el lenguaje natural ejecutable, tales sistemas pueden estimar o clasificar a los candidatos semánticos, pero un límite de aceptación de alta seguridad aún se beneficia de un comprobador simbólico separado.

La revisión neurosimbólica NLP de 2026 distingue los sistemas inyectivos, que compilan información simbólica en modelos neuronales, de los sistemas federativos, que preservan los componentes autónomos. Informa que los sistemas de inyección dominan la literatura, pero a menudo muestran ganancias modestas en tareas amplias NLU, mientras que los sistemas federativos muestran mayores ganancias en las tareas de razonamiento. Los autores advierten explícitamente que la comparación se ve confundida por diferentes puntos de referencia y que la generalización fuera de dominio sigue siendo un problema serio [Chatzikyriakidis and Lappin 2026]. La interpretación equilibrada es que la federación es arquitectónicamente atractiva para la auditabilidad y la descomposición de tareas, pero aún no se ha demostrado empíricamente como una estrategia de escala universal.

El linaje de análisis semántico también aclara el papel de un lenguaje intermedio. Una representación debe ser lo suficientemente expresiva como para capturar el significado relevante para la tarea, pero lo suficientemente limitada como para compilar de manera confiable. Si está demasiado cerca del texto de la superficie, la ejecución permanece implícita. Si es demasiado general, la complejidad de la traducción y el solucionador se vuelven inmanejables. Beiser y Penz describen esto como un problema de lenguaje intermedio en los sistemas neuro-simbólicos contemporáneos: elegir un formalismo puede moverse en lugar de resolver la dificultad [Beiser and Penz 2025]. Una arquitectura de multiformalismo reduce la presión sobre cualquier idioma, pero introduce un nuevo requisito para el enrutamiento tipificado y la consistencia del formalismo cruzado.

Por lo tanto, un pivote semántico útil no debería pretender ser la lógica final. Debe preservar la evidencia y las distinciones neutrales de la tarea: entidades, menciones, eventos, relaciones, tiempo, modalidad, cantidades, mundos, procedencia, alternativas y cobertura, al tiempo que permite la reducción a ejecutores especializados. Algunas consultas se pueden responder directamente sobre el pivote utilizando operaciones relacionales. Otros requieren una fórmula temporal compilada, un sistema de restricción o una máquina de estado. El pivote es valioso

porque separa la comprensión del lenguaje de cualquier solucionador en particular mientras conserva suficiente información para la ejecución basada en la fuente.

El programa de investigación pre-LLM logró un éxito sustancial pero limitado en el dominio. Mostró que el lenguaje natural se puede compilar en programas ejecutables, que los intérpretes simbólicos mejoran la sistemáticamente, que los tipos y esquemas reducen los errores, y que la supervisión débil crea atajos semánticos. También demostró que la amplia cobertura y la semántica exacta están en tensión. LLMs mejora sustancialmente la cobertura y la generación de candidatos, pero heredan todos los problemas estructurales identificados por el análisis semántico. La síntesis correcta no es descartar la maquinaria anterior. Es colocar LLMs frente y alrededor de él, con interfaces formales que hacen que las viejas lecciones sean exigibles a nueva escala.

3. Interpretación abstracta y ejecución simbólica del lenguaje

3.1 Interpretación abstracta como una semántica de incertidumbre limitada

La interpretación abstracta se desarrolló como una teoría general para la computación de aproximaciones seguras del comportamiento del programa [Cousot and Cousot 1977]. Un programa puede tener infinitas entradas y estados, haciendo imposible la ejecución exhaustiva. En lugar de enumerar cada estado, un análisis representa conjuntos de estados a través de propiedades abstractas como intervalos, signos, posibles tipos de objetos, que alcanzan definiciones o relaciones de alias. Las operaciones abstractas están diseñadas para que cada comportamiento concreto relevante para la propiedad permanezca representado. El análisis puede incluir comportamientos imposibles y, por lo tanto, reportar falsas alarmas, pero una sobreaproximación de sonido no omite silenciosamente un comportamiento que la semántica concreta permite.

La idea operativa es sencilla. Una ejecución concreta rastrea los valores exactos. Un análisis de intervalos puede rastrear que una variable se encuentra entre cero y diez. Cuando el programa añade cinco, la operación de transferencia abstracta produce un intervalo entre cinco y quince. En una rama, los estados se refinan de acuerdo con la condición. En una fusión, la información se une. Para los bucles y los procesos recursivos, el análisis busca un punto fijo. El ensanchamiento puede forzar la convergencia sacrificando la precisión, mientras que el estrechamiento puede recuperar algunos detalles después. El marco matemático proporciona un vocabulario disciplinado para la precisión, la solidez, la convergencia y el refinamiento.

Aplicar el marco al lenguaje requiere una semántica concreta. El objeto que se aproxima no puede ser “significado” en un sentido indefinido. Para una tarea limitada, el dominio concreto puede ser el conjunto de estructuras semánticas admisibles bajo una gramática específica, ontología, política de puesta a tierra, revisión de origen y teoría de fondo. Cada estructura contiene posibles entidades, asignaciones de eventos, relaciones temporales, modalidades y enlaces de origen. La ambigüedad crea varias estructuras. La falta de información deja las dimensiones sin restricciones. Las fuentes en conflicto pueden generar mundos alternativos en lugar de una unión inconsistente.

Un estado semántico abstracto resume ese conjunto. Puede registrar que un pronombre puede referirse a cualquiera de las dos personas, que un evento ocurre dentro de un intervalo de tiempo, que una obligación está presente bajo una interpretación y ausente bajo otra, o que una cantidad se encuentra dentro de un rango. Un hecho obligatorio se mantiene en cada interpretación representada. Un hecho puede permanecer en al menos uno. Un hecho no puede sostenerse en ninguno. Desconocido indica que el sistema carece de la representación, cobertura o recursos computacionales necesarios para decidir. Esta distinción de cuatro vías es más informativa que una probabilidad sola y más honesta que forzar cada pregunta a ser verdadera o falsa.

Considere un documento de política que indique: “El operador envía el aviso después de la validación. Si el modo de emergencia está activo, el supervisor puede renunciar al período de

espera”. Supongamos que no está claro si “el período de espera” se refiere al período anterior a la validación o un período posterior. Un solo análisis hace una elección frágil. Un estado abstracto puede conservar dos modelos temporales. Si ambos implican que ese aviso debe seguir la validación, esa conclusión es imprescindible. Si solo uno permite el aviso de inmediato, el envío inmediato es mayor. Una regla de cumplimiento puede identificar qué veredicto depende de la frase ambigua y solicitar aclaración solo cuando sea necesario.

El principal beneficio no es que la interpretación abstracta entienda mágicamente el lenguaje. Es que da una semántica de ejecución de principios una vez que se ha construido un conjunto limitado de lecturas. Le dice al sistema cómo agregar alternativas sin confundir la posibilidad con la necesidad. También soporta dominios modulares. La identidad, el tiempo, la cantidad, la modalidad, la procedencia y el estatus normativo pueden tener su propia abstracción, y luego se combinan a través de productos reducidos que intercambian información cuando sea útil.

Un dominio de identidad puede representar clases de equivalencia candidata con relaciones explícitas iguales, diferentes y no resueltas. No debe fusionar entidades simplemente porque los nombres coinciden. Un dominio temporal puede usar órdenes parciales, intervalos o restricciones de diferencia. Un dominio de modalidad podría distinguir contenido afirmado, reportado, pretendido, permitido, requerido, prohibido, hipotético y contrafáctico. Un dominio de cantidad puede usar valores exactos, intervalos, unidades y compatibilidad dimensional. Un dominio de procedencia podría ordenar estados de evidencia como los extraídos, propuestos, asumidos, derivados, verificados y confirmados por humanos sin permitir que la confianza borre su origen. Un dominio de cobertura podría rastrear si una búsqueda está lo suficientemente completa como para respaldar las reclamaciones de ausencia.

Los dominios deben ser elegidos por la propiedad que se está comprobando. Un líter terminológico no necesita una rica red deontic. Una regla temporal no necesita una semántica léxica completa si los eventos y las fechas ya están materializados. Esta selección basada en tareas reduce la complejidad y refleja el análisis estático impulsado por la demanda. En un tiempo de ejecución de un documento largo, los circuitos o reglas declaran los tipos de observación y las dimensiones abstractas que requieren. El compilador materializa solo aquellas dimensiones y rechaza la ejecución cuando no se puede suministrar un dominio crítico.

Las funciones de transferencia son la parte más difícil. En el análisis ordinario del programa, la semántica de una asignación o operación aritmética se fija por el lenguaje de programación. En el análisis del lenguaje natural, la operación que actualiza el estado semántico puede ser propuesta por un LLM. Una oración podría introducir un evento, modificar un estado anterior, establecer una relación temporal, crear una excepción o informar la creencia de otra persona. Si un traductor aprendido se convierte directamente en la función de transferencia, se pierde la solidez clásica.

Un diseño defendible separa la generación de candidatos de la transferencia admitida. El LLM propone un conjunto finito de actualizaciones semánticas mecanografiadas con lapsos de origen y alternativas. Un materializador determinista valida que la evidencia citada existe, los campos satisfacen un esquema, unidades y tipos son compatibles, las referencias apuntan a objetos admisibles y no se actualiza silenciosamente ningún estado. El intérprete de resúmenes aplica funciones de transferencia registradas a los registros admitidos. El análisis resultante es sólido en relación con el conjunto de candidatos admitidos, no necesariamente en relación con

cada interpretación que un humano podría reconocer. Esta condicionalidad debe ser declarada explícitamente.

Hay varias maneras de fortalecer la suposición establecida por los candidatos. Múltiples indicaciones o modelos independientes pueden proponer alternativas. Los generadores basados en gramática o ontología pueden asegurar la cobertura de fragmentos controlados. Las variantes de parafrase pueden probar la estabilidad. Los ejemplos contrastantes pueden revelar interpretaciones faltantes. La revisión humana puede centrarse en las ambigüedades relevantes para los resultados. Un compilador puede sobregenerar deliberadamente dentro de una gramática limitada y dejar que las restricciones posteriores eliminen candidatos imposibles. Cada método cambia el costo de confianza de que el conjunto contiene todas las lecturas materialmente plausibles.

La interpretación abstracta también es relevante para la propagación de reglas a escala de documentos. Muchos problemas de linting y cumplimiento se asemejan al análisis de flujo de datos. Los hechos se generan en ubicaciones de origen, se propagan a través de alcances, se unen a través de secciones y se invalidan mediante declaraciones posteriores. Una definición puede estar disponible después de su declaración, pero no antes. Un objeto puede cambiar de estado a través de una narrativa. Un requisito puede aplicarse dentro de una jurisdicción o versión. Una reclamación puede adquirir pruebas más adelante. Un motor de punto fijo incremental puede calcular las consecuencias y actualizar solo las regiones afectadas cuando cambia el documento.

Esta perspectiva evita volver a ejecutar un general LLM en un documento completo después de cada edición. Los artefactos estructurales y semánticos estables se pueden almacenar en caché mediante el resumen de contenido. Un párrafo modificado invalida las observaciones locales y el resumen aguas abajo establece que dependen de ellas. El gráfico de análisis identifica qué circuitos necesitan reproducción. Esto es análogo a los marcos de compilación incremental y de análisis estático. También hace explicaciones precisas: el sistema puede mostrar qué cambio de fuente alteró qué resumen invariante y por qué.

La ampliación y la pérdida de precisión tienen análogos semánticos. Una larga narrativa puede contener cientos de posibles identidades de objetos o relaciones temporales. Retener cada combinación no es práctico. El sistema puede resumir “uno de estos objetos”, “algún tiempo dentro de este intervalo” o “una de estas jurisdicciones”. Tal ampliación puede introducir advertencias espurias. El estrecho o el refinamiento local pueden recuperar la precisión en torno a una presunta violación. El objetivo del diseño no es evitar la abstracción sino hacer visible y reversible su pérdida de precisión.

La literatura establecida NLP sobre robustez certificada utiliza ideas de interpretación abstracta de una manera diferente. Jia y sus colegas utilizaron la propagación de límites de intervalos para entrenar modelos cuyas predicciones están certificadas contra cada sustitución de palabras en una familia predefinida, reportando una precisión adversarial del 75 por ciento en IMDB y SNLI bajo su modelo de amenaza [Jia et al. 2019]. El trabajo posterior utilizó suavizado aleatorio, incrustaciones conscientes de la robustez y dominios de Transformer especializados para mejorar los límites certificados [Ye et al. 2020; Wang et al. 2023; Huang et al. 2026]. Estas son garantías genuinas, pero el dominio concreto es un conjunto de entradas de modelos perturbados, no un conjunto de interpretaciones semánticas. La familia de transformación se define externamente y se supone que preserva la etiqueta.

Esa distinción es importante. La certificación de que la predicción de un clasificador es invariante bajo todas las sustituciones en una lista de sinónimos no prueba que cada sustitución conserve el significado de la oración en contexto. No prueba que el clasificador esté en lo correcto en la oración original. Certifica la robustez en relación con un conjunto formal de perturbaciones. Esta literatura sigue siendo muy relevante porque demuestra cómo los dominios abstractos pueden escalar sobre las entradas combinatorias del lenguaje y cómo la precisión de verificación depende de la representación de las correlaciones. También proporciona una advertencia metodológica: la afirmación formal más fuerte es siempre relativa al conjunto concreto realmente definido.

Una dirección de investigación prometedora es conectar los dos usos. El análisis neuronal certificado podría unir el comportamiento del modelo de propuesta semántica bajo parafrases controladas, mientras que la interpretación abstracta semántica podría unir las consecuencias de las interpretaciones alternativas fundamentadas. Los primeros preguntan si la extracción de un modelo cambia bajo transformaciones de superficie permitidas. Este último pregunta si una conclusión a nivel de documento cambia a través de estructuras semánticas admisibles. Juntos podrían identificar si la incertidumbre se origina en la inestabilidad del modelo o la auténtica ambigüedad semántica.

LLMs también puede ayudar a construir intérpretes abstractos, pero solo bajo aceptación mecánica. SAIL demuestra este principio mediante el uso de LLMs para buscar transformadores abstractos mientras se hacen cumplir las restricciones sintácticas y semánticas y semantiza un costo que mide la falta de solidez. Los transformadores aceptados son de sonido global, y el sistema puede sintetizar transformadores precisos para operadores no lineales que no estaban disponibles anteriormente [Gu et al. 2026]. La lección más amplia es arquitectónica: los modelos generativos pueden explorar un inmenso espacio de diseño si un pequeño validador formal controla la admisión.

Pedir directamente a un LLM que realice una interpretación abstracta es menos confiable. Los experimentos en los puntos de referencia del análisis de programas muestran que los modelos pueden seguir estilos de interpretación abstracta pero cometer errores lógicos en estructuras complejas y estados alucinados [Mitchell et al. 2025]. Este resultado debería templar propuestas en las que un modelo narra una invariante y la narración es aceptada como análisis. El modelo es valioso para sugerir invariantes, particiones de dominio o predicados de refinamiento. Los cálculos de punto fijo y las comprobaciones de solidez deben seguir siendo simbólicos.

Para el lenguaje natural ejecutable, la interpretación abstracta no es, por lo tanto, ni una metáfora ni una solución completa. Es una teoría para la aproximación disciplinada después de que el sistema ha definido lo que cuenta como una posible interpretación. Soporta la semántica puede/debe, propagación incremental, composición de dominio, razonamiento de ausencia conservador y refinamiento dirigido. Su garantía está condicionada a la cobertura semántica. Esta limitación no es fatal; es el punto exacto en el que la interpretación asistida por modelo debe ser evaluada y gobernada.

3.2 Ejecución simbólica, resolución de restricciones, comprobación de modelos y refinamiento semántico

La ejecución simbólica clásica ejecuta un programa con entradas simbólicas en lugar de valores específicos [King 1976]. Un estado simbólico contiene una ubicación de control, expresiones simbólicas para variables de programa y una condición de ruta que describe qué entradas alcanzan ese estado. En una sucursal, la ejecución se bifurca. Un camino añade la condición de la rama; el otro añade su negación. Un solucionador de restricciones determina si las condiciones son satisfactibles y pueden producir entradas concretas que realizan una trayectoria factible. El método es poderoso porque produce testigos: no solo una advertencia de que puede existir un error, sino una entrada y un camino que lo demuestran.

La dificultad es la explosión del camino. Las ramas se multiplican, los bucles y la recursión crean caminos ilimitados, las teorías complejas desafían a los solucionadores y las bibliotecas externas son difíciles de modelar. Las encuestas de ejecución simbólica describen un gran espacio de diseño de estrategias de búsqueda, fusión de estado, resúmenes de composición, ejecución cóncola, modelos de entorno y optimizaciones de solucionadores [Baldoni et al. 2018]. Ningún motor explora todos los comportamientos del software real. Por lo tanto, las reivindicaciones de ejecución simbólica incluyen límites, suposiciones de modelización y limitaciones de cobertura.

El lenguaje natural se conecta con la ejecución simbólica de varias maneras distintas. El más simple es el lenguaje a código: un LLM genera un programa, y un ejecutor simbólico convencional analiza ese programa. Otro es el lenguaje a la especificación: el modelo genera una propiedad o restricción, que se compara con el código existente. Un tercero es la ejecución del programa semántico: un documento se compila en una máquina de estado o programa de reglas cuyas interpretaciones alternativas se convierten en elecciones simbólicas. Una cuarta es la exploración asistida por LLM: un modelo prioriza las rutas, propone resúmenes o razones sobre restricciones que los solucionadores convencionales no pueden expresar fácilmente.

La terminología debe seguir siendo precisa. La ejecución de SQL, Python o una forma lógica generada a partir de una pregunta es ejecución formal, pero no es necesariamente una ejecución simbólica en el sentido sensible a la ruta. La diferencia importa porque la ejecución simbólica ofrece condiciones de camino y testigos concretos, mientras que la ejecución ordinaria proporciona un resultado para un estado concreto. Ambos son útiles. Un estudio que los agrupa sin distinción hace que las garantías parezcan más fuertes de lo que son.

SymGPT proporciona uno de los ejemplos más claros de lenguaje a especificación. El sistema estudia 132 reglas de lenguaje natural de tres estándares ERC, los traduce con un LLM en una gramática restringida, sintetiza restricciones que describen violaciones y utiliza la ejecución simbólica para buscar contratos de Ethereum. Los autores reportan 5.783 violaciones en 4.000 contratos, incluidos 1.375 con rutas de ataque explícitas por robo, e informan un desempeño más sólido que seis técnicas automatizadas y un servicio de auditoría de expertos en seguridad [Xia et al. 2026]. La fuerza del diseño radica en la gramática estrecha, la síntesis sistemática de restricciones y los caminos de programas concretos. Su riesgo semántico restante radica en si cada regla ERC se tradujo fielmente.

Este patrón se generaliza. Una política se puede compilar en condiciones previas, transiciones prohibidas y excepciones. Un procedimiento puede ser compilado en estados y acciones. Una regla de continuidad narrativa puede modelar objetos y eventos, luego buscar un

camino en el que se use un objeto después de quedarse atrás sin recuperarlo. Un protocolo clínico puede modelar secuencias de elegibilidad e intervención. Un flujo de trabajo científico puede modelar las transformaciones de datos y los requisitos de procedencia. En cada caso, el ejecutor simbólico debe operar en un programa mecanografiado en lugar de en la prosa cruda.

La ejecución simbólica es especialmente útil para los contraejemplos. El análisis abstracto puede indicar que una violación es posible porque ha perdido una correlación. Una búsqueda de camino puede intentar elegir identidades, tiempos, condiciones y excepciones concretas que se den cuenta de la violación. Si el camino es satisfactorio, el testigo puede ser renderizado como un escenario basado en la fuente. Si es insatisfecho, el sistema aprende que la advertencia abstracta era espuria bajo la semántica actual.

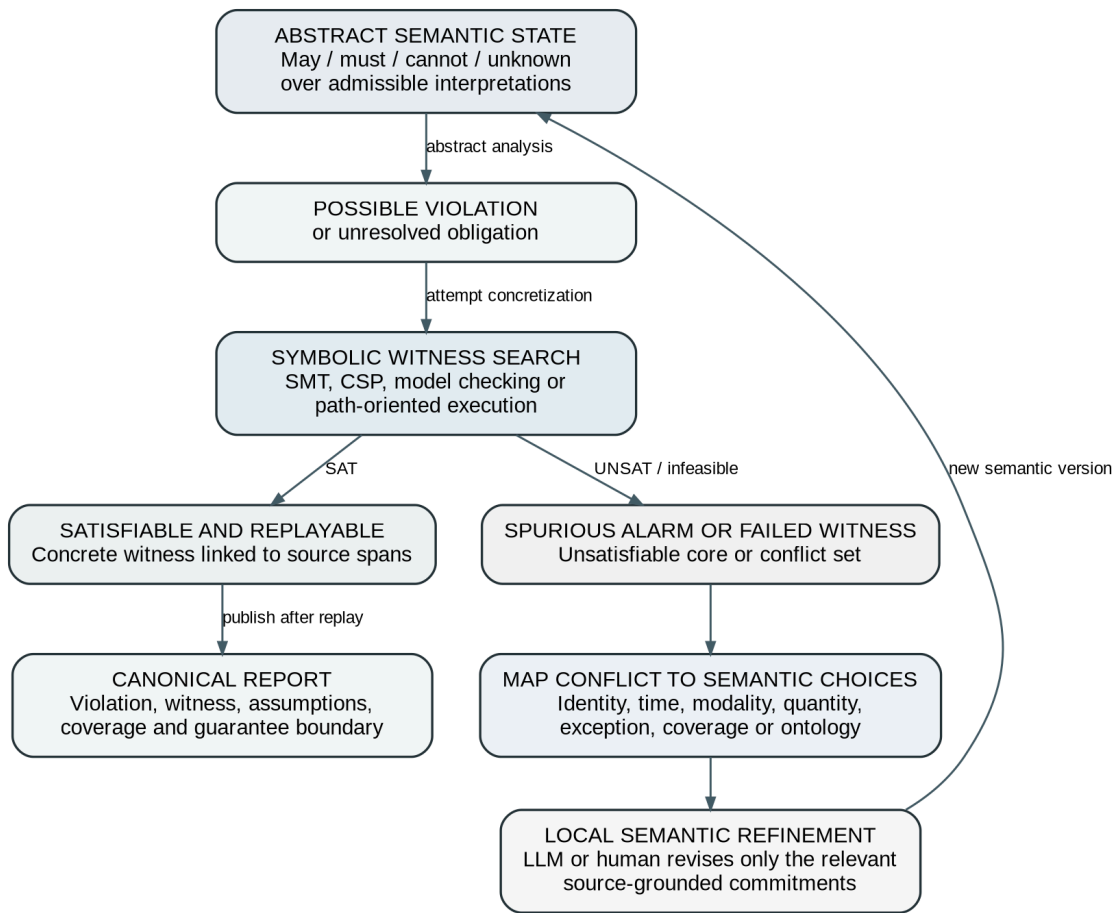


Figura 2. Refinamiento semántico guiado por contraejemplo. El análisis abstracto identifica una posible violación; la búsqueda de testigos simbólicos confirma un caso rejugable o expone las distinciones semánticas que deben ser refinadas.

Esto conduce a un refinamiento semántico guiado por contraejemplo. La CEGAR tradicional comienza con una abstracción del programa grueso. La comprobación de modelos encuentra un contraejemplo abstracto. Una comprobación de viabilidad determina si el contraejemplo corresponde a una ejecución concreta. Si no, se añaden predicados para refinar la abstracción, y el análisis se repite. En el entorno del idioma, la abstracción incluye opciones semánticas. Un contraejemplo espurio puede surgir porque se fusionaron dos menciones, se ignió una excepción, se amplió un intervalo temporal o se trató una declaración modal como un hecho.

Un bucle de refinamiento semántico debe preservar la explicación simbólica del fracaso. Supongamos que una advertencia de cumplimiento depende de la suposición de que “la junta” en dos secciones denota el mismo cuerpo. La repetición simbólica muestra que el camino se

vuelve inviable si las entidades son distintas. El sistema mapea el predicado de identidad a las menciones relevantes y hace una pregunta enfocada: ¿se pretende que estas referencias denoten la misma junta? Un LLM puede reexaminar el contexto local y proponer alternativas; un humano puede adjudicar si es necesario. El resto del modelo de documento permanece sin cambios. Esto es más confiable y eficiente que pedirle a un modelo que vuelva a leer todo y “volver a intentarlo”.

Los núcleos insatisfechos proporcionan un mecanismo relacionado. Cuando un conjunto de restricciones es inconsistente, un solucionador a menudo puede devolver un subconjunto responsable del conflicto. El mapeo del núcleo de vuelta a los intervalos de origen identifica el conjunto mínimo de declaraciones e interpretaciones que no pueden coexistir. En una revisión de políticas, esto puede revelar obligaciones contradictorias. En el análisis sintáctico semántico, puede revelar que un cuantificador, un vínculo de identidad y una suposición temporal crean conjuntamente imposibilidad. El usuario recibe una explicación basada en el conflicto formal en lugar de una crítica de modelo genérico.

No todos los problemas se representan mejor como ejecución de ruta. La comprobación de modelos analiza los sistemas de transición contra las propiedades temporales. La lógica temporal lineal puede expresar que algo eventualmente sucede, siempre se sostiene o ocurre antes de otro evento. La computación-árbol y las lógicas probabilísticas expresan una ramificación o un comportamiento incierto. Los sistemas de lenguaje natural a lógico temporal como NL2TL y nl2spec muestran que LLMs puede construir especificaciones temporales cuando el vocabulario y el dominio están controlados [Chen et al. 2023; Cosler et al. 2023]. El ejecutor es entonces un corrector de modelo o planificador, no un LLM.

La resolución de restricciones cubre un amplio término medio. SAT representa opciones booleanas. SMT añade teorías como aritmética, matrices, cadenas y funciones no interpretadas. CSP y CP-SAT manejan dominios finitos, programación y optimización. MaxSAT y las restricciones suaves expresan preferencias. Muchas tareas en lenguaje natural, programación, asignación de recursos, verificación de la coherencia, relaciones familiares, rompecabezas y combinaciones de políticas, se compilan mejor en restricciones que en el código de procedimiento. SatLM mostró que las especificaciones declarativas más un solucionador pueden superar a los enfoques asistidos por programas en tareas aritméticas y lógicas [Ye et al. 2023]. El resultado admite una opción arquitectónica central: el LLM debe describir el problema, mientras que el solucionador realiza la búsqueda.

La planificación formal proporciona semántica de acción y búsqueda dirigida a objetivos. Trabajos recientes posicionan a LLMs como formalizadores de planificación en lugar de planificadores. Hao y sus colegas traducen las limitaciones de planificación del mundo real en herramientas de verificación formal, utilizan núcleos insatisfiables para la reparación e informan un éxito del 93,9 por ciento en TravelPlannerrrr en su entorno, en comparación con las líneas de base de modelos directos mucho más bajas [Hao et al. 2025]. Un estudio de 2025 concluye que la principal oportunidad no es reemplazar a los planificadores, sino utilizar LLMs para construir y refinar modelos de planificación [Tantakoun et al. 2025]. Esta es precisamente la división defendida aquí.

Los motores Dataflow y Datalog proporcionan la ejecución de puntos fijos en lugar de la búsqueda de ruta. Son útiles para las consecuencias monótonas y la propagación en todo el documento. Los motores de autómatas y patrones manejan secuencias de eventos ordenadas. Las tablas de decisión manejan políticas finitas. La arquitectura debe encaminar cada problema

hacia el formalismo más simple que lo expresa. La ejecución simbólica no debe convertirse en una etiqueta universal para cada componente simbólico.

LLMs puede ayudar a la ejecución simbólica tradicional de manera más segura y arriesgada. Un papel más seguro es la priorización de la trayectoria. Un modelo puede identificar regiones de código que probablemente contengan vulnerabilidades o rutas que probablemente alcancen un objetivo, mientras que KLEE u otro motor conserva la decisión de viabilidad final. Trabajos recientes como KLEECopilot reportan mejoras sustanciales en la cobertura de la búsqueda guiada por LLM, aunque el estado de la preimpresión y la sensibilidad del modelo requieren una interpretación cautelosa [Chen et al. 2026]. El principio es sólido: la heurística aprendida decide dónde buscar, no si un camino es válido.

Un papel más arriesgado es reemplazar al solucionador. AutoBug, presentado por sus autores como una ejecución simbólica impulsada por un modelo de lenguaje grande, descompone el código en tareas de razonamiento basadas en rutas y pide a un modelo que razone directamente sobre ellas sin traducir todo en SMT. El enfoque es ligero y agnóstico del lenguaje, y los experimentos informados mejoran el análisis de programas basado en LLM [Li et al. 2025]. Los autores lo caracterizan como aproximado. Puede analizar construcciones que son difíciles para los solucionadores convencionales, pero no proporciona solidez clásica. Para un uso de alta seguridad, un modelo de este tipo se trata mejor como un motor en una cartera cuyos resultados requieren pruebas, repetición o confirmación independiente.

LLMs también puede generar pruebas, resúmenes o modelos que satisfacen las rutas para API externas. Los estudios informan una cobertura útil sobre los programas que las herramientas tradicionales luchan por manejar, pero el rendimiento disminuye con rutas largas, API complejas, recursión y clasificación de caminos inviables. Estos métodos son prometedores porque los entornos de software contienen semántica que son costosas de axiomatizar. Su estado de resultado debe permanecer “candidato” hasta que la ejecución concreta confirme el camino o un solucionador demuestre la condición.

La seguridad es una preocupación importante. Las reglas del lenguaje natural y los documentos fuente son insumos no confiables. Un documento malicioso puede intentar la inyección rápida, fabricar referencias o explotar un traductor generador de código. Los artefactos formales deben ser datos, no código ejecutable sin restricciones. Cuando se requieren algoritmos personalizados, se deben revisar las extensiones de host con digests bloqueados, entradas y salidas escritas, límites de recursos y ninguna autoridad ambiental. Las invocaciones de solver necesitan presupuestos de tiempo y memoria. Un testigo debe incluir la versión exacta de la teoría, el compilador y el motor que la produjo.

El patrón de análisis combinado es ahora claro. Un análisis semántico y de flujo de datos grueso identifica posibles problemas en un documento largo. Los motores formales especializados prueban solo las rebanadas relevantes. La ejecución simbólica construye testigos de reclamos dependientes de la ruta. La comprobación del modelo maneja los comportamientos temporales. Los solucionadores de restricciones manejan opciones combinatorias. Los testigos fallidos y los núcleos de los solucionadores impulsan el refinamiento semántico local. Cada conclusión aceptada se devuelve con un estatus y procedencia que refleja el eslabón más débil de la cadena.

Esta arquitectura no promete la verificación total del lenguaje sin restricciones. Promete algo más operativo: cada reclamo formal tiene un significado ejecutable; cada símbolo tiene una fuente o suposición declarada; cada contraejemplo se puede reproducir; cada reclamo de

ausencia está vinculado a la cobertura; y cada ambigüedad restante se representa en lugar de oculta. La interpretación abstracta proporciona escala y agregación conservadora. La ejecución simbólica proporciona precisión y testigos. Su combinación es una de las bases más plausibles para el lenguaje natural ejecutable.

4. Lo que la era LLM ha logrado realmente

4.1 Autoformalización, razonamiento asistido por programas, lógica, especificaciones temporales y planificación

La llegada de grandes modelos de lenguaje cambió el límite práctico de la compilación semántica. Los sistemas anteriores requerían gramáticas específicas de tareas, formas lógicas supervisadas o léxicos cuidadosamente diseñados. LLMs puede proponer artefactos formales a partir de descripciones abiertas con poca o ninguna capacitación específica para la tarea. Esto amplía sustancialmente la cobertura, pero también crea la tentación de inferir más de las ganancias de referencia de lo que los mecanismos justifican. La pregunta relevante no es simplemente si un método mejora la precisión de la respuesta. Es a lo que se han transferido los errores a un componente simbólico, qué errores permanecen en la formalización, y qué evidencia independiente existe de que el artefacto formal representa la fuente.

Los modelos de lenguaje con ayuda de programa, o PAL, proporcionan un punto de partida limpio. El LLM lee un problema, genera un programa Python y delega el cálculo al intérprete [Gao et al. 2022]. En GSM8K, el sistema informado mejoró sustancialmente con respecto a la indicación de cadena de pensamiento. El mecanismo es convincente porque los modelos de lenguaje a menudo son mejores para descomponer un problema que para llevar a cabo la aritmética larga exactamente. Una vez que la descomposición es correcta, Python suministra operaciones deterministas y estado.

La limitación de PAL es igualmente instructiva. El intérprete certifica únicamente que ejecutó el programa generado. No certifica que el programa modele fielmente la cuestión del lenguaje natural. Una variable equivocada, una condición omitida o una fórmula incorrecta pueden ejecutarse sin error. Por lo tanto, el éxito del sistema depende de una distribución en la que los programas son cortos, los ejemplos limitan fuertemente la estructura y los errores de ejecución revelan muchos errores. Para el razonamiento documental, el análisis de políticas y la interpretación científica, el mapeo es menos limitado y el intérprete proporciona una retroalimentación más débil.

SatLM reemplaza el código de procedimiento con restricciones declarativas [Ye et al. 2023]. El modelo establece variables y relaciones, y un motor de satisfacción busca una solución. Esto puede ser más fácil que generar un algoritmo porque el modelo no necesita decidir el orden de las operaciones. Las representaciones declarativas también permiten múltiples modelos, pruebas de inviabilidad y depuración a nivel de restricción. Los avances reportados en las tareas de razonamiento duro respaldan la idea de que la elección del solucionador debe seguir la estructura del problema. Un programa es naturalmente un problema de restricción; un proceso dinámico puede ser una máquina de estado; un problema de palabra aritmética puede ser un programa o restricciones.

Logic-LM formaliza problemas de lógica de lenguaje natural e invoca a los solucionadores simbólicos [Pan et al. 2023]. También utiliza errores de solucionador para solicitar el auto-refinamiento. En varios conjuntos de datos, los autores informan grandes mejoras promedio sobre el estándar y la indicación de la cadena de pensamiento. La arquitectura demuestra dos principios útiles. En primer lugar, el motor simbólico elimina la inconsistencia deductiva una vez que existe una formalización válida. En segundo lugar, los mensajes de error

formales proporcionan una mejor señal de reparación que una solicitud genérica para reconsiderar.

Sin embargo, la reparación de la sintaxis no es una reparación semántica. Una fórmula puede estar bien formada, satisfecha y equivocada. La tubería de Logic-LM puede corregir predicados malformados o fallas de solucionadores, pero puede no detectar que “algunos” se convirtieron en “todos” o que se omitió una excepción. El trabajo posterior como Logic-LM ++ agrega refinamiento de varios pasos, y LTRAG recupera ejemplos de formalización guiados por el pensamiento, reportando mejoras sobre Logic-LM y LINC en FOLIO y AR-LSAT [Kirtania et al. 2024; Hu et al. 2025]. Estas ganancias sugieren que la formalización en sí misma se beneficia de ejemplos y crítica estructurada. No eliminan la necesidad de evaluar directamente la fidelidad semántica.

LINC hace explícita la modularidad: un LLM sirve como un analizador semántico, la lógica de primer orden representa el problema, y un probador de teorema realiza la deducción [Olausson et al. 2023]. En ProofWriter, la combinación reportada produjo grandes ganancias sobre los sistemas de cadena de pensamiento, incluidos los modelos sustancialmente más grandes que el analizador. El análisis de errores mostró modos de falla complementarios: el sistema simbólico evita muchos errores de razonamiento, mientras que los fallos de formalización permanecen. Esta es exactamente la evidencia que se espera de una arquitectura federada. El sistema es más fuerte cuando la tarea del lenguaje natural ya corresponde a una teoría compacta de primer orden.

FOLIO proporciona un punto de referencia útil porque contiene problemas de razonamiento de lenguaje natural de autor humano emparejados con lógica de primer orden verificada [Han et al. 2024]. Evalúa tanto el razonamiento como la traducción NL-to-FOL. El conjunto de datos sigue siendo un desafío incluso para los modelos fuertes, lo que socava la idea de que la formalización de primer orden es un paso de preprocesamiento resuelto. FOLIO también ilustra la importancia de la complejidad de las personas. Los conjuntos de datos de reglas sintéticas pueden hacer que el lenguaje sea inusualmente regular y permitir que los modelos exploten plantillas. Los ejemplos humanos introducen variaciones léxicas, conocimiento del mundo y opciones de alcance más realistas.

FoVer adopta otro enfoque al verificar el razonamiento del lenguaje natural a través de la lógica [Pei et al. 2025] de primer orden. En lugar de tratar el texto de razonamiento fluido como autoautentificante, construye obligaciones formales y las verifica. Este es un patrón prometedor para la generación restringida a pruebas: un LLM puede proponer un argumento, pero un verificador acepta solo pasos inferenciales que corresponden a consecuencias formales válidas. La cuestión restante es la alineación entre cada oración y los predicados formales utilizados para comprobarlo.

La investigación de autoformalización trata cada vez más la traducción como una disciplina distinta en lugar de un truco rápido. Las encuestas publicadas en 2025 catalogan la traducción en probadores de teoremas, lenguajes lógicos y especificaciones formales, enfatizando la necesidad de bucles de retroalimentación, recuperación, representaciones intermedias y puntos de referencia que separan la sintaxis de la semántica [Weng et al. 2025]. El trabajo directo NL-to-FOL reporta un progreso significativo, pero también errores sistemáticos en el conocimiento implícito, el alcance del cuantificador y el diseño onológico [Yang et al. 2024; Lalwani et al. 2025]. Agregar conocimientos de fondo implícitos puede mejorar el rendimiento

de las tareas al tiempo que aumenta el riesgo de que el sistema formal demuestre una premisa no autorizada por la fuente.

El tratamiento del conocimiento de fondo es una elección arquitectónica decisiva. En un punto de referencia de rompecabezas, las premisas de sentido común pueden ser implícitamente esperadas. En la verificación legal o científica, la introducción de locales no declarados puede invalidar el resultado. Un sistema riguroso debe distinguir las premisas de origen, las premisas de la autoridad, los axiomas de dominio y los supuestos sugeridos por el modelo. El solucionador puede razonar sobre todos ellos, pero la explicación final debe mostrar qué conclusiones dependen de qué autoridad. Si una suposición es esencial y no verificada, el resultado debe ser condicional.

La traducción temporal-lógica trae diferentes retos. NL2TL utiliza representaciones elevadas para aprender la estructura temporal a través de los dominios e informa de una fuerte adaptación con relativamente pocos datos de dominio objetivo [Chen et al. 2023]. La estrategia de levantamiento esconde proposiciones atómicas concretas mientras aprende patrones lógicos reutilizables. Esta es una valiosa visión del diseño: separar la estructura de “siempre antes”, “eventualmente después” o “hasta” de la base de eventos específica del dominio.

nl2spec enfatiza la interacción [Cosler et al. 2023]. El sistema mapea partes del requisito del lenguaje natural a partes de la fórmula temporal generada, lo que permite a los usuarios reparar las traducciones erróneas locales. Esta interfaz reconoce que la ambigüedad no siempre se puede resolver automáticamente. También sugiere cómo debería funcionar el refinamiento semántico basado en LLM: el usuario debería ver la decisión lingüística, no simplemente una cadena de operadores temporales.

La conexión a tierra sigue siendo el punto frágil. Una fórmula temporal puede tener la forma correcta mientras que sus proposiciones atómicas se refieren a los eventos u objetos equivocados. El trabajo sobre la equivalencia lógica basada informa que los mecanismos de base explícitos mejoran la corrección, reforzando la distinción entre la sintaxis de la fórmula y el apego semántico. Para un tiempo de ejecución de lenguaje ejecutable, los átomos temporales deben ser referencias a observaciones mecanografiadas con tramos e identidades, no etiquetas de forma libre inventadas por el traductor.

La planificación automatizada proporciona un backend simbólico particularmente maduro. Los lenguajes de planificación definen tipos, predicados, acciones, condiciones previas, efectos, estados iniciales y objetivos. Los planificadores buscan una secuencia de acción válida. LLMs son pobres planificadores de horizonte largo cuando deben mantener el estado y las restricciones internamente, pero pueden ser eficaces para formalizar un problema para un planificador. Un estudio de 2025 identifica la construcción y el refinamiento modelo como el papel central de LLMs en esta área [Tantakoun et al. 2025].

Hao y sus colegas demuestran una versión rigurosa del patrón en la planificación del mundo real [Hao et al. 2025]. El sistema formaliza las restricciones, invoca herramientas de verificación y utiliza núcleos insatisfechos para reparar problemas. Los autores reportan un éxito del 93,9 por ciento en TravelPlannerrrr en su entorno experimental, muy por encima de las líneas de base directas de LLM que comparan. El resultado es importante porque muestra que las herramientas formales pueden transformar la fiabilidad de la planificación cuando el problema es representable. No significa que el LLM se haya convertido en un planificador de sonido. Significa que el planificador y el verificador llevan el estado de horizonte largo mientras que el LLM construye y repara la instancia formal.

La planificación también expone las restricciones ocultas. Las solicitudes de lenguaje natural incluyen expectativas de sentido común, preferencias y condiciones de viabilidad que pueden no ser establecidas como requisitos formales. Sistemas como LRPlan intentan la colaboración entre el lenguaje y los modelos de razonamiento para restricciones explícitas e implícitas, mientras que puntos de referencia como PlanningArena muestran que incluso los modelos fuertes luchan con la selección de herramientas, la memoria contextual y la consistencia lógica [Karthikeyan et al. 2025; Zheng et al. 2025]. Un sistema de alta seguridad debe formalizar las restricciones implícitas de una fuente confiable o etiquetarlas como suposiciones. El “commonsense” no es una categoría de procedencia suficiente.

RuleArena y los puntos de referencia relacionados prueban LLMs contra sistemas de reglas realistas y encuentran que el aumento de herramientas ayuda, pero no hace que la formalización sea trivial. Las tareas de SemEval 2026 en el contenido y el razonamiento formal muestran además que los modelos están influenciados por el contenido semántico, incluso cuando la forma lógica debe determinar la respuesta. Las entradas neurosimbólicas modulares utilizan FOL y Z3 para reducir el sesgo de contenido, pero la traducción formal y el ruido multilingüe siguen siendo difíciles [Grimaldi 2026; Kartáč et al. 2026; Wang et al. 2026]. Estos resultados refuerzan la necesidad de desenredar la interpretación lingüística de la inferencia formal.

Una de las observaciones más consecuentes a lo largo de esta literatura es que los sistemas exitosos utilizan lenguajes intermedios específicos de destino. PAL utiliza Python. SatLM utiliza restricciones declarativas. Logic-LM y LINC utilizan la lógica. NL2TL utiliza la lógica temporal. Los sistemas de planificación utilizan modelos tipo PDDL. SymGPT utiliza una gramática adaptada a las reglas de ERC. El campo no ha descubierto una representación formal universal para el lenguaje natural. Se ha descubierto que LLMs puede enrutar el lenguaje a múltiples ecosistemas formales cuando hay ejemplos, esquemas y comentarios disponibles.

Esto soporta una arquitectura de cartera en lugar de un “bytecode de lenguaje” universal. Un pivote semántico puede preservar la evidencia, entidades, eventos, alcance y modalidad, mientras que un selector de formalismo elige el objetivo de ejecución. El selector en sí puede ser asistido por un LLM, pero su decisión debe validarse contra los contratos de capacidad. Un formalismo es compatible solo si puede expresar las distinciones requeridas y existen las observaciones fundamentadas necesarias. Un planificador no debe ser seleccionado para una consulta puramente taxonómica; la lógica de primer orden no debe aproximarse silenciosamente a la dinámica temporal incierta; un simple motor relacional no debe pretender decidir excepciones legales desviables.

Por lo tanto, la evidencia empírica apoya una afirmación más estrecha pero sustancial. LLMs puede mejorar drásticamente la construcción de objetos formales ejecutables. Los sistemas simbólicos externos pueden mejorar la corrección, la consistencia y la interpretabilidad. La retroalimentación del solucionador puede guiar la reparación. Los resultados más fuertes aparecen en dominios limitados con lenguajes explícitos, esquemas claros y respuestas evaluables. El desafío no resuelto es la compilación semántica: garantizar que el objeto formal preserve el significado relevante de la fuente y que cada premisa esté fundamentada y autorizada.

4.2 Análisis de programas, exploración simbólica, transformadores abstractos y LLM certificados

La interacción entre LLMs y el análisis de programas se ejecuta en ambas direcciones. LLMs puede ayudar a las herramientas formales a manejar especificaciones, patrones de código y entornos que son difíciles de modelar. Las herramientas formales pueden verificar el código generado por LLMs, verificar las redes neuronales y restringir los componentes aprendidos. Esta literatura es especialmente valiosa para el lenguaje natural ejecutable porque pone a prueba la misma cuestión arquitectónica bajo una semántica más clara: ¿dónde puede contribuir un modelo probabilístico sin convertirse en el oráculo final?

El análisis estático tradicional depende de abstracciones diseñadas a mano, funciones de transferencia, resúmenes y modelos de entorno. La ejecución simbólica depende de la búsqueda de ruta y de las codificaciones de los solucionadores. Estas herramientas ofrecen garantías de principios, pero luchan con características de lenguaje dinámico, código de biblioteca, manipulación de cadenas complejas, servicios externos y escalabilidad. LLMs puede reconocer la intención del código, proponer invariantes, resumir funciones, identificar ubicaciones de errores probables y generar pruebas. Un estudio de 2025 sobre el análisis de programas asistidos por LLM categoriza el trabajo en métodos estáticos, dinámicos e híbridos y enfatiza tanto la oportunidad como los riesgos de los resultados probabilísticos [Wang et al. 2025].

Un papel arquitectónico es la guía heurística. Un ejecutor simbólico puede retener su semántica y usar un LLM para clasificar rutas, identificar regiones vulnerables o seleccionar límites de bucle. Si la heurística está mal, la cobertura puede sufrir, pero los caminos aceptados permanecen revisados por el solucionador. Esto es análogo al uso de un modelo neuronal para ordenar la búsqueda en la prueba de teoremas. La distinción entre orientación y aceptación es fundamental. Una puntuación aprendida puede determinar dónde se gastan los recursos; no debe determinar si una condición de camino es satisfactoria.

KLEECopilot representa esta dirección. La preimpresión 2026 utiliza la guía LLM para dirigir KLEE hacia vulnerabilidades probables e introduce heurísticas para el escape de bucles, reportando mejoras sustanciales en la cobertura de bloques básicos y líneas y docenas de violaciones únicas [Chen et al. 2026]. Como una preimpresión reciente, los resultados requieren una replicación independiente y sensibilidad a los asuntos familiares modelo. Sin embargo, el patrón de diseño es plausible: LLMs contribuye con la priorización semántica de la que carece la búsqueda clásica, mientras que KLEE conserva la semántica de ejecución.

Otro papel es la generación de pruebas que satisfacen el camino. Un modelo puede leer código y proponer entradas que se espera que atravesasen una ruta de destino. La ejecución concreta valida entonces si se alcanzó la ruta. Este enfoque puede manejar las convenciones API o los formatos de datos que son onerosos para SMT. Los estudios reportan tasas de éxito útiles, pero también fracasos en caminos largos, recursión, ramas inviables y bibliotecas complejas. La interpretación segura es que el LLM es un generador de pruebas. Un recorrido concreto, un rastro de instrumentación o un solucionador sigue siendo el validador.

AutoBug adopta un enfoque más radical al usar LLMs para realizar el razonamiento descompuesto de ruta directamente [Li et al. 2025]. En lugar de traducir todas las restricciones en una teoría de solucionadores, presenta condiciones de ruta a nivel de código al modelo. Esto puede ser agnóstico del lenguaje y puede analizar la semántica que es difícil de codificar.

Los autores informan mejoras en varios puntos de referencia de análisis de programas, particularmente para modelos más pequeños. También caracterizan el análisis como aproximado. El sistema no ofrece la solidez clásica, y sus conclusiones pueden variar con el modelo.

AutoBug es valioso porque muestra dónde los métodos simbólicos fallan operativamente. Muchos programas reales llaman bibliotecas opacas, manipulan objetos complejos o dependen de contratos informales. Un modelo de lenguaje puede inferir un comportamiento probable de los nombres, comentarios y patrones de código aprendidos. Pero esta flexibilidad es la misma fuente de incertidumbre. En una cartera orientada a la garantía, el análisis aproximado de LLM debe generar candidatos, resúmenes o comprobaciones priorizadas. Los resultados deben ser confirmados por pruebas, contratos, monitoreo de tiempo de ejecución o un motor formal siempre que se haga una reclamación de seguridad.

LLMs puede proponer invariantes para la interpretación abstracta. Un candidato invariante puede acelerar la prueba si un comprobador verifica que se mantiene inicialmente, se conserva por transiciones e implica la propiedad. Esta es una poderosa división del trabajo: el descubrimiento invariante es creativo y combinatorio, mientras que la comprobación invariante es mecánica. Patrones similares aparecen en la prueba de teoremas y la síntesis de programas. El modelo de lenguaje busca; el verificador acepta.

SAIL extiende este principio a la síntesis de los transformadores abstractos [Gu et al. 2026]. Construir un transformador que sea a la vez sólido y preciso para un operador no lineal es difícil. SAIL pide a LLMs que genere candidatos, pero impone restricciones sintácticas y semánticas duras y utiliza un costo matemáticamente fundamentado para la falta de solidez. El sistema reportado sintetiza transformadores de sonido a nivel mundial en varios dominios, combina diseños manuales y encuentra transformadores precisos que no anteriormente en la literatura. La importancia va más allá de la verificación de la red neuronal. Es evidencia de que LLMs puede automatizar la construcción de métodos formales cuando cada artefacto aceptado está sujeto a obligaciones de prueba independientes.

El contraste con pedir a un LLM que “actúe como un intérprete abstracto” es agudo. Mitchell y sus colegas motivaron modelos a la razón a través de la interpretación abstracta en los programas SV-COMP [Mitchell et al. 2025]. Los modelos podrían seguir estilos de composición o de punto fijo en algunos casos, pero cometieron errores lógicos y alucinaron en estructuras complejas. El resultado es consistente con un patrón general: los modelos pueden imitar los procesos formales y generar candidatos útiles, pero la confiabilidad disminuye cuando el proceso requiere un estado exacto persistente. Un verdadero intérprete abstracto debe ejecutar las operaciones de la red; el modelo puede sugerir dominios, predicados o explicaciones.

La verificación formal de los modelos neuronales NLP es otra rama importante. El trabajo de robustez certificado pregunta si la predicción de un modelo permanece sin cambios en todas las entradas en un conjunto de perturbaciones definido. Jia y sus colegas usaron la propagación unida a intervalos para sustituciones de palabras e informaron una precisión adversaria del 75 por ciento en IMDB y SNLI bajo las familias de sustitución construidas [Jia et al. 2019]. SAFER utilizó suavizado aleatorio y podría aplicarse a los modelos de caja negra, incluyendo BERT [Ye et al. 2020]. Las incrustaciones conscientes de la robustez reforzaron los límites de certificación, mejorando la precisión robusta certificada en IMDB de 76.73 a 84.78 por ciento en el entorno reportado [Wang et al. 2023].

La verificación del transformador es difícil porque la autoatención contiene productos, operaciones softmax y correlaciones entre posiciones. Los dominios especializados mejoran los límites. La interpretación abstracta parametrizada para la verificación de transformador introdujo dominios afines parametrizados para productos internos y reportó límites más estrictos y verificación de instancias que los métodos anteriores no podían probar [Huang et al. 2026]. Esta línea de trabajo demuestra que el análisis formal puede escalar a arquitecturas no triviales NLP cuando la propiedad y el conjunto de entrada se definen con precisión.

El alcance de la garantía no debe ser borroso. Un clasificador certificado puede ser robusto a cada sustitución extraída de un conjunto específico y aún así estar equivocado. Una lista de sustitución puede incluir “sinónimos” o omitir parafrases significativas de manera contextualmente inapropiadas. La certificación dice que el modelo es invariante sobre la familia de perturbaciones formal. No establece la equivalencia semántica en el lenguaje humano. Esta distinción debe ser explícita en cualquier sistema de lenguaje ejecutable que utilice modelos de extracción certificados.

Una integración útil adjuntaría un certificado de robustez a una observación semántica. Supongamos que un extractor de eventos identifica que una sentencia establece una obligación. Un modelo certificado podría mostrar que esta clasificación es estable bajo una familia controlada de sustituciones léxicas. La observación todavía se propone un modelo, pero tiene una propiedad de estabilidad adicional. El análisis simbólico descendente puede registrar esta evidencia sin actualizar la observación a los hechos. Tal estado en capas es más informativo que una puntuación de confianza.

Los métodos formales también verifican el código generado por LLMs. La generación de pruebas, el análisis estático, la comprobación de tipos y los asistentes de prueba pueden detectar muchos errores. Los sistemas de síntesis de programas utilizan cada vez más la retroalimentación del compilador y la prueba para la reparación iterativa. El mismo patrón debería gobernar los circuitos semánticos generados y el código de formalización. Un agente codificante puede crear una implementación de regla, pero la publicación debe requerir compilación, pruebas de mutación, casos de referencia de lenguaje natural, repetición y verificadores independientes. No se debe permitir que un LLM cree un comportamiento ejecutable oculto dentro de un artefacto de teoría declarativa.

La literatura sobre modelado de protocolos del lenguaje natural ofrece un resultado aleccionador. El trabajo presentado en ICSE 2025 investigó la generación de modelos de Tamarin o ProVerif a partir de la documentación del protocolo. El uso ingenuo de LLM fue ineficaz; la tubería exitosa requirió descomposición semántica, desambiguación humana limitada y transformaciones intermedias de sonido. Generó modelos correctos a escala moderada para 10 de los 18 protocolos reales [Mao et al. 2025]. El resultado es significativo porque los protocolos contienen roles, mensajes, suposiciones criptográficas, estado y orden temporal. También muestra que incluso una sólida documentación estructurada no se compila automáticamente en modelos de seguridad formales.

Los casos de fracaso son instructivos. La documentación omite los supuestos que los expertos infieren. El mismo término puede indicar un tipo de mensaje, un estado o una instancia. Las propiedades de seguridad se pueden indicar de manera informal o distribuidas en todas las secciones. Un modelo de protocolo formal debe hacer explícitas las capacidades adversarias. Estos son análogos directos del razonamiento documental. La solución no fue un

aviso más grande, sino una arquitectura que separa el análisis semántico, la desambiguación y la transformación verificada.

Los exámenes externos del campo más amplio convergen en una conclusión similar. La revisión neurosimbólica 2026 NLP favorece una investigación más sólida de las arquitecturas federativas, pero advierte que las comparaciones actuales no están controladas y que no se ha establecido un sistema federativo a escala fundacional [Chatzikyriakidis and Lappin 2026]. La encuesta de planificación-formalizador trata a LLMs como constructores y refinadores de modelos formales, no como reemplazos para planificadores [Tantakoun et al. 2025]. La encuesta de análisis de programas hace hincapié en los sistemas híbridos y la necesidad de gestionar la alucinación y la reproducibilidad [Wang et al. 2025]. Estas son líneas de evidencia independientes para la misma división del trabajo.

Por lo tanto, el campo ha logrado más que un patrón superficial de “LLM más solucionador”. Ha demostrado reparación guiada por solucionador, formalizaciones comprobadas, planificación formal, guía de ruta, propuesta invariante, síntesis de transformadores y robustez certificada. También ha expuesto un límite estable. Cuando el LLM controla la aceptación final, las garantías se aproximan. Cuando el LLM propone artefactos que un sistema formal separado verifica, se pueden mantener fuertes garantías para la propiedad comprobada. La pregunta sin respuesta es cómo hacer que el límite de lenguaje natural a artefacto sea comparablemente disciplinado.

La arquitectura propuesta más adelante en este libro aplica directamente las lecciones de análisis de programas. Las transformaciones semánticas candidatas se tratan como código sintetizado. Tienen contratos tipificados, mapas de origen, límites de recursos y pruebas. Los dominios abstractos se registran en lugar de improvisar durante una carrera. Los predicados de refinamiento son sugeridos por LLMs pero validados a través de distinciones concretas. Los portafolios de Solver exponen objetos o testigos de prueba. El renderizador de lenguaje consume resultados canónicos, no trazas crudas. En este diseño, la flexibilidad del LLM es un activo precisamente porque está rodeado de puertas de aceptación simbólicas.

5. Por qué fallan los sistemas de lenguaje ejecutable

5.1 Base, identidad, formalizaciones espurias, ontologías y conocimiento mundial

El error dominante en los sistemas de lenguaje ejecutable no es una deducción incorrecta de las premisas aceptadas. Es la admisión de las premisas, objetos o relaciones equivocadas. La conexión a tierra es el proceso por el cual los símbolos adquieren referencia operativa: un predicado está conectado a una frase, una entidad a una o más menciones, un evento a la evidencia, una expresión de tiempo a un intervalo, un campo de base de datos a un concepto, o una acción a una capacidad de entorno. Un programa formal sin fundamento fiable es exacto en el sentido menos útil. Se ejecuta consistentemente mientras describe el mundo equivocado.

La conexión a tierra comienza con la ubicación de la fuente. Cada observación semántica debe estar vinculada a la evidencia exacta de la fuente o explícitamente marcada como una suposición externa. Una cita debe ser revalidada contra la revisión de la fuente inmutable. Un resumen del documento no puede sustituir a la fuente porque puede omitir las calificaciones. Los resultados de la recuperación no pueden soportar una afirmación sobre la ausencia a menos que el proceso de recuperación esté completo para el dominio relevante. Esta disciplina de origen evita un fallo común en los sistemas LLM: un modelo produce un texto de soporte plausible que no se produce en el documento, y la salida estructurada aguas abajo da a la invención una apariencia de autoridad.

La base léxica conecta el lenguaje con la ontología. La frase “aprobación” puede denotar una acción, un estado, un artefacto firmado, un permiso o una decisión institucional. Un esquema de dominio debe elegir y documentar la interpretación que necesita una regla. LLMs son útiles porque pueden inferir contexto y proponer mapeos, pero un mapeo debe probarse en contra de ejemplos y alternativas. Una coincidencia de cadena entre una frase y un nombre de tipo no es compatibilidad semántica. La documentación de nllAgent advierte explícitamente contra el mapeo de dos conceptos simplemente porque sus etiquetas coinciden; requiere un adaptador o una sonda semántica cuando los significados difieren.

La base de identidad es más difícil y más consecuente. El análisis de documentos a menudo necesita saber si dos menciones denotan la misma persona, organización, objeto, conjunto de datos, requisito o evento. Los nombres iguales no implican identidad, y los nombres diferentes no implican diferencia. Los títulos cambian, se producen alias, los pronombres son ambiguos, las organizaciones tienen subdivisiones, y las menciones ficticias o citadas pueden no pertenecer al mundo de origen. Una capa de identidad segura separa las menciones de las hipótesis de la entidad y los candidatos a la identidad. Registra el productor, la evidencia, el alcance, el mundo y las alternativas para cada enlace.

La fusión de identidad precoz crea errores en cascada. En una narración, dos personajes con el mismo apellido pueden fusionarse, produciendo violaciones de continuidad imposibles. En un documento reglamentario, “la Autoridad” puede referirse a diferentes organismos en diferentes jurisdicciones. En la documentación de software, “cliente” puede denotar un usuario, un componente de aplicación o un punto final HTTP. Una vez fusionadas, las reglas posteriores tratan todas las propiedades como pertenecientes a un objeto. El error se hace difícil de diagnosticar porque el solucionador ve un identificador consistente. Un sistema conservador

debe permitir la identidad no resuelta y regresar desconocido cuando una regla requiere un enlace que no se ha establecido.

La conexión a tierra temporal tiene capas similares. Las fechas pueden ser explícitas o relativas. “En un plazo de treinta días” requiere un evento de anclaje y una política de calendario. “Antes de la liberación” puede referirse a una versión planificada o real. “Después de la aprobación” puede significar inmediatamente después o en cualquier momento posterior. Las jurisdicciones definen los días hábiles de manera diferente. Un solucionador temporal puede gestionar los intervalos y ordenar exactamente una vez que se realizan estas elecciones, pero no puede inferir el anclaje previsto de una etiqueta arbitraria. Por lo tanto, las observaciones temporales deben incluir la expresión textual, el valor normalizado o la restricción, el anclaje, el calendario, la zona horaria, la confianza y las alternativas.

La conexión a tierra normativa se maneja con frecuencia mal. “Debe”, “deberá”, “debería”, “puede”, “puede” y “se espera que” tenga fuerza específica de dominio. Los convenios de redacción jurídica difieren del lenguaje ordinario. Una excepción puede derrotar una regla, reducir su alcance o crear un permiso separado. Las autoridades en conflicto pueden tener prioridades. Una simple representación booleana del cumplimiento pierde estas distinciones. La lógica deóntica, las prioridades de las reglas, las tablas de decisión o los sistemas desviados pueden modelarlos, pero el mapeo de origen a regla sigue siendo un juicio lingüístico e institucional.

Las formalizaciones espurias surgen cuando el programa equivocado produce el resultado esperado. El análisis semántico débilmente supervisado hizo explícito este problema, pero es omnipresente en la evaluación LLM. Una fórmula generada puede resolver un punto de referencia porque los ejemplos tienen plantillas regulares. Una consulta de texto a SQL puede devolver las filas actuales correctas a pesar de una unión incorrecta. Un modelo de planificación puede encontrar un plan porque una restricción omitida es irrelevante en la instancia probada. Un circuito de reglas puede pasar diez ejemplos mientras se codifica el concepto incorrecto. La precisión de la respuesta final por sí sola no puede distinguir estos casos.

La defensa apropiada es la mutación semántica. El evaluador cambia un aspecto de la fuente o el medio ambiente mientras preserva otros, luego verifica si la formalización cambia de la manera esperada. Si “todo” se convierte en “alguno”, el cuantificador debería cambiar. Si se elimina una excepción, el camino previamente exento debería estar prohibido. Si dos entidades se renombran a la misma cadena, la identidad no debería fusionarse automáticamente. Si la instancia de la base de datos cambia, una consulta SQL correcta accidentalmente debería fallar. Las pruebas de mutación evalúan el significado del programa en lugar de solo su comportamiento en un solo accesorio.

La generación de contraejemplos puede comparar formalizaciones competitivas. Dados dos programas lógicos o ejecutables, un solucionador puede buscar un modelo en el que sus salidas difieran. El escenario resultante se traduce al lenguaje natural y se presenta a un revisor. Esto convierte una comparación de fórmulas opacas en una pregunta concreta: “En la interpretación A, un supervisor que no ha completado la formación puede aprobar; en la interpretación B, no pueden. ¿Cuál está destinado?” El método es especialmente útil cuando varias LLMs producen formas plausibles pero no equivalentes.

El diseño de ontología crea otro modo de fallo. Un esquema universal es atractivo para la reutilización, pero puede ocultar los desacuerdos del dominio y forzar los conceptos en

categorías inapropiadas. Un esquema extremadamente estrecho, por el contrario, produce muchos idiomas locales incompatibles. Es preferible un diseño en capas. La capa superior define la fuente, la evidencia, las hipótesis de identidad, los mundos, el tiempo, la modalidad, el estado, la cobertura y las brechas. Los paquetes de dominio definen tipos y relaciones sustanciales. Los paquetes de mapeo declaran correspondencias, pérdidas y suposiciones entre dominios.

La compatibilidad de la ontología debe ser operativa, no retórica. Un circuito declara los tipos y garantiza que consume. Un productor declara qué tipos puede materializar, bajo qué estados y modos de cobertura. La publicación comprueba si existe un productor compatible. El tiempo de ejecución comprueba si el documento concreto arrojó las observaciones requeridas. Si el circuito necesita identidades confirmadas por el ser humano, pero solo existen candidatos de identidad propuestos por modelos, no puede correr en la garantía reclamada. Si una regla temporal necesita un calendario jurisdiccional que está ausente, el resultado es una brecha operativa, no una fecha adivinada.

El conocimiento mundial es particularmente peligroso. LLMs contiene un amplio conocimiento pero no una procedencia estable, fecha efectiva o jurisdicción. El tratamiento de la memoria del modelo como una base de datos formal crea dependencias invisibles y conclusiones obsoletas. Un sistema simbólico debe cargar paquetes de conocimiento explícitos con identificadores de origen, digestos, intervalos efectivos, jurisdicciones, políticas de conflicto y techos de garantía. El paquete puede ser curado, recuperado o asistido por modelo, pero sus reivindicaciones aceptadas son artefactos externos. Un paquete legal puede expirar. Un paquete científico puede ser reemplazado. Un paquete de mundo ficticio puede contradecir deliberadamente la realidad actual.

Los conflictos entre fuentes no deben borrarse mediante el voto por mayoría a menos que la tarea defina dicha política. El sistema puede necesitar un razonamiento paraconsistente, prioridad de fuente, versionamiento temporal o mundos separados. Una afirmación puede ser apoyada por una fuente y contradicha por otra. El resultado canónico debe preservar ambos y establecer qué política, si la hay, los resuelve. Esto es esencial para las encuestas científicas y el análisis regulatorio, donde el desacuerdo es información más que ruido.

Los supuestos de fondo deben ser visibles. Los puntos de referencia lógicos a menudo asumen que los nombres denotan individuos distintos o que las categorías son disjuntas. Los documentos reales rara vez indican todos esos axiomas. Un LLM puede insertarlos porque hacen que el rompecabezas sea solucionable. En un sistema de investigación o cumplimiento, esos supuestos deben aparecer en una capa de autoridad separada. La respuesta final puede entonces indicar que una conclusión solo se sigue si la suposición se mantiene. Esto impide que el conocimiento implícito del mundo se haga pasar por evidencia documental.

La cobertura es la base de la ausencia. Para concluir que un documento no contiene una aprobación requerida, el sistema debe saber qué se buscó, qué secciones y canales se incluyeron, cómo se reconoció el concepto y si el método de búsqueda estaba completo para la reclamación. Una exploración léxica puede ser completa para una frase exacta, pero no para las parafrases. La recuperación puede ser un alto recuerdo, pero no un mundo cerrado. Un registro curado por humanos puede estar completo para una liberación. La cobertura debe ser un objeto de primera clase consumido por reglas negativas.

La arquitectura nllAgent contiene una versión importante de este principio. LongTextJS registra cobertura y brechas junto con observaciones. La compatibilidad del circuito comprueba

si el programa real satisface el estado y la cobertura requeridos. El conocimiento faltante no se informa como cumplimiento. Esto es más fuerte que el patrón común de pedir a un LLM que devuelva true, false o no encontrado, porque no se encuentra se interpreta en relación con un dominio de búsqueda documentado [AssistOS-AI 2026].

La dificultad restante es cómo validar las observaciones producidas por el modelo. La validación del esquema es necesaria pero débil. El anclaje de la evidencia impide las citas inventadas, pero no la mala interpretación. Los puntajes de calibración ayudan a clasificar a los candidatos pero no establecen significado. El enfoque práctico más fuerte combina señales independientes: analizadores de lenguaje controlado para fragmentos tratables; conjuntos modelo o diversas indicaciones; restricciones de ontología; consistencia de frase cruzada; mutaciones semánticas; ejemplos negativos; revisión humana para distinciones de alto impacto; y contraejemplos aguas abajo que exponen consecuencias.

La conexión a tierra también debe ser impulsada por consulta. Un sistema no necesita resolver cada entidad o evento en un documento de 300 páginas. Debe derivar la demanda semántica de la regla que se está ejecutando, recuperar los alcances relevantes y materializar solo los tipos necesarios. Sin embargo, el procesamiento de consulta primero no debe confundir la relevancia con la integridad. Si una regla pregunta si existe alguna excepción en cualquier parte del texto de la autoridad, la recuperación debe proporcionar un argumento de cobertura o la respuesta sigue siendo desconocida. La ejecución impulsada por la demanda reduce el trabajo; no justifica el cierre prematuro.

La conclusión fundamental es severa pero constructiva. No hay un remedio puramente simbólico para un símbolo adjunto al referente equivocado. La conexión a tierra no es un detalle preliminar NLP; es parte del cálculo de confianza. El sistema debe exponer la cadena de fuente a símbolo, preservar alternativas, rastrear la cobertura y rechazar la ejecución incompatible. Una vez que esa cadena es disciplinada, las herramientas formales maduras pueden agregar una confiabilidad sustancial. Sin ella, las herramientas simplemente certifican las consecuencias de una ficción plausible.

5.2 Pragmática, no monotonicidad, escala, seguridad y falla de evaluación

Incluso una representación bien fundamentada puede fracasar porque el lenguaje natural hace más que los hechos monótonos del estado. Los oradores implican, presuponen, revisan, citan, bromea, negocian y confían en el contexto institucional. Los documentos contienen excepciones, prioridades, definiciones que cambian más tarde y declaraciones cuya fuerza depende del género. El lenguaje natural ejecutable debe decidir cuál de estos fenómenos pretende apoyar y debe volver desconocido fuera de ese ámbito.

La pragmática es la primera frontera. Una oración puede comunicar más que sus condiciones de verdad literales. “El informe está casi completo” puede implicar que no esté completo. “Algunas pruebas aprobadas” pueden sugerir pragmáticamente que no todo lo hizo, aunque la lógica clásica no implica eso. La ironía revierte el sentimiento aparente. Las solicitudes indirectas utilizan declaraciones como acciones. Un formalizador que trata las inferencias pragmáticas como hechos puede extralimitarse; una que los ignora puede perder la intención práctica del autor. El tratamiento adecuado depende de la aplicación. Un motor de cumplimiento generalmente debe privilegiar la autoridad explícita y etiquetar la interpretación pragmática como se propone. Un agente conversacional puede necesitar modelos pragmáticos más ricos, pero no debe representarlos como evidencia sólida.

La presuposición crea un desafío relacionado. “El comité dejó de revisar la propuesta” presupone que la había estado revisando. Las descripciones definidas a menudo presuponen la existencia o la singularidad. Los documentos pueden explotar tales construcciones sin afirmar explícitamente los antecedentes. Un sistema formal debe decidir si las presuposiciones entran como suposiciones, inferencias desviables o requisitos para la aclaración. Afirmarlos automáticamente puede ser inseguro; ignorarlos puede hacer que el discurso sea incoherente. El sistema de estado debe preservar su origen.

Las reglas del lenguaje natural son a menudo no monotónicas. La nueva información puede retractarse de una conclusión. “Los empleados pueden acceder al sistema” pueden ser derrotados por “a menos que su certificación haya expirado”. La lógica predeterminada, la programación de conjunto de respuestas, las lógicas desviables y las reglas priorizadas proporcionan mecanismos formales para las excepciones. La lógica clásica de primer orden puede codificar las excepciones explícitamente, pero los grandes conjuntos de reglas se vuelven difíciles de administrar y la información inconsistente puede trivializar la inferencia si se aplica una explosión ordinaria.

Un tiempo de ejecución práctico debe apoyar la semántica de múltiples reglas. Monotone Datalog es apropiado cuando los hechos solo se acumulan. Las reglas desviables son apropiadas para los incumplimientos y excepciones. El razonamiento paraconsistente es apropiado cuando las fuentes entran en conflicto y el sistema debe evitar derivar todo. Las tablas de decisiones son apropiadas cuando las combinaciones de políticas son finitas y explícitas. La semántica elegida debe ser versionada con el paquete de reglas; no puede ser inferida ad hoc por el LLM durante la ejecución.

Los sistemas normativos añaden autoridad y prioridad. Una regla posterior puede anular una anterior. Una regulación de nivel superior puede adelantarse a un procedimiento local. La jurisdicción y la fecha efectiva importan. Los permisos y obligaciones pueden entrar en conflicto. Las paradojas deónticas muestran que las codificaciones lógicas ingenuas pueden

producir resultados contraintuitivos. Muchos casos prácticos son mejor manejados por prioridades explícitas de reglas, excepciones y condiciones de violación que por un cálculo deontic universal. El lenguaje natural ejecutable debe favorecer la claridad operativa sobre la grandeza teórica.

La escala crea varias explosiones que interactúan. Los documentos largos contienen muchos tramos y observaciones de candidatos. Las menciones ambiguas crean asignaciones de identidad combinatorias. Las relaciones temporales crean alternativas de orden parcial. Las reglas crean consecuencias derivadas. La ejecución simbólica crea rutas. Los solucionadores se enfrentan a grandes sistemas de restricción. Las ventanas y los costos del contexto LLM imponen límites separados. Una arquitectura ingenua que formaliza cada frase en cada lógica posible es inviable.

La materialización impulsada por la demanda es la primera respuesta. Los circuitos declaran lo que consumen. La estructura de los índices de tiempo de ejecución y la información léxica estable, luego produce una semántica más rica solo para los alcances relevantes. La computación incremental reutiliza artefactos sin cambios. La interpretación abstracta resume las alternativas. Caminos de control de exploración simbólica de fusión y delimitación del Estado. Las carteras de solver enrutan casos simples a motores baratos. Los presupuestos duros producen resultados explícitos incompletos en lugar de truncamiento oculto.

La segunda respuesta es la modularidad semántica. Un documento se puede analizar a través de vistas: secciones, ventanas de tiempo, jurisdicciones, escenas narrativas, etapas experimentales o capas de autoridad. La identidad y las consecuencias de las reglas pueden ser locales a la vista. Las relaciones entre vistas se introducen solo cuando es necesario. Esto reduce la fusión global accidental y apoya el paralelismo. El sistema debe seguir registrando cuando una conclusión depende de la suposición de que una opinión está completa.

La tercera respuesta es la precisión por etapas. Un paso estructural o léxico barato elimina muchos casos. Un pase de extracción asistido por modelo produce candidatos para los restantes. El análisis abstracto identifica posibles violaciones. El refinamiento simbólico solo maneja ambigüedades relevantes para los resultados. La revisión humana está reservada para opciones no resueltas de alto impacto. Esto se asemeja a las tuberías de análisis estático que utilizan análisis baratos antes de que el teorema sea costoso.

Las amenazas de seguridad surgen en cada etapa. La inyección rápida en los documentos de origen puede indicar al LLM que ignore los esquemas o los resultados. El traductor debe recibir contenido de origen como datos bajo un mensaje específico de rol, y su salida debe validarse de forma independiente. La fuente nunca debe obtener autoridad para modificar el tiempo de ejecución, el paquete de reglas o la política de publicación. Cualquier texto que diga ser una instrucción del sistema sigue siendo contenido de documento.

El código generado es un riesgo aún mayor. Un DSL que permite JavaScript, Python, importaciones o acceso a la red arbitraria convierte el aprendizaje semántico en ejecución de código. Los circuitos declarativos deben normalizarse a los datos de línea y llamar solo a los operadores registrados. Los algoritmos personalizados deben instalarse como extensiones revisadas con contratos mecanografiados, efectos, límites de recursos y registros bloqueados. El sandboxing sigue siendo necesario, pero la restricción de capacidad es más fuerte que confiar solo en un límite de proceso.

Los ataques de solucionador incluyen la denegación de servicio a través de restricciones patológicas, objetos de prueba mal formada y la explotación de bibliotecas nativas. Se requieren presupuestos, restricciones teóricas, verificación de pruebas y fijación de versiones. Un modelo puede construir deliberada o accidentalmente una fórmula que es técnicamente válida pero computacionalmente explosiva. La selección del formalismo debe estimar los costos y rechazar tareas fuera de la política.

La intoxicación por datos afecta a los productores semánticos aprendidos y a las suites de referencia. Si un agente de codificación genera tanto un circuito como los ejemplos que lo validan, los artefactos pueden compartir la misma incompreensión. La generación de pruebas independientes, las mutaciones semánticas, los casos de anclaje curados manualmente y los roles de revisión separados reducen este riesgo. La arquitectura de aprendizaje nllAgent impide correctamente que un agente de codificación publique su propio candidato. La publicación manual es un control de gobernanza, aunque su eficacia depende de la calidad de los puntos de referencia y de los controles independientes.

La realización del lenguaje natural puede reintroducir las reclamaciones no admitidas. Un modelo puede convertir “posible bajo una interpretación” en “el documento viola la regla”. Puede omitir suposiciones o convertir una búsqueda incompleta en ausencia. Por lo tanto, la realización debe verse limitada por un esquema de resultados canónico y verificado a nivel de reclamación. Los resultados de alto impacto pueden utilizar plantillas controladas. Los comentarios explicativos deben estar claramente separados de los hallazgos verificados.

La evaluación es quizás la falla más sistémica. Muchos puntos de referencia miden solo la precisión de la respuesta final. Esto confunde la extracción, la formalización, el razonamiento y la representación. Un modelo puede llegar a la respuesta correcta por la razón equivocada. Un sistema formal puede ser penalizado por devolver desconocido cuando el índice de referencia asume una convención no declarada. Los conjuntos de datos sintéticos pueden recompensar el reconocimiento de plantillas. Las métricas de LLM-as-a-judge pueden compartir sesgos con los sistemas que se evalúan.

Un punto de referencia riguroso debe contener múltiples capas. La capa de origen incluye texto y estructura exactos. La capa semántica incluye una o más interpretaciones admisibles con intervalos de evidencia. La capa formal incluye programas objetivo o pruebas de equivalencia extensiva. La capa de ejecución incluye modelos esperados, pruebas, testigos o núcleos insatisfecibles. La capa de resultados incluye estados canónicos y cobertura. La capa de realización incluye reclamaciones permitidas y exageraciones prohibidas. Los errores pueden atribuirse en lugar de colapsarse en una sola puntuación.

Los pares mínimos son esenciales. Cambiar una palabra o relación debe producir una diferencia formal predecible. Las familias de parafraseo prueban la invariancia. Recuperación de pruebas de secciones de distractor. Las colisiones de identidad prueban la puesta a tierra. La falta de evidencia prueba un comportamiento desconocido. Las fuentes en conflicto prueban la política de paraconsistencia o autoridad. Los casos de límites temporales prueban calendarios e intervalos. Aislamiento de seguridad de pruebas de texto rápido de adversarios. Las mutaciones semánticas prueban si un circuito realmente implementa su regla.

La evaluación también debería medir la abstención. Un sistema confiable debe saber cuándo carece de ontología, productor, cobertura o presupuesto. Desconocido no es un fracaso cuando la pregunta no está soportada. Las métricas deben distinguir la abstención justificada de la incompletitud evitable. Las curvas de calidad de cobertura pueden mostrar cómo la

precisión y la falsa garantía cambian a medida que el sistema admite más semántica propuesta por el modelo.

Asuntos de implementación independientes. Un compilador de circuito y su evaluador de “sombra” no deben compartir todas las funciones críticas si se reclaman pruebas diferenciales. Las puntuaciones de mutación deben provenir de mutaciones ejecutables, no de descripciones en prosa. Los perfiles de modelos en vivo deben evaluarse por separado de los accesorios deterministas. La planificación y la realización necesitan puntos de referencia específicos de la tarea en lugar de inferirse del desempeño de la auditoría. Estos son precisamente varios de los problemas graves reconocidos en el repositorio nllAgent [AssistOS-AI 2026].

Las revisiones externas deben interpretarse con un cuidado similar. La encuesta de Frontiers 2026 es revisada por pares y útil, pero su afirmación de que los sistemas federativos funcionan mejor está explícitamente calificada por la confusión de tareas. Las encuestas de planificación y análisis de programas mapean las tendencias, pero no pueden establecer que una arquitectura propuesta funcione en documentos sin restricciones. No se encontró una revisión independiente de nllAgent. Por lo tanto, un libro responsable debe usar revisiones para identificar el consenso y el desacuerdo, y luego declarar sus propios juicios donde la evidencia está ausente.

El principio central de la evaluación es la atribución causal. Cuando un sistema mejora, los investigadores deben determinar si la ganancia provino de un mejor análisis semántico, corrección del solucionador, recuperación, muestreo de tiempo de prueba o fugas de referencia. Cuando falla, deben identificar la capa. Esto no es simplemente una higiene científica. Los datos de error en capas son lo que permite que la arquitectura mejore sin reemplazar todo el sistema ni ocultar regresiones en las puntuaciones agregadas.

El lenguaje natural ejecutable seguirá siendo una capacidad limitada. La pragmática, el conocimiento de mundo abierto, el contexto social y las instituciones en evolución aseguran que ningún núcleo simbólico finito capture todo significado. El objetivo no es eliminar este límite, sino hacerlo explícito y útil. Un sistema puede ofrecer fuertes garantías para fragmentos estructurales, relacionales, temporales, cuantitativos y gobernados por reglas mientras devuelve resultados condicionales en otros lugares. El sistema más peligroso no es uno que admita límites. Es uno que convierte el lenguaje no modelado en una salida simbólica segura y llama al resultado verificado.

6. Familias de arquitectura para lenguaje natural ejecutable

6.1 Diseños de la competencia: Representaciones universales, carteras de formalismo, compilación controlada y sistemas de consulta primero

Las arquitecturas de lenguaje ejecutable se pueden organizar de acuerdo a donde ponen el compromiso semántico. Algunos intentan traducir cada entrada en una representación universal. Otros conservan varias formas candidatas y seleccionan un formalismo por tarea. Algunos restringen el lenguaje de entrada hasta que la compilación sea determinista. Otros preservan el lenguaje abierto, pero formalizan solo las observaciones exigidas por una consulta. Cada diseño resuelve un problema diferente, y un sistema maduro probablemente los combinará en lugar de seleccionar uno exclusivamente.

La representación intermedia universal es la arquitectura más intuitiva. El texto se compila en un rico gráfico semántico o lenguaje lógico, y cada operación descendente consulta o transforma ese objeto común. Las ventajas son la unidad conceptual, la reutilización y la posibilidad de controles de coherencia globales. Una sola representación puede admitir múltiples aplicaciones y crear una interfaz estable entre NLP y el razonamiento.

Su debilidad es la extralimitación semántica. Para servir a cada tarea, la representación debe cubrir entidades, eventos, discurso, tiempo, modalidad, cantidad, causalidad, normas, incertidumbre, procedencia y conceptos específicos del dominio. O el lenguaje se vuelve extremadamente complejo o simplifica las distinciones que alguna tarea necesita. Una representación universal también crea un único punto de fracaso: un sinatizado global equivocado contamina cada análisis. La presión para elegir una identidad, alcance u ontología temprano puede ocultar la ambigüedad.

Un pivote semántico es una variante más modesta. No pretende ser una forma lógica completa. Estandariza los intervalos de origen, menciones, hipótesis de entidades, observaciones, mundos, valores que llevan el tiempo, modalidad, procedencia, alternativas, cobertura y brechas. Los compiladores especializados rebajan las porciones seleccionadas en objetivos formales. Esto preserva la infraestructura común sin pretender que una semántica de ejecución se ajusta a todo significado. LongTextJS, como se documenta en nllAgent, está cerca de esta idea: es un mundo de origen direccionable y un programa de observación en lugar de un lenguaje universal de prueba de teoremas [AssistOS-AI 2026].

La arquitectura de múltiples candidatos trata la formalización como búsqueda. Un LLM produce varias formas lógicas, programas o gráficos semánticos. Las comprobaciones de tipo y las restricciones de ontología eliminan a los candidatos no válidos. La ejecución, los ejemplos y los contraejemplos clasifican o distinguen a los supervivientes. El sistema puede devolver una conclusión obligada cuando todos los candidatos están de acuerdo y una conclusión de cuándo solo algunos lo hacen. Esta arquitectura es apropiada cuando la ambigüedad es genuina o cuando la fiabilidad de la formalización es limitada.

Su costo es combinatorio. Los formularios de candidatos se multiplican entre oraciones y elecciones de identidad. Un producto cruzado ingenuo se vuelve inviable. La solución es representar la estructura compartida y las alternativas locales de manera compacta, usar

dominios abstractos para resumirlos y refinar solo las opciones relevantes para los resultados. La confianza puede guiar la exploración, pero los candidatos de baja probabilidad no deben ser eliminados de un análisis sólido a menos que la política acepte explícitamente la incompletitud probabilística.

La compilación de lenguaje controlado adopta el enfoque opuesto. La entrada se reescribe en un lenguaje natural restringido cuya gramática y semántica son deterministas o altamente restringidas. Los usuarios pueden autorizar directamente en el idioma controlado, o un LLM puede proponer una parafrase controlada para su confirmación. Una vez aceptada, la compilación es confiable. Esta arquitectura es particularmente fuerte para los requisitos, contratos, procedimientos y creación de reglas.

Su limitación es cobertura y adopción. Los documentos ordinarios contienen construcciones fuera del fragmento controlado. La reescritura puede cambiar de significado. Los usuarios pueden aprobar una paráfrasis sin notar una diferencia sutil. La arquitectura funciona mejor cuando la forma controlada se muestra junto a la fuente, se localizan las diferencias y los contraejemplos formales hacen que las decisiones importantes sean concretas. El lenguaje controlado debe ser un punto de control semántico explícito, no una normalización invisible.

Las arquitecturas de consulta primero evitan la formalización total. Un circuito o pregunta declara qué observaciones y relaciones necesita. El tiempo de ejecución preserva la fuente estructuralmente, recupera regiones relevantes y materializa solo los tipos semánticos demandados. Las consultas relacionales, las tablas y las reglas de decisión operan sobre el mundo finito resultante. Este diseño escala mejor a documentos largos y alinea el cálculo con el propósito.

Una visión estrechamente relacionada de 2026 de sistemas de bases de datos distingue a los operadores semánticos interpretados de los compilados. En el diseño interpretado, se invoca un LLM dentro del bucle de procesamiento de datos para cada fila, registro o par candidato. En el diseño compilado, el LLM se invoca durante una fase de construcción separada para traducir la operación semántica en código local determinista que se puede ejecutar repetidamente sin una llamada modelo. Los experimentos preliminares informaron reducciones sustanciales en la latencia y uso del modelo, conservando gran parte de la calidad de salida [Dong and Wang 2026]. La propuesta es importante porque alcanza de forma independiente la misma intuición arquitectónica desarrollada en este libro: el modelo de lenguaje es más útil como compilador o constructor de planos, mientras que el tiempo de ejecución repetido debe ser estable e inspeccionable. Su limitación es igualmente instructiva. El código generado puede aproximarse al operador definido por el modelo de manera eficiente sin proporcionar una prueba semántica de que captura el concepto deseado del usuario. La compilación elimina la estocasticidad por artículo; no resuelve por sí misma la conexión a tierra o la fidelidad de las especificaciones.

El riesgo es un cierre falso. La recuperación puede faltar evidencia relevante, y el análisis de la demanda puede omitir un concepto necesario. Los sistemas de consulta primero deben realizar un seguimiento de la cobertura y los productores. Si una conclusión negativa requiere buscar en todo el documento cualquier excepción, algunos pasajes recuperados son insuficientes. La arquitectura debe distinguir un resultado de consulta limitado de una garantía de mundo cerrado. Los contratos de compatibilidad y cobertura de nllAgent son un mecanismo importante en esta familia.

Una arquitectura de formalismo único reduce todas las tareas en una lógica, a menudo lógica de primer orden, Datalog o SMT. Simplifica las obligaciones de implementación y prueba. Para un dominio estrecho, esta puede ser la mejor opción. Logic-LM y LINC obtienen gran parte de su valor de un idioma objetivo estable. Un sistema para la programación puede utilizar razonablemente un lenguaje de restricción. Un verificador de protocolo puede usar un cálculo de proceso.

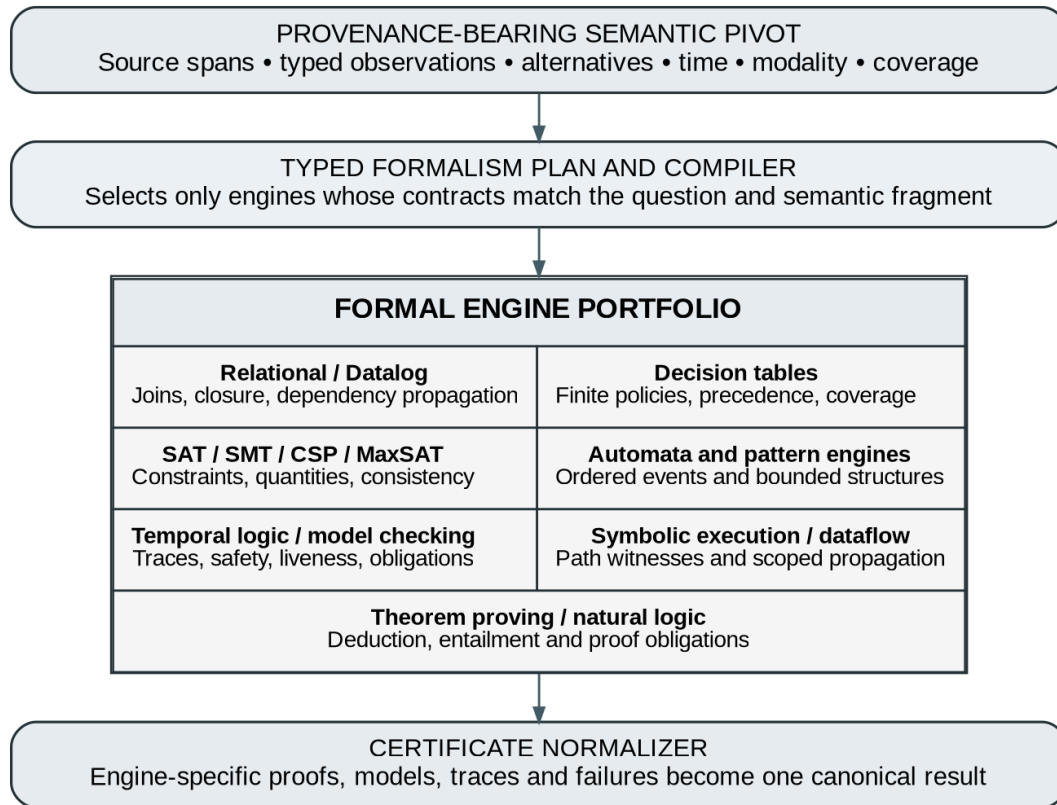


Figura 3. Un pivote semántico que tiene procedencia se puede compilar en una cartera de motores formales especializados y normalizarse en un solo resultado canónico.

Para tareas amplias de lenguaje natural, una cartera de solucionadores es más plausible. La evaluación relacional maneja las consultas de documentos. Datalog calcula el cierre monótono. Las tablas de decisión expresan políticas finitas. SAT y SMT manejan las restricciones y la aritmética. La comprobación del modelo maneja el comportamiento temporal. La ejecución simbólica construye testigos de ruta. La lógica natural maneja ciertas relaciones de implicación cerca del lenguaje de la superficie. Los autómatas reconocen los patrones ordenados. El pivote semántico suministra átomos conectados a tierra, y un plan de formalismo selecciona uno o más motores.

La cartera crea nuevas obligaciones. El router debe conocer las capacidades expresivas y operativas de cada motor. Un compilador debe preservar el significado a través de las bajadas. Los resultados de diferentes motores necesitan un modelo de estado y procedencia común. Si dos motores razonan sobre los predicados superpuestos, la consistencia debe ser marcada o explícitamente abierta. Las políticas de recursos deben impedir que un modelo seleccione un solucionador innecesariamente costoso.

Un contrato de formalismo útil establece los tipos de entrada, operadores compatibles, semántica, cobertura requerida, clase de garantía, certificado esperado, modelo de costo y

limitaciones conocidas. Un motor de lógica temporal puede requerir eventos conectados a tierra y un sistema de transición finito. Un motor SMT puede soportar funciones reales no lineales enteras pero no restringidas. Una tabla de decisiones puede requerir condiciones enumeradas finitas. Un motor de lógica natural puede soportar la monotonicidad pero no la coreferencia de sentido transversal arbitraria. El router puede rechazar un plan propuesto cuyos requisitos superen el motor.

La generación de pruebas es otra familia de arquitectura. El artefacto principal no es la respuesta del lenguaje natural, sino un resultado canónico más una prueba, testigo, seguimiento de ejecución o derivación verificable. El modelo de lenguaje genera una explicación a partir de ese artefacto. Cada reclamo explicativo está vinculado a un elemento de resultado. Un comprobador independiente asegura que la explicación no exceda el contenido formal.

Este diseño es valioso porque la generación y la verificación tienen diferentes fortalezas. Un LLM puede escribir una explicación clara de una restricción contraejemplo o insatisfactoria. Un resultado simbólico puede garantizar que el escenario sea real bajo el modelo aceptado. La explicación todavía puede ser engañosa si oculta suposiciones o incertidumbre, por lo que el resultado canónico debe incluir esas dimensiones y el contrato de renderizador debe requerirlas cuando sea material.

Las arquitecturas abstracto-semánticas colocan una capa de interpretación abstracta antes de la ejecución especializada. Las observaciones de candidatos poblan pueden y deben ser dominios. La propagación barata determina qué propiedades están definitivamente satisfechas, definitivamente son imposibles o potencialmente violadas. Solo las violaciones potenciales se reducen a costosos problemas simbólicos. Esto reduce la carga del solucionador y proporciona estados desconocidos de principios. También crea una arquitectura incremental natural para ediciones de documentos.

Las arquitecturas guiadas por contraejemplo cierran el bucle. Una prueba fallida o un testigo espurio no es simplemente un error; identifica qué distinción semántica importa. El sistema mapea los hechos del solucionador a los intervalos de origen, pide al LLM o al usuario que refine la interpretación relevante y reproduce el análisis. Esta arquitectura es especialmente prometedora porque transforma la retroalimentación formal en comprensión del lenguaje dirigido en lugar de la auto-reflexión genérica.

Las arquitecturas de aprendizaje rigen cómo nuevas reglas, ontologías y circuitos entran en producción. Un extremo permite que un agente LLM genere código ejecutable durante cada ejecución. Esto maximiza la flexibilidad y minimiza la seguridad. El extremo opuesto solo permite reglas escritas a mano y formalmente verificadas. Un medio práctico utiliza agentes de codificación en espacios de trabajo de puesta en escena, genera puntos de referencia y mutaciones, restringe los artefactos a formas declarativas y requiere una publicación explícita. El tiempo de ejecución de producción lee lanzamientos inmutables. Este es el modelo de gobernanza implementado en nllAgent y debe ser generalizado.

La siguiente comparación resume la compensación central sin afirmar que un diseño domina cada tarea.

Cuadro 1. Principales familias de arquitectura y sus compensaciones esenciales.

Arquitectura	Fuerza esencial y principal riesgo
Representación semántica universal	Reutilización y consistencia global; riesgos de sobrecomplejidad y compromiso semántico prematuro.

Arquitectura	Fuerza esencial y principal riesgo
Pivote semántico más descensos especializados	Conserva evidencia común y apoya múltiples motores; requiere contratos cuidadosos de compiladores y motores cruzados.
Formalizaciones de múltiples candidatos	Representa la ambigüedad y permite conclusiones pueden/deben; corre el riesgo de crecimiento combinatorio.
Compilación de lenguaje controlado	Una fuerte semántica y formas inspeccionables por el usuario; sacrifica la cobertura y puede introducir cambios de significado durante la reescritura.
Materialización de Consulta primero	Las escalas a documentos largos y necesidades específicas de la tarea; pueden confundir la recuperación con integridad a menos que la cobertura sea explícita.
Objetivo formal único	Base de confianza simple y evaluación en un dominio limitado; pobre ajuste para fenómenos de lenguaje heterogéneos.
Cartera de solver	Coincide con el método computacional con la estructura del problema; agrega enrutamiento, interoperabilidad y obligaciones de normalización de certificados.
Generación de prueba	Limita las respuestas en lenguaje natural al contenido verificado; requiere una sólida verificación de la realización del nivel de reclamo.
Análisis abstracto más refinamiento simbólico	Equilibra la escala y la precisión; la solidez permanece condicionada a la cobertura de interpretación.
Circuitos aprendidos gobernados	Permite la evolución sin automodificación en tiempo de ejecución; depende de puntos de referencia fuertes, pruebas de mutaciones y revisión de publicaciones.

La arquitectura general más creíble es un híbrido de las opciones medias. Utiliza un pivote semántico que lleva la procedencia, preserva alternativas donde importan, realiza la materialización impulsada por consultas, dirige a una cartera de formalismo y acepta solo resultados respaldados por pruebas o por testigos. El lenguaje controlado puede servir como un punto de control interactivo para formalizaciones de alto impacto. La interpretación abstracta controla la escala, mientras que el refinamiento guiado por contraejemplo mejora la precisión. El aprendizaje permanece fuera de la producción.

Esta conclusión no es solo teórica. Se refleja en la literatura empírica. Los traductores de lógica temporal, SatLM, Logic-LM, Logic-LM, LINC, los formalizadores de planificación, SymGPT y los sistemas de verificación certificada tienen éxito al reducir el objetivo formal. Ningún resultado demuestra que una lógica universal o un modelo de extremo a extremo pueda reemplazar la cartera. Por lo tanto, la arquitectura debe abarcar la heterogeneidad al tiempo que minimiza las interfaces de confianza entre los componentes.

6.2 Un pipeline de referencia del texto a los símbolos, la ejecución y las respuestas restringidas en lenguaje natural

Una canalización de lenguaje ejecutable de referencia debe entenderse como un compilador y una cadena de herramientas de verificación en lugar de un mensaje de agente. Cada etapa materializa un artefacto inmutable o con contenido. Una carrera se puede reproducir sin pedir al modelo que reconstruya su razonamiento anterior. Los errores se asignan a etapas. El aprendizaje puede mejorar un productor o una teoría sin cambiar silenciosamente el significado de los viejos resultados.

El primer artefacto es el paquete fuente. Contiene el texto o datos canónicos, resumen de revisión, lenguaje, estructura, canales y tramos estables. Los párrafos, encabezados, tablas, citas y bloques de código conservan el orden y la identidad. Los registros externos tienen claves igualmente estables. El paquete fuente es la autoridad léxica: establece lo que estaba presente, no lo que significa.

El segundo artefacto es el contrato de tarea. Establece la pregunta, la publicación de reglas, el alcance de la fuente, la garantía deseada, los supuestos permitidos, el mundo y la jurisdicción, los presupuestos y la política de resultados. La tarea determina la demanda semántica. Una solicitud para encontrar acrónimos indefinidos requiere diferentes observaciones de una solicitud para verificar el cumplimiento temporal. El contrato también define si se puede concluir la ausencia y qué formas de incertidumbre requieren la revisión humana.

La tercera etapa construye un gráfico de evidencia. Los productores deterministas añaden observaciones estructurales. Los analizadores de lenguaje controlado agregan observaciones para los fragmentos que reconocen. Los productores respaldados por LLM agregan eventos, relaciones, identidades, modalidades o cantidades candidatas en el esquema. Cada observación registra anclajes, productores, modelos o versiones de algoritmos, estado, alternativas y cobertura. El modelo no escribe directamente en el gráfico canónico; un materializador vuelve a comprobar los intervalos, los tipos y la integridad referencial.

La identidad y la gestión del mundo se producen en la misma capa, pero no se resuelven silenciosamente. Las menciones son evidencia local. Las hipótesis de las entidades recogen menciones bajo un mundo y alcance. Los candidatos de identidad establecen relaciones iguales, diferentes o no resueltas con apoyo. Las interpretaciones mutuamente exclusivas pertenecen a mundos separados o alternativas vigiladas. Por lo tanto, el gráfico puede representar desacuerdos sin corromper la consistencia global.

La cuarta etapa calcula un estado semántico abstracto. Las abstracciones específicas de dominio resumen posibles identidades, órdenes temporales, estados normativos, cantidades y niveles de evidencia. Los hechos deben mantenerse en todos los mundos admitidos; pueden los hechos mantener al menos uno. Unknown registra una brecha en lugar de una suposición probabilística. El estado abstracto apoya la propagación de reglas baratas e identifica qué preguntas requieren mayor precisión.

La quinta etapa produce un plan de formalismo. El planificador no es una decisión de forma libre LLM. Compara la tarea con los contratos de formalismo registrados. Una simple consulta de documento puede permanecer relacional. Una política puede compilarse en tablas de decisión y restricciones. Un procedimiento puede compilarse en un sistema de transición. Una violación dependiente de la ruta puede invocar la ejecución simbólica. Una reclamación

deductiva puede invocar un probador de teorema. Los nombres de los planes requerían entradas, bajadas, versiones de motores, límites de recursos y certificados esperados.

El LLM puede proponer un plan porque reconoce la estructura del problema y el lenguaje de dominio. Un comprobador determinista valida que el plan utiliza formalismos apoyados, que cada observación requerida tiene un productor compatible, y que no se pierde ninguna distinción semántica esencial para la tarea. Por ejemplo, una regla que implica excepciones y prioridad no se puede compilar en un motor monótono sin una codificación explícita. Una regla que involucra identidad no puede proceder si el dominio de identidad no está resuelto y el veredicto depende de ello.

La sexta etapa realiza la compilación mecanografiada. Este compilador debe ser pequeño, determinista y muy probado. Convierte objetos semánticos en variables, relaciones, transiciones o predicados de los solucionadores. Los mapas de origen conectan cada elemento formal a observaciones y tramos. Los compiladores deben generar obligaciones de prueba cuando una reducción es una pérdida o depende de suposiciones. Diferentes compiladores se pueden comparar a través de mutaciones semánticas y modelos distintivos.

La séptima etapa es la ejecución formal. Los motores relacionales y Datalog calculan los cierres. Las tablas de decisión identifican filas emparejadas y conflictivas. Los solucionadores de SAT/SMT/CSP encuentran modelos o prueban la inviabilidad bajo sus teorías apoyadas. Los verificadores de modelos evalúan las propiedades temporales. Los ejecutores simbólicos exploran las condiciones del camino y devuelven testigos concretos. Los asistentes de prueba o los probadores de teoremas producen derivaciones. La salida del motor aún no es un veredicto orientado al usuario; es un registro de ejecución con certificado.

La octava etapa es la verificación de repetición. Un verificador separado verifica el testimonio o la prueba contra el problema formal vinculado a la fuente y la teoría publicada. Para un hallazgo de documento, confirma que las observaciones citadas existen, los intervalos coinciden con la revisión de la fuente, los estados satisfacen la regla y la traza de ejecución respalda la reclamación. Para un resultado negativo, confirma que las suposiciones de cobertura y exhaustividad son suficientes. La repetición es valiosa incluso cuando se confía en el solucionador subyacente porque valida la conexión entre la salida del candidato y la política liberada.

Si el análisis abstracto predijo una violación pero la ejecución simbólica no puede darse cuenta, comienza el refinamiento semántico. El sistema extrae el núcleo insatisfiable, los predicados contradictorios o la falla de prueba. Los mapas de origen identifican los tramos relevantes y las opciones semánticas. El LLM recibe una tarea limitada: reconsiderar este enlace de identidad, alcance temporal, mapeo de excepciones o alineación de predicado. Propone candidatos revisados; el materializador y el compilador los revalidan; la ejecución se repite. Se solicita la revisión humana cuando la distinción tiene un alto impacto o permanece sin resolver.

Este bucle debe ser monótono con respecto a la evidencia, no simplemente la confianza. Un refinamiento puede dividir una entidad, reducir un intervalo de tiempo, agregar una excepción pasada por alto o marcar una premisa sin soporte. No debería actualizar una observación propuesta a los hechos porque el nuevo resultado es conveniente. Cada cambio se registra como una nueva revisión semántica. Los caminos anteriores siguen siendo reproducibles.

El noveno artefacto es un resultado canónico. Contiene la regla o la identidad de la consulta, el estado aceptado, los supuestos, las conclusiones de los que deben y pueden, los candidatos rechazados, las incógnitas, la cobertura, la evidencia de la fuente, las referencias de prueba o testigo y los límites de ejecución. También puede incluir un gráfico de explicación conciso de la máquina comprobable. Este artefacto es autoritario sobre cualquier representación en prosa.

La décima etapa realiza el lenguaje natural. El renderizador recibe solo el resultado canónico y los extractos de origen seleccionados. No vuelve a ejecutar el razonamiento original. Cada frase generada está asociada con una o más afirmaciones de resultados. Un comprobador compara la oración con las afirmaciones permitidas, garantiza que se preserve la modalidad y la incertidumbre, y rechaza las adiciones no admitidas. El resultado puede distinguir los hallazgos, supuestos, interpretaciones y recomendaciones.

Por ejemplo, la respuesta correcta puede ser: “Bajo la interpretación de que ambas referencias a la Junta denotan el mismo órgano, la regla se viola porque la aprobación precede a la revisión de seguridad registrada. El documento no establece si las referencias denotan el mismo órgano; por lo tanto, se requiere la revisión del resultado general.” Un renderizador fluido pero inseguro podría omitir la condición y anunciar una violación. El resultado canónico impide esta simplificación.

La tubería también soporta la generación, no solo el linting. Un usuario provee una idea o un objetivo. Los circuitos de planificación construyen una especificación controlada con testigos de reglas a plan. Un LLM realiza un borrador. Los circuitos de auditoría analizan el borrador utilizando la misma evidencia y la misma ruta de verificación. La revisión continúa hasta que la auditoría canónica satisfaga la política de liberación o el presupuesto se agote. El modelo de generación nunca se convierte en el oráculo final de si su propio texto satisface las reglas.

La gobernanza rodea la cadena de herramientas. Las ontologías, los productores, los circuitos, los compiladores, los contratos de formalismo y los renderizadores tienen versiones y digestos. Los agentes de aprendizaje operan en áreas de puesta en escena desechables. La publicación requiere puntos de referencia, mutaciones semánticas, comprobaciones de compatibilidad y aprobación explícita. La producción lee lanzamientos inmutables. Los perfiles de modelos en vivo se evalúan por separado porque los accesorios deterministas no establecen la calidad del proveedor. Los paquetes de conocimiento del mundo actual tienen fechas y jurisdicciones efectivas.

La arquitectura debe exponer una jerarquía de garantías. Las garantías estructurales se refieren a los lapsos exactos, el ordenamiento y la validez del esquema. Las garantías de conexión a tierra se refieren a la alineación de la evidencia y el estado de identidad. Las garantías de ejecución se refieren a las consecuencias de un problema formal. Las garantías de extremo a extremo las combinan condicionalmente. El usuario debería poder ver la capa más débil. Un resultado derivado de eventos propuestos por modelos y una transición verificada mecánicamente no debe etiquetarse simplemente como “verificado”; debe indicar que la ejecución se verifica sobre las observaciones propuestas.

Un pequeño núcleo de confianza es un objetivo de diseño importante. El kernel incluye canonicalización de origen, comprobación de esquema y tipo, compiladores formales, ejecutores registrados o verificadores de pruebas, verificadores de repetición y validación de resultados canónicos. Los avisos de LLM, los generadores de candidatos, la heurística de recuperación y los modelos de explicación permanecen afuera. Los componentes de confianza aún pueden contener errores, pero su comportamiento es estable, comprobable y revisable.

Esta es una mejora sustancial sobre la confianza en un modelo de base en evolución como un razonador monolítico.

El gasoducto propuesto no es la única implementación posible, sino que capta las obligaciones reveladas por la literatura. Se explica por qué un solucionador es necesario pero insuficiente. Da a la interpretación abstracta y a la ejecución simbólica distintos roles. Es compatible con múltiples lenguajes formales sin abandonar una capa de evidencia común. Permite LLM la creatividad en el descubrimiento, reparación y explicación, preservando la autoridad simbólica sobre la conexión a tierra y los resultados aceptados. Lo más importante es que convierte el fracaso en un artefacto con nombre, un tipo sin soporte, una identidad no resuelta, una cobertura incompleta, una pérdida de compilación, una formalización insatisfecha o una oración sin licencia, en lugar de una alucinación vaga.

7. LongTextJS, CircuitJS y nllAgent: Un estudio de caso crítico

El repositorio público de NaturalLanguageLinterAgent es uno de los pocos intentos de hacer que el límite entre la interpretación del lenguaje y la ejecución de reglas sea explícito en una implementación ejecutable. El proyecto no es un razonador general del lenguaje natural, y su documentación no lo presenta como uno. Es una biblioteca Node.js 22+ ESM y tiempo de ejecución de la línea de comandos con dos modos de producto en lenguaje controlado sobre la misma arquitectura LongTextJS-CircuitJS. El modo de auditoría aplica una teoría versionada a un documento Markdown existente y emite un producto de auditoría canónica. El modo de especificación aplica teorías de planificación a una idea de alto nivel y emite un plan de generación verificado; un modelo de lenguaje opcional puede darse cuenta de que el plan como prosa, pero los circuitos de auditoría sin cambios siguen siendo el oráculo de aceptación [AssistOS-AI 2026].

El repositorio es útil como estudio de caso porque encarna la tesis central de este libro en forma de software: los modelos pueden proponer observaciones semánticas o prosa, pero la autoridad de producción se asigna a versiones inmutables, comprobaciones de compatibilidad explícitas, ejecución de circuitos restringidos, verificadores y artefactos canónicos. El proyecto también mantiene un registro público de asuntos graves. Esa práctica es importante porque una gran superficie de documentación puede hacer que las intenciones arquitectónicas parezcan garantías implementadas. La discusión a continuación distingue lo que se establece operativamente de lo que sigue siendo una hipótesis de investigación.

No se localizó ninguna evaluación independiente revisada por pares o revisión técnica de terceros de nllAgent, LongTextJS o CircuitJS a través de búsquedas de los nombres del proyecto, el nombre del repositorio, la documentación URL y las combinaciones con “revisión”, “evaluación”, “referencia” y “replicación” a partir del 2 de agosto de 2026. Los resultados de la búsqueda llevaron al proyecto en sí en lugar de a una evaluación externa. La sentencia de este capítulo es, por lo tanto, la revisión arquitectónica independiente del autor, basada en el repositorio público, la documentación, los ejemplos ejecutables, el proceso de liberación declarado y el archivo de asuntos graves actuales. La ausencia de una revisión externa no es ni elogio ni crítica; significa que las reclamaciones del proyecto deben permanecer estrechas hasta que se reproduzcan de forma independiente.

7.1 Lo que la arquitectura pública implementa y por qué sus límites importan

El sistema define tres actividades de producción que están deliberadamente separadas. La auditoría de producción lee una versión publicada inmutable, compila un documento en LongTextJS, ejecuta los circuitos de validación de la versión, verifica los hallazgos candidatos y persiste un resultado canónico de CNL/Audit-1 con un informe renderizado. La especificación de producción compila una idea en observaciones LongTextJS, aplica circuitos de planificación y produce un artefacto canónico CNL/Plan-1; la realización opcional puede generar Markdown y enviarlo a la misma maquinaria de auditoría. El aprendizaje se produce en un espacio de trabajo de puesta en escena desechable. Un agente de codificación puede generar artefactos y puntos de referencia de creación de candidatos, pero los controles del lado del host admiten solo archivos incluidos en la lista blanca, y un mantenedor debe publicar y activar explícitamente una versión. Esto hace que el aprendizaje sea análogo al desarrollo de software gobernado en lugar de la automodificación en tiempo de ejecución.

La unidad de trabajo más importante no es el informe final, sino el directorio de ejecución. Una auditoría conserva la entrada copiada, el programa fuente, la base seleccionada, los resultados de compatibilidad y cobertura, la traza de circuito, los resultados del verificador, los hallazgos, el informe canónico legible por máquina y la vista legible por humanos. Las ejecuciones de planificación preservan los artefactos del plan correspondiente. Una carrera que se detiene porque falta conocimiento sigue registrando el fracaso y puede crear un problema reutilizable. La regla de diseño de que la falta de conocimiento nunca se reporta como cumplimiento es fundamental. Evita un fallo común de los agentes de documentos: traducir “No encontré evidencia” en “el documento satisface la regla”.

LongTextJS debe entenderse como un programa del lado de la fuente solo en un sentido restringido. No asigna comportamiento imperativo a las oraciones arbitrarias. Recopila una revisión de fuente fija en un mundo finito y cuestionable de estructuras y observaciones. Las propiedades valiosas son la identidad de origen, los intervalos estables, los tipos de observación versionados, la identidad del productor, la cobertura, las brechas explícitas y la rejugabilidad. Un artefacto del lado del documento se puede ejecutar porque los consumidores registrados pueden consultar y transformar estas relaciones bajo la semántica declarada; está más cerca de una representación intermedia semántica o programa de carga de base de datos que del código de aplicación no restringido.

Esta distinción es más que terminología. Cuando un modelo traduce una oración, su salida no debe volverse autoritaria simplemente porque está serializada en un lenguaje de programación. El texto fuente sigue siendo evidencia; la observación traducida registra una interpretación de esa evidencia, generada por un productor nombrado bajo un esquema y perfil de modelo. El circuito sigue siendo una teoría separada. Un hallazgo se puede publicar solo después de que un verificador comprueba las condiciones esperadas por el lanzamiento. Estas capas impiden que un analizador se convierta silenciosamente en juez y evitan que una explicación se convierta en el único registro de una inferencia.

El paquete de cimentación actual ilustra el alcance semántico previsto. Los comandos ordinarios de auditoría, plan y punto de referencia seleccionan un foundation-core@1.1.0 limitado a menos que se deshabilite explícitamente. El paquete reconoce las formas documentadas de inglés controlado para el estado, el tipo, la precedencia temporal exacta expresada por “antes”,

la aritmética exacta, la cantidad y las afirmaciones de emoción literal. Cinco circuitos verificados de repetición comprueban solo los invariantes lógicos, temporales, aritméticos, físicos elementales y de tipo lógicos. El paquete y la digestión seleccionados persisten. Los hechos políticos, geográficos, sociales, económicos y legales cambiantes están intencionalmente excluidos de la verdad de la fundación atemporal. Este es un límite de modelo mundial defendible. Un sistema que codifica los hechos actuales sin fechas efectivas, jurisdicciones, fuentes y política de conflicto no puede ofrecer una ejecución semántica reproducible.

El sistema soporta la materialización determinista y asistida por modelo. La selección de la traducción puede preferir un agente de modelo de lenguaje AchillesagentLib configurado, usar el adaptador de agente de codificación restringido actual o ejecutar sin un modelo para agentes deterministas. Cada llamada de modelo recibe una habilidad específica de rol en lugar de todo el espacio de trabajo del agente. Si una observación crítica no se puede materializar dentro del esquema requerido y el presupuesto de recursos, el resultado es incompatible, incompleto o agotado. El modelo no controla la construcción de la fuente, la política de publicación o el significado de un verificador. Esta es precisamente la dirección correcta para un sistema neuro-simbólico federativo: el modelo es poderoso en el borde, mientras que los artefactos simbólicos y la aceptación del control de la política del anfitrión.

La cobertura se trata como una obligación ejecutable en lugar de una puntuación de confianza informal. La compilación de circuitos funciona hacia atrás desde nodos de salida alcanzables para determinar qué observaciones externas pueden influir en un resultado. Los registros de publicaciones vinculan esas demandas a los productores disponibles. La compatibilidad en tiempo de ejecución pregunta entonces si el documento concreto realmente arrojó observaciones con el tipo, estado, cardinalidad, estructura, canal y cobertura requeridos. La capacidad declarada de un productor no se confunde con la evidencia de que el documento actual se haya procesado con éxito. Cuando no se satisface una demanda crítica, la carrera se detiene en un límite con nombre.

Este diseño impulsado por la demanda es muy adecuado para documentos largos. Evita pedir un modelo para producir una interpretación multiuso de cada oración antes de que se conozca cualquier pregunta. Un circuito puede solicitar las dimensiones semánticas que necesita: términos exactos, afirmaciones de estado, cantidades, relaciones temporales u observaciones de dominio. El compilador sólo puede materializar observaciones relevantes para la producción y persistir en lo que estaba y no estaba cubierto. La ventaja científica no es solo un menor costo. Limita los compromisos semánticos innecesarios y hace visible la formalización incompleta.

CircuitJS es la representación del lado teórico. Los módulos de circuito editables por agente están restringidos a una expresión declarativa, se normalizan a datos de forma plana y se pasan a través del mismo compilador utilizado para descripciones de circuitos no de código. Solo pueden invocar operadores y verificadores registrados; no pueden ocultar el comportamiento ejecutable arbitrario dentro de una regla aprendida. Una aplicación host puede instalar una extensión de tiempo de ejecución revisada que contiene algoritmos reales, pero las entradas y el contexto son copias de datos sin formato congelados, los resultados deben ser datos sin formato, la identidad de caché incluye el resumen de extensión y las versiones que dependen del bloqueo de extensión que digieran. La interfaz de línea de comandos estándar no acepta rutas de extensión ejecutables arbitrarias.

Este límite aborda uno de los riesgos prácticos más graves en los sistemas de reglas “aprendidos por agentes”. Si un agente de codificación puede generar JavaScript arbitrario en cada circuito, aprender una línea se convierte en generación remota de código y el comportamiento semántico de una versión se hace difícil de inspeccionar. La restricción de la creación de circuitos ordinarios a las operaciones registradas crea un vocabulario operativo finito. Los algoritmos especializados siguen siendo posibles, pero ingresan a través de una ruta de ingeniería, revisión y versionado separada. Por lo tanto, la arquitectura distingue la evolución de las teorías de la evolución del tiempo de ejecución de confianza.

El límite del verificador es igualmente importante. La ejecución del circuito puede crear hallazgos de candidatos, pero un verificador controla si esos candidatos ingresan a la auditoría canónica o al plan. En los ejemplos limitados, el verificador puede releer los anclajes de fuente, observaciones, metadatos de liberación y la política que se está aplicando. No es suficiente que un nodo de ejecución emita un objeto cuyos campos parezcan una advertencia. El verificador establece la propiedad específica que espera el producto. Esto no demuestra automáticamente la interpretación semántica correcta, pero detiene la salida de circuito accidental o malicioso de hacerse pasar por un hallazgo aceptado.

El modo de especificación demuestra cómo la misma arquitectura puede soportar la generación controlada sin otorgar autoridad al modelo de escritor. Los circuitos de planificación construyen un plan canónico y requieren de reglas para planificar las reglas aplicadas. La realización opcional transforma el plan en prosa. La prosa candidata pasa entonces por el modo de auditoría, y el modelo de realización no juzga su propio cumplimiento. Esto es más fuerte que la generación limitada solo porque el objeto autoritario es el plan y el rastro de auditoría, no la adherencia autoinformada del modelo.

El subconjunto experimental de consulta primero también es significativo. Canonical LongTextJS está expuesto como un mundo de consulta de solo lectura finito. Las consultas escritas y los valores de tabla conscientes de la evidencia se reducen a la ejecución ordinaria de CircuitJS. El compilador verifica las políticas de relaciones, expresiones, anti-uniones, pedidos y golpes; registros de publicaciones consulta contratos y mapas de origen; los puntos de referencia comparan la evaluación de referencia directa con la ejecución reducida. La versión editorial de producción actual sigue siendo deliberadamente más simple, mientras que los agregados, los patrones ordenados, los índices nativos y el razonamiento avanzado se manejan a través de operadores directos o gráficos. Esta restricción es preferible a reclamar un lenguaje de consulta universal maduro antes de que la semántica y la optimización sean estables.

El modelo de publicación del repositorio refuerza la reproducibilidad. Los agentes nombrados poseen material de autoridad separado, esquemas, perfiles de extracción, circuitos, suites de referencia, candidatos, lanzamientos, ejecuciones, problemas y comentarios. Un agente de codificación puede proponer candidatos, pero no puede hacerlos activos. La producción lee un lanzamiento inmutable. Esta estructura permite reproducir qué teoría, base, perfil de productor, digestión de extensión y verificador generó un resultado. También permite que varios agentes de codificación o familias modelo construyan teorías competitivas sin compartir silenciosamente una ontología o liberación activa.

La afirmación más fuerte implementada es, por lo tanto, arquitectónica más que semántica: nllAgent tiene un camino disciplinado desde la fuente inmutable hasta las observaciones versionadas, desde la demanda de observación hasta la compatibilidad, desde los circuitos

restringidos hasta la publicación controlada por verificador, y desde un producto canónico hasta un informe legible por humanos. Tiene estados de parada explícitos. No necesita pretender que cada documento sea ontológicamente compatible. Trata los resultados de los modelos como propuestas consolidadas. Estos son logros sustanciales en comparación con las tuberías convencionales de agente LLM, donde la pronta interpretación, el razonamiento y la explicación a menudo existen solo en un contexto transitorio.

Sin embargo, la arquitectura no establece que el lenguaje natural arbitrario se haya basado correctamente. Los tramos estables prueban que la evidencia existe en un lugar; no prueban que un actor, evento, excepción o modalidad se interpretó correctamente. Una observación mecanografiada todavía puede referirse a la entidad equivocada. Una carrera completa puede estar completa solo para el contrato de observación limitado, no por cada significado relevante que un humano pueda inferir. La propia documentación del proyecto y el registro de emisiones son apropiadamente cautelosos en este punto. El valor de la implementación es que estas limitaciones tienen lugares que representar en lugar de estar ocultos dentro de una respuesta final.

7.2 Evaluación independiente: fortalezas, problemas graves actuales y la evolución requerida

Mi evaluación general es positiva sobre los límites de la confianza y cautelosa sobre las afirmaciones semánticas. nllAgent es más que un envoltorio rápido, más que una tubería de recuperación de vectores y más que una colección de esquemas JSON. Sus artefactos del lado fuente y de la teoría, publicación inmutable, límite de autor restringido, verificación de repetición, comprobaciones de cobertura y productos canónicos constituyen un sustrato de investigación creíble. La arquitectura está especialmente bien alineada con el patrón federativo identificado en las revisiones neurosimbólicas recientes: el componente neuronal propone o realiza el lenguaje, mientras que el componente simbólico conserva su propia identidad, semántica y criterios de aceptación [Chatzikyriakidis and Lappin 2026].

La evidencia actual del proyecto, sin embargo, apoya la viabilidad limitada, no el lenguaje natural general ejecutable. La base por defecto reconoce un pequeño fragmento controlado. La demostración editorial utiliza operadores y verificadores estrechos. La compilación de Consulta primero es experimental. La realización controlada es limitada. Ninguna evaluación independiente ha establecido la fidelidad de formalización, la resolución de identidad, la transferencia a través de dominios, la calibración de errores o la superioridad sobre las líneas de base más simples. Por lo tanto, la estrategia de publicación correcta consiste en indicar exactamente qué propiedades se implementan y adjuntar cada reclamo más fuerte a un punto de referencia y una envolvente de garantía.

El actual registro de cuestiones graves identifica seis limitaciones de ingeniería y semántica. La primera es la incrementalidad del planificador limitado. El tiempo de ejecución tiene mecanismos útiles de dependencia y almacenamiento en caché, pero aún no reclama un motor de punto fijo incremental completamente general para circuitos arbitrarios, relaciones recursivas, mundos alternativos y actualizaciones no monotónicas. Esto es importante para documentos largos porque una pequeña edición debe invalidar solo los artefactos semánticos y los estados del circuito que dependen de él. Una repetición completa ingenua es correcta pero costosa; un programador incremental poco sólido es rápido pero peligroso. La solución requiere contratos explícitos de monotonidad y efecto, procedencia de dependencia, separación de época para retracciones y comparaciones de referencia entre ejecución incremental y completa.

El segundo problema es que el soporte de interpretación aún no fluye uniformemente a través de cada ruta de ejecución. Un documento puede contener lecturas competitivas, mundos, contextos o condiciones de presencia, pero un operador, una clave de caché, una agregación o un verificador pueden colapsarlos accidentalmente. Este es el obstáculo central para tratar LongTextJS como un estado semántico abstracto. Todo valor que depende de una interpretación debe llevar su contexto. Las uniones deben combinar contextos compatibles. La agregación debe decir si varía dentro de un mundo o a través de alternativas. La identidad de caché debe incluir la fórmula del contexto. Los operadores que no pueden preservar alternativas deben ser declarados incompatibles en lugar de elegir silenciosamente una.

Este problema no es una brecha menor de características. La promesa científica central del lenguaje natural ejecutable es a menudo que el sistema puede decir lo que es verdad bajo todas las lecturas admisibles, lo que es posible bajo al menos una, y qué resultado depende de una interpretación en disputa. Si se pierden alternativas en una ruta de ejecución interna, una

interfaz pulida puede/debe ser engañosa. Por lo tanto, el proyecto debería hacer que la preservación de la interpretación sea una invariante de tiempo completo y probarla a través de mundos finitos generados, propiedades metamórficas y reproducción diferencial.

El tercer problema es la compatibilidad conservadora para el comportamiento procesal opaco. Las extensiones y operadores registrados son necesarios porque algunos algoritmos útiles son incómodos de expresar de forma declarativa. Pero un compilador no puede inferir demandas, efectos, monotonicidad, uso de recursos u obligaciones de testigos de las obligaciones opacas de JavaScript. El rechazo conservador es más seguro que la aceptación optimista, sin embargo, la opacidad excesiva reduce la composabilidad y hace que la materialización impulsada por la demanda sea menos efectiva. Cada primitiva de procedimiento necesita un contrato de propiedad del registro que declare los tipos de entrada y salida, los estados epistémicos aceptados, los requisitos de cobertura, los posibles efectos, el determinismo, la monotonicidad, el comportamiento de la caché, la clase de costo y el testigo legible por verificador. El contrato debe ser más autorizado que los comentarios escritos por el autor de extensión.

El cuarto problema es la planificación mínima de consultas y la indexación temporal. Un mundo de consulta finito puede ejecutarse correctamente sin una optimización sofisticada, pero el uso a escala de documento requerirá una planificación conjunta, estimaciones de selectividad, índices de fuente y entidad, índices de intervalos temporales y mantenimiento incremental. Estas optimizaciones no son meramente detalles de rendimiento. Un planificador que reescribe las consultas debe preservar la multiplicidad de evidencia, fórmulas de contexto, ordenamiento, semántica anti-unión y cobertura. Los índices nativos deben seguir siendo aceleradores desechables en lugar de fuentes de verdad. Las pruebas diferenciales deben comparar la evaluación optimizada y de referencia sobre los mundos generados, especialmente para las consultas de ausencia, la superposición temporal, las interpretaciones alternativas y la evidencia duplicada.

El quinto problema es la generación controlada deliberadamente estrecha. La realización exacta o verificada de los hallazgos y planes limitados es valiosa, pero aún no equivale a una prosa verificada de formato largo. La generación de larga duración introduce la agregación, los pronombres, la elipsis, el énfasis retórico, las relaciones del discurso y las implicaciones creadas por el orden de la oración. Una realización puede contener solo declaraciones atómicas con licencia y aún implicar una narrativa causal o normativa sin soporte. El siguiente paso debería ser la realización de pruebas en lugar de un escritor más amplio y sin restricciones. Un resultado canónico debe proporcionar identificadores de reclamos, ranuras de entidad y cantidad, modalidad, incertidumbre y relaciones de discurso permitidas. El texto generado debe volver a analizarse y compararse con esas afirmaciones. Los desajustes críticos deben rechazar el texto; el material estilístico fuera del fragmento comprobado debe etiquetarse a un nivel de garantía más bajo.

El sexto problema es el soporte de mutaciones específicos de escenario. La precisión de referencia ordinaria puede ocultar reglas que son inertes, redundantes o incorrectamente al alcance. Un circuito puede pasar todos los ejemplos curados incluso si se ignora una excepción porque ningún caso la activa. Una evaluación sería debería generar mutantes semánticos tipificados que cambien umbrales, comparadores, cuantificadores, negación, modalidad, enlaces de identidad, orden temporal, estado de evidencia, predicados de cobertura, alcance de excepción, prioridad de regla y rutas de verificador. Un conjunto de pruebas "mata" a un

mutante cuando el resultado canónico esperado cambia de la manera requerida. Los mutantes supervivientes revelan pruebas débiles o cláusulas semánticamente redundantes. La mutación debe cubrir programas de origen, teorías de circuitos, reducciones de compiladores y restricciones de realización, no solo cadenas en los ejemplos.

Estos seis números son más informativos que una etiqueta genérica de “prototipo” porque identifican dónde la arquitectura actual puede perder escala o significado. También sugieren un camino de evolución coherente. El primer hito debe ser una semántica formal limitada para el mundo de la tienda y la consulta semántica. No es necesario formalizar el inglés sin restricciones. Debe definir mundos de origen inmutables, identidad de observación, estado epistémico, cobertura, alternativas, transacciones, resultados de consulta, publicación y retractación. Las invariantes clave se pueden verificar en el modelo en instancias finitas y probar en cuanto a propiedades en la implementación.

El segundo hito debe ser la ejecución consciente de la interpretación. Las condiciones de presencia o los contextos factorizados deben acompañar a las observaciones alternativas dependientes. El planificador, el motor de consulta, el registro primitivo, la caché, el verificador y los productos canónicos deben preservar esos contextos o declarar la incompatibilidad. May, debe, no puede, y desconocido debe ser calculado a partir de conjuntos explícitos o abstracciones de mundos admisibles, no a partir de la confianza del modelo. Esto crearía el puente concreto desde la arquitectura existente hasta la interpretación abstracta.

El tercer hito debería ser un registro de formalismo en lugar de un CircuitJS en expansión indefinida. CircuitJS se utiliza mejor como un lenguaje de orquestación, contrato de evidencia y publicación. Los motores especializados deben poseer semántica especializada: evaluación relacional y Datalog para cierres y unidas; tablas de decisión para la precedencia de la política; SAT, SMT, CSP o MaxSAT para restricciones; autómatas para patrones ordenados; solucionadores temporales y verificadores de modelos para rastros; ejecución simbólica para testigos de ruta; marcos de flujo de datos para la propagación. Cada adaptador debe declarar un fragmento de entrada escrito, suposiciones de traducción, certificado de salida, comprobador de reproducción y modos de fallo.

El cuarto hito debe ser el refinamiento semántico guiado por contraejemplo. Un análisis abstracto grueso puede identificar una posible violación. Un solucionador o ejecutor simbólico busca entonces un testimonio coherente. Si el testigo es satisfactible y rejugable, el sistema lo informa con las extensiones de fuente. Si es espurio, un núcleo insatisfrito o conjunto de conflictos debe identificar qué identidad, temporal, modal, cantidad, excepción o distinción de cobertura se perdió. La solicitud de perfeccionamiento debe dirigirse únicamente a la fuente y al productor pertinentes. El artefacto semántico reparado se convierte en una nueva versión y el contraejemplo se convierte en un caso de regresión. Este bucle sería una síntesis genuinamente novedosa de la propuesta semántica basada en LLM, la interpretación abstracta y la ejecución simbólica.

El quinto hito debe ser una capa de conexión a tierra más fuerte. Los anclajes de tipo seguridad y fuente son necesarios pero insuficientes. El tiempo de ejecución necesita conjuntos de identidad de candidatos, restricciones explícitas iguales / diferentes, validez temporal, comprobaciones de unidad y jurisdicción, autoridad de origen y políticas de conflicto. Los enlaces top-one generados por modelos no deben fusionar objetos destructivamente. Las entidades críticas deben utilizar identificadores autorizados o confirmación humana. El conocimiento externo debe ingresar a través de paquetes de solo lectura con contenido con

fuentes, intervalos efectivos, jurisdicción, caducidad, resolución de conflictos y techos de garantía. La recuperación web puede proponer actualizaciones de paquetes, pero no debería convertirse en una verdad atemporal implícita.

El sexto hito debe ser una evaluación independiente. Los accesorios deterministas establecen la reproducibilidad del software, pero no la calidad de la traducción semántica en vivo. Un perfil de evaluación opt-in debe fijar el modelo, el proveedor, las instrucciones, las versiones de la herramienta, la temperatura y la política de captura; repetir casos para medir la varianza; registrar el costo y la latencia; y prohibir el retroceso silencioso. Un evaluador de referencia implementado por separado debe comparar los resultados de consulta y circuito cuando sea factible. La anotación humana debe centrarse en la fidelidad de origen a semántica y las ambigüedades relevantes para los resultados. Las líneas de base deben incluir la sentencia directa LLM, la recuperación de LLM más, el formalismo LLM a uno, la compilación de lenguaje controlado y los sistemas de patrones deterministas. El proyecto debe demostrar cuando su arquitectura adicional produce un valor medible.

Se puede resumir una evaluación útil sin inflar las reivindicaciones. La preservación de la fuente, las liberaciones inmutables, la persistencia de rastros, la separación de las observaciones de documentos de la teoría reutilizable, la cobertura explícita, la creación de circuitos restringidos, la publicación controlada por verificador y el aprendizaje gobernado son decisiones de diseño sólidas. La comprensión semántica general, la ejecución completa de la interpretación, la planificación de consultas a escala de producción, la generación de forma larga que lleva la prueba, la mutación integral y la validación empírica independiente permanecen abiertas. La credibilidad de la arquitectura proviene en parte de la admisión de este límite.

La principal oportunidad científica es no añadir más sintaxis DSL. Es demostrar la solidez semántica condicional para un fragmento limitado pero no trivial. Un resultado publicable podría especificar un lenguaje de documento que contiene entidades, eventos, relaciones temporales, cantidades, modalidad, excepciones explícitas y referencias alternativas; definir el conjunto de interpretación admitido; probar o probar mecánicamente que cada resultado publicado puede/debe ser correcto en relación con ese conjunto; mostrar que el refinamiento simbólico elimina una clase medible de advertencias espurias; y verificar que las explicaciones generadas preservan el resultado canónico. Tal resultado sería más estrecho que el “inglés ejecutable”, pero sustancialmente más importante.

La principal oportunidad práctica es una nueva clase de sistemas de documentos y agentes que están dispuestos a detenerse. Un asistente normal LLM está optimizado para continuar produciendo una respuesta. Un tiempo de ejecución simbólicamente fundamentado puede devolver la ontología incompatible, la identidad no resuelta, la cobertura insuficiente, el formalismo sin apoyo, el agotamiento presupuestario, el contraejemplo espurio o la realización sin licencia. Estos resultados pueden parecer menos impresionantes en una demostración, pero son requisitos previos para un uso confiable en la ciencia, la regulación, la ingeniería y la toma de decisiones institucionales.

Por lo tanto, mi juicio independiente es que nllAgent ha elegido muchos de los límites arquitectónicos correctos y debería resistir la presión para ampliar sus afirmaciones más rápido de lo que amplía su evidencia semántica. Ya es un prototipo creíble de análisis de lenguaje gobernado, basado en la fuente y separado por la teoría. Todavía no es evidencia de que el lenguaje natural sin restricciones pueda ser ejecutado con garantías formales. Su mejor camino

a seguir es convertirse en una plataforma experimental para la afirmación más estrecha y defendible desarrollada a lo largo de este libro: LLMs descubrir candidatos formales, una base y consecuencia simbólica de controles del kernel, y la producción de lenguaje natural permanece subordinada a un resultado canónico rejugable.

8. Construcción y validación de un runtime neuro-simbólico

8.1 Estrategia de implementación: un sistema de lenguaje ejecutable mínimo pero científicamente honesto

La arquitectura propuesta en este libro es ambiciosa solo si se describe vagamente. Una vez que se separan sus obligaciones, el primer sistema implementable es comparativamente modesto. No necesita entender el lenguaje arbitrario, construir una ontología universal o probar cada conclusión. Debe aceptar una clase de documento limitada, preservar la evidencia exacta, materializar una pequeña familia de observaciones mecanografiadas, compilar una familia restringida de reglas en uno o dos motores formales, devolver testigos o incógnitas explícitas, y evitar que la capa final de lenguaje natural cambie el resultado simbólico. El valor científico proviene de hacer cada limitación visible y medible.

La primera decisión de diseño es la reclamación objetivo. Un prototipo de investigación no debería comenzar con “el sistema entiende y ejecuta el lenguaje natural”. Debería comenzar con una afirmación como: documentos dados de un género declarado, una ontología versionada, una familia finita de productores de observación y un conjunto de reglas publicadas, el tiempo de ejecución determina si cada regla está satisfecha, violada, potencialmente violada, no soportada o fuera de la cobertura semántica del comunicado. El resultado es rejugable y está vinculado a los lapsos de origen. Esta afirmación es lo suficientemente estrecha como para probar y lo suficientemente amplia como para respaldar la revisión del cumplimiento, el análisis de requisitos, el linding de documentos científicos, la inspección de protocolos y la planificación controlada.

La capa fuente debe ser deliberadamente aburrida. Cada documento se almacena como una revisión inmutable con un resumen estable. Las unidades estructurales: encabezados, párrafos, oraciones, elementos de lista, celdas de tabla, subtítulos, notas al pie y objetos incrustados, reciben identificadores persistentes. La normalización se registra en lugar de aplicarse silenciosamente. Los desplazamientos de caracteres deben referirse a una representación de fuente canónica, mientras que las asignaciones conservan las posiciones en el archivo original. Una declaración en una tabla y una oración visualmente idéntica en un párrafo pueden tener diferentes funciones estructurales; esa distinción debe sobrevivir a la extracción. Cuando OCR es inevitable, la salida OCR y la imagen de la página son objetos de evidencia separados, y la incertidumbre en la transcripción no se contrae en el texto ordinario.

Esta disciplina de origen no es la sobrecarga administrativa. Es lo que hace que las conclusiones simbólicas sean impugnables. Un hallazgo que cita solo un resumen generado no se puede verificar de forma independiente. Un hallazgo vinculado al texto exacto, las celdas de tabla y los registros de transformación se puede revisar incluso si cada modelo de lenguaje involucrado ha cambiado. Por lo tanto, el paquete fuente debe considerarse parte del registro de auditoría de confianza, aunque los extractores que lo crean siguen siendo falibles y requieren pruebas específicas de formato.

La segunda capa es un gráfico de evidencia en lugar de un solo análisis. El gráfico contiene menciones, entidades, eventos, estados, cantidades, expresiones temporales, modalidad, relaciones y enlaces de origen. De manera crucial, también representa alternativas y estatus.

Un enlace de entidad propuesto por modelo no se almacena como una igualdad incondicional. Es una relación de identidad candidata con un productor, puntaje, lapsos de apoyo, evidencia conflictiva y un estado epistémico como propuesto, corroborado, aceptado, rechazado o no resuelto. Una relación temporal inferida de “poco después” no debe normalizarse a una duración exacta arbitraria; debe conservar la relación cualitativa y cualquier interpretación de dominio declarada.

Esta capa debe permanecer finita y limitada a la tarea. El sistema no está construyendo un modelo mundial completo. Se materializa las distinciones exigidas por los circuitos publicados y por la consulta actual. La materialización de query-first, ya explorada en nllAgent, es el valor predeterminado correcto. Un circuito declara que necesita identidad de actor, relaciones de orden de documentos, modalidad deontic y fechas efectivas. El compilador deriva esas demandas antes de una interpretación costosa. Los productores generan solo las observaciones necesarias para decidir la compatibilidad y ejecutar la regla. Esto reduce el costo y, lo que es más importante, limita la superficie semántica en la que pueden ocurrir errores.

La tercera capa es un núcleo de conexión a tierra simbólico. “Junción” se utiliza a menudo libremente para significar que una respuesta cita un pasaje. Aquí tiene un significado más estricto: cada símbolo admitido a la ejecución formal tiene un referente, procedencia, alcance y estatus tipificados suficiente para el formalismo objetivo. Un predicado como aprobado (x, y, t) no puede ser introducido simplemente porque un LLM produjo esos tokens. El kernel comprueba que x e y se refieren a tipos de entidad admisibles, que t es un valor o intervalo de tiempo soportado, que el productor de observación está autorizado para este predicado, que el intervalo de origen existe en la revisión declarada, y que el estado cumple con el requisito de garantía de la regla.

Los contratos de base deben distinguir la evidencia de la ontología. La evidencia dice que un lapso apoya una observación. La ontología dice qué tipo de cosas denota la observación y qué relaciones son admisibles. Los axiomas de dominio dicen lo que se desprende de esas relaciones. El conocimiento mundial proporciona reclamaciones de origen externo. Estas capas no deben fusionarse. Un registro de la empresa puede establecer que dos nombres denotan una entidad legal durante un período establecido. Una ontología de dominio puede declarar que las personas jurídicas pueden firmar acuerdos. Una teoría de política puede declarar que una firma de un representante autorizado cuenta como aprobación. Un modelo puede sugerir las tres conexiones, pero cada conexión pertenece a una autoridad diferente y debe ser aceptada o rechazada por separado.

La identidad merece su propio servicio porque es la dependencia oculta más común en el razonamiento de documentos largos. El servicio debe administrar clústeres de menciones, alias explícitos, identificadores, roles organizacionales, validez temporal y mundos seleccionados. Debería permitir varios candidatos no resueltos en lugar de forzar la unificación temprana. En una primera versión, la identidad puede ser intencionalmente conservadora: se aceptan identificadores explícitos y alias declarados exactos; se proponen enlaces contextuales y basados en roles; la identidad de documentos cruzados no compatible detiene las reglas que lo requieren. Esta política conservadora reducirá la cobertura aparente, pero mejorará en gran medida el significado de los resultados aceptados.

El tiempo necesita un dominio igualmente explícito. Una representación inicial práctica puede combinar instantes exactos, intervalos cerrados o abiertos, orden cualitativo y límites

desconocidos. El tiempo de ejecución debe distinguir el orden del documento del orden del evento y el tiempo de publicación del tiempo efectivo. Frases temporales como “antes de la implementación”, “dentro de los treinta días”, “mientras la autorización sigue siendo válida” y “después de cualquier cambio material” inducen diferentes estructuras. El compilador debe rechazar una regla temporal si el gráfico de evidencia solo tiene observaciones de orden de documento cuando se requiere semántica de tiempo de evento. La sustitución silenciosa entre dominios temporales es una fuente clásica de conclusiones plausibles pero inválidas.

La modalidad y la fuerza normativa requieren una pequeña red en lugar de una booleana. Como mínimo, el sistema debe distinguir la afirmación, el informe, la creencia, la posibilidad, la capacidad, el permiso, la obligación, la prohibición, la recomendación, la intención, la condición y el contrafáctico. También debe registrar al titular y beneficiario de una norma cuando proceda. “El proveedor puede divulgar datos” y “el proveedor debe revelar datos” compartir vocabulario, pero tiene consecuencias de cumplimiento opuesto. “El documento establece que el proveedor cumplió” es evidencia de un informe, no necesariamente evidencia del evento subyacente. Un tiempo de ejecución formal se vuelve útil precisamente al rechazar estos colapsos.

Una vez que existen observaciones fundamentadas, la cuarta capa calcula un estado semántico abstracto. Un dominio mínimo puede representar, para cada proposición relevante para la consulta, si es verdadera en todas las interpretaciones admisibles, verdadera en algunas, falsa en total, contradicha o no resuelta debido a la falta de cobertura. La implementación no tiene que comenzar con una sofisticada biblioteca de celosía. Puede comenzar con alternativas finitas explícitas y un resumen de may/most de derivación mecánica. A medida que crece la escala, los dominios abstractos pueden resumir candidatos a la identidad, rangos numéricos, intervalos temporales, estados modales y conjuntos de procedencia sin enumerar todos los mundos.

El estado abstracto realiza dos trabajos. En primer lugar, proporciona una propagación conservadora en todo el documento. Si cada interpretación aceptada vincula una definición a un término, esa relación es un hecho imprescindible. Si solo una de las tres asignaciones de identidad plausibles crea un conflicto de intereses, el conflicto es un hallazgo de lata en lugar de una violación verificada. En segundo lugar, decide dónde está justificada la ejecución más cara. Un resultado seguro de las cosas debe terminar temprano. Un resultado de la violación de la maya desencadena una búsqueda de testigos. Un desconocido causado por la ausencia de cobertura del productor desencadena un problema de compatibilidad semántica en lugar de una llamada de solucionador.

La quinta capa es un plan de formalismo. El plan no es una prosa de forma libre escrita por un agente. Es una decisión de compilación mecanografiada que establece qué pregunta se hará de qué motor, qué observaciones son las entradas, qué garantías devuelve el motor y cómo su resultado se asigna al vocabulario de resultados canónicos. La elección debe ser impulsada por la forma semántica en lugar de la moda.

La evaluación relacional es el primer motor a implementar porque muchas comprobaciones útiles de documentos reducen a uniones, comparaciones de pedidos, agrupaciones y diferencias establecidas. Las reglas como “cada término técnico definido se usa de manera consistente”, “cada reclamo aceptado tiene al menos una fuente”, “ningún identificador de requisito se duplica” o “todos los riesgos enumerados tienen un propietario de mitigación” no requieren un probador de teorema. Un núcleo relacional es transparente, eficiente y fácil de probar. Datalog extiende este núcleo cuando se requiere cierre transitivo o dependencias

recursivas. Los ejemplos incluyen herencia, propagación parcial, cadenas de dependencia y accesibilidad a través de gráficos de citas o requisitos.

Los cuadros de decisión son el segundo formalismo útil para las políticas limitadas. Una tabla de decisiones hace que las combinaciones de condiciones y los resultados sean explícitos, expone filas superpuestas o faltantes, y admite el análisis de cobertura. Es especialmente apropiado para la elegibilidad, el enrutamiento, la clasificación y las reglas de política cuya complejidad proviene de combinaciones en lugar de recursión. Un circuito puede compilar una política de lenguaje natural en una tabla candidata, pero la publicación debe requerir ejemplos a nivel de fila, detección de superposición, comprobaciones de integridad y aprobación humana de la partición semántica.

Los motores SAT, SMT, CSP y MaxSAT se vuelven útiles cuando la tarea contiene restricciones de interacción, aritmética, horarios, asignaciones u optimización. El compilador debe elegir teorías apoyadas y evitar ocultar la semántica no admitida en cadenas no interpretadas. Un resultado SMT puede demostrar infasibilidad bajo restricciones codificadas, producir un modelo satisfactorio o exponer un núcleo insatisfiable. El núcleo debería ser asignado a las obligaciones y supuestos de las fuentes. Este mapeo es más informativo que pedir a un LLM que explique un error genérico de solucionador; identifica los compromisos formales más pequeños que no pueden sostenerse conjuntamente.

La lógica temporal y la comprobación de modelos son apropiadas para procedimientos, flujos de trabajo, protocolos y reglas del ciclo de vida. El gráfico de evidencia proporciona eventos y proposiciones de estado; el compilador construye un sistema de transición y una propiedad temporal; el verificador devuelve satisfacción, una traza de contraejemplo o un resultado no concluyente bajo límites. La parte difícil no es el comprobador temporal. Es decidir si la fuente define un sistema de transición completo, si las transiciones omitidas son imposibles o simplemente indocumentadas, y si la equidad o las suposiciones ambientales están justificadas. El resultado canónico debe, por lo tanto, exponer el límite del modelo.

La ejecución simbólica es apropiada cuando una regla se refiere a rutas factibles a través de código ejecutable, un procedimiento de estado o un artefacto semántico similar a un programa. Su producción debe ser un testigo concreto: asignaciones, condiciones de la rama, transiciones estatales y enlaces de origen. En entornos solo para documentos, la ejecución simbólica puede explorar enlaces alternativos o ramas de procedimiento, pero no debe invocarse simplemente porque el sistema contiene símbolos. El término debe conservar su significado sensible a la trayectoria. SymGPT es instructivo porque la regla del lenguaje natural se compila en una condición de violación y luego se prueba contra las rutas de contrato reales en lugar de contra la prosa más generada [Xia et al. 2026].

La lógica natural, los autómatas finitos, el análisis de gráficos, la computación de seminerva y los probadores de teoremas especializados pertenecen a la cartera cuando sus suposiciones estructurales encajan. La lógica natural puede manejar eficientemente la monotonidad y ciertos pilares léxicos cerca del lenguaje de la superficie. Los autómatas son excelentes para los patrones locales ordenados. Los métodos de gráficos y semiling pueden mantener múltiples análisis con el cálculo compartido. Los asistentes de prueba son apropiados cuando un pequeño número de reclamos de alto valor justifican la formalización manual y las pruebas verificadas por el kernel. La arquitectura debería hacer que la adición de un motor sea una extensión escrita, no una invitación a incrustar código ejecutable arbitrario en los circuitos.

La sexta capa es un refinamiento semántico guiado por contraejemplo. Un hallazgo de maya se compila en un problema de viabilidad. Si el solucionador devuelve a un testigo, el tiempo de ejecución lo repite contra el gráfico de evidencia e informa las opciones de interpretación exactas requeridas. Si la ruta es inviable, el motor devuelve un núcleo insatisfiable, una condición previa fallida o un conjunto de conflictos. El sistema rastrea ese objeto a través de la compilación a las observaciones de la fuente y los mapeos de candidatos que lo produjeron. Solo entonces se le pide ayuda a un LLM.

El aviso de reparación debe ser local y escrito. Puede preguntar si dos menciones fueron incorrectamente unificadas, si una condición fue prevista como necesaria o suficiente, si se aplica una excepción o si una relación temporal tiene el criterio de valoración incorrecto. Debe presentar los tramos de origen relevantes, los contratos de ontología, las alternativas actuales y el conflicto simbólico. El LLM propone uno o más artefactos semánticos revisados. Esas propuestas pasan por la misma base y comprobaciones de compilación que los originales. Un ser humano puede ser insertado cuando la ambigüedad es normativa o científicamente material.

Este proceso es fundamentalmente diferente de la autorreflexión genérica. Al modelo no se le dice que su respuesta fue incorrecta e invitada a improvisar otra cadena de pensamiento. Se le da un conflicto identificado mecánicamente en un artefacto semántico persistente. La reparación puede ser probada por regresión. Si la reparación cambia una ontología o productor, la publicación requiere nuevos casos de referencia y un diff semántico. El contraejemplo se convierte en una parte permanente del corpus de evaluación.

La séptima capa es el resultado canónico. Debe ser un producto estable e independiente del solucionador que separe el veredicto, la evidencia, los supuestos, las interpretaciones, la ejecución y las limitaciones. Para una regla de auditoría, registra la versión de la regla, el alcance aplicable, el estado de compatibilidad, la cobertura, las observaciones aceptadas, las observaciones rechazadas, el resultado de la ejecución, la referencia de testigo o prueba, el resultado del verificador y los problemas no resueltos. Para un plan, registra objetivos, restricciones, pasos, dependencias, recursos, alternativas y las comprobaciones exactas que satisface el plan. El producto canónico es la autoridad para cualquier respuesta posterior en lenguaje natural.

La ruta de retorno al lenguaje debe ser tratada como compilación, no como generación libre. Un renderizador recibe un conjunto finito de reclamaciones con licencia. Cada reclamo tiene modalidad, polaridad, cantidad, referencias de entidades, incertidumbre y procedencia. El representador puede elegir el orden del discurso y la fraseo explicativo, pero no puede introducir nuevas entidades, fortalecer el poder debe, convertir la evidencia faltante en ausencia, suprimir una suposición material o inventar una explicación causal. Los identificadores de reclamaciones a nivel de sentencia deben permanecer disponibles para la auditoría. Un segundo comprobador debe comparar el texto renderizado con el resultado canónico, utilizando pruebas deterministas para nombres, cantidades, fechas, negación y modalidad y, cuando sea necesario, un modelo separado limitado a la implicación textual en lugar de la evaluación abierta.

La realización con pruebas restringidas es especialmente importante porque los usuarios juzgarán el sistema por su prosa. Un rastro simbólico perfecto puede verse socavado por un resumen elegante pero inexacto. Por lo tanto, la arquitectura debería hacer que la verbosidad sea opcional pero obligatoria. En el modo de alta garantía, el sistema puede generar un

lenguaje sencillo y repetitivo porque cada oración sigue una plantilla con licencia por el tipo de resultado. En el modo explicativo, un LLM puede mejorar la legibilidad, pero las oraciones no admitidas son rechazadas y regeneradas. El artefacto canónico, no la prosa, sigue siendo el registro de decisión.

La seguridad y la gobernanza deben integrarse desde el principio. Los documentos en lenguaje natural son entradas no confiables y pueden contener inyección rápida, ejemplos adversarios, formato engañoso o fragmentos de código. Los productores semánticos deben recibir datos de origen a través de interfaces mecanografiadas en lugar de un mensaje de sistema privilegiado que pueda sobrescribirse por contenido del documento. Los artefactos formales generados deben ser analizados por gramáticas restrictivas. Los solucionadores deben correr bajo los límites de recursos. Las extensiones de tiempo de ejecución deben revisarse, versionarse y cerrarse con la pérdida de digestión. Los espacios de trabajo de aprendizaje no deben publicar directamente en la producción. Estos límites ya están parcialmente presentes en nllAgent y deben reforzarse en lugar de pasarse por alto por conveniencia [AssistOS-AI 2026].

Un programa de ejecución realista puede proceder en cinco lanzamientos. La primera versión debe admitir Markdown inmutable o ingestión de DOCX, lapsos de origen, observaciones estructurales, un motor de consulta relacional, observaciones restringidas en inglés controlado, estados de cobertura, circuitos reutilizables, verificación de repetición y hallazgos canónicos. Debe resolver tareas limitadas como la integridad de la procedencia, la consistencia de la definición, la cobertura de requisitos, las reglas de la sección ordenada y las distinciones modales simples. No se debe reclamar ninguna identidad general, inferencia temporal o recuperación del mundo actual.

La segunda versión debe agregar identidad y tiempo de tipo. La identidad comienza con identificadores explícitos, alias y roles declarados por documentos; la semántica temporal comienza con fechas exactas, intervalos, orden de documentos y un pequeño conjunto de patrones de orden de eventos. Los puntos de referencia deben contener alias ambiguos, roles cambiantes, intervalos superpuestos y enlaces deliberadamente perdidos. El sistema debe detenerse en lugar de adivinar cuando un circuito requiere una identidad más fuerte de la que proporciona la liberación.

La tercera versión debería agregar tablas de decisión y ejecución SMT/CSP, con mapeo de núcleo insatisfiable y refinamiento semántico guiado por contraejemplo. Esta versión puede abordar las políticas de elegibilidad, los horarios, las restricciones de asignación, la consistencia cuantitativa y la viabilidad del procedimiento limitado. Cada resultado de solucionador debe reproducirse a través de un verificador independiente o un comprobador más pequeño cuando sea posible. Las pruebas de mutación deberían alterar las restricciones y observaciones para demostrar que las suites de referencia detectan regresiones semánticas.

La cuarta versión debe agregar un adaptador de verificación temporal/de modelo y un adaptador de ejecución simbólica sensible a la ruta para un dominio concreto, como especificaciones de flujo de trabajo o reglas de contrato inteligente. El propósito no es una cobertura amplia, sino una prueba de que el registro de formalismo y la interfaz de certificado canónico admiten una semántica genuinamente diferente. Las pruebas de consistencia de formalismo cruzado deben verificar que los hechos fundamentados compartidos retengan la identidad, el tiempo, la modalidad y la procedencia en todas las reducciones.

La quinta versión debería agregar planificación y realización controlada. Una idea se compila en un plan canónico, se compara con las restricciones formales, se realiza en prosa y se audita por el mismo sistema de reglas. La evaluación debe distinguir la validez del plan, la fidelidad de realización, la calidad lingüística y la eficiencia de la reparación. El plan no se vuelve confiable simplemente porque la prosa se generó a partir de un objeto estructurado; se vuelve más auditable porque el objeto y sus controles son persistentes.

Artifact stack for a replayable executable-language runtime

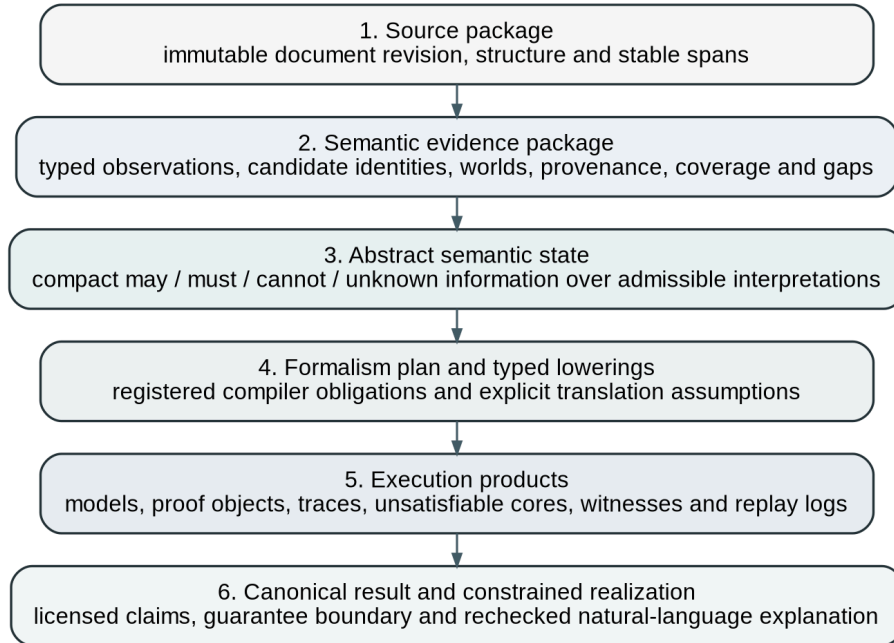


Figura 4. Artifact stack para un tiempo de ejecución ejecutable-natural-en inglés rejugable. Cada capa está versionada, mapeada en la fuente e inspeccionable de forma independiente.

Esta implementación por etapas encarna un principio metodológico importante: la ambición semántica debería crecer solo cuando el sistema pueda detectar que su representación anterior es inadecuada. El tiempo de ejecución no debería ocultar el lenguaje no soportado detrás de la confianza del modelo. Debería exponer las brechas de compatibilidad como resultados de primera clase y convertir las brechas recurrentes en nuevos productores, ontología, motor y trabajo de referencia. El lenguaje natural ejecutable avanza a través de la expansión gobernada de una envoltura de solidez, no a través de un salto a la comprensión universal.

8.2 Preguntas de investigación, puntos de referencia y un programa de publicación

La arquitectura se convierte en una contribución científica solo cuando sus afirmaciones son refutables. Una demostración en la que un LLM genera un artefacto formal plausible y un solucionador devuelve una respuesta es insuficiente. El mismo modelo puede generar el punto de referencia, la traducción y la explicación, creando un bucle cerrado de error correlacionado. Una evaluación seria debe descomponer la tubería y probar cada obligación de forma independiente. También debe incluir las condiciones de fracaso en las que el enfoque propuesto debe ser rechazado o reducido.

La primera pregunta de la investigación se refiere a la representación: ¿mantener varias interpretaciones semánticas admisibles mejora la confiabilidad sobre la selección de una traducción? La hipótesis correspondiente es que la semántica establecida por candidatos, combinada con el análisis de may/most, reduce la falsa seguridad en entradas ambiguas y subespecificadas. Puede reducir la tasa de respuesta aparente porque más casos se vuelven condicionales o desconocidos. Eso no es necesariamente un defecto. Las medidas de comparación pertinentes sobre si las conclusiones aceptadas deben ser más a menudo correctas y si los hallazgos pueden capturar alternativas materialmente plausibles sin revisores abrumadores.

Un punto de referencia para esta cuestión debe contener textos con conjuntos de interpretación anotados por expertos en lugar de una forma lógica de oro. Los pares mínimos deben variar el alcance del cuantificador, la negación, el apego de la excepción, la coordinación, la correferenciación, los límites temporales, la fuerza modal y la jurisdicción. Algunos elementos deben tener una lectura claramente prevista, algunas lecturas legítimas y algunas pruebas insuficientes para cualquier respuesta determinada. Los sistemas se evalúan en la cobertura de lecturas admisibles, exclusión de lecturas inadmisibles, calibración del estado de may/most y veredictos posteriores. La coincidencia exacta del gráfico es demasiado frágil; la equivalencia semántica debe probarse a través de conjuntos de consultas o el embalaje bidireccional bajo la ontología de referencia.

La segunda pregunta de investigación se refiere al refinamiento: ¿son los conflictos derivados de los solucionadores más útiles para reparar la formalización que la autocrítica del modelo genérico? La hipótesis es que los contraejemplos, los núcleos insatisfiables y las obligaciones de prueba fallidas producen reparaciones más localizadas, estables y eficientes en la muestra porque identifican la dependencia responsable del resultado. Un experimento controlado debe comparar cuatro condiciones: reintento directo, autorreflexión verbal, retroalimentación de error de ejecución y refinamiento semántico guiado por contraejemplo con mapeo de fuente. El mismo modelo, temperatura y presupuesto deben utilizarse siempre que sea posible.

La calidad de reparación debe medirse no solo por si el ejemplo objetivo se vuelve correcto. La reparación debe preservar casos no relacionados, reducir la diferencia semántica y evitar la introducción de supuestos sin soporte. Las suites de regresión basadas en mutaciones son esenciales. Una reparación que resuelve un ejemplo debilitando la regla a nivel mundial no es un éxito. El experimento debería registrar el número de iteraciones, fichas, intervenciones humanas, mutantes supervivientes, observaciones cambiadas y si el artefacto final era reproducible de forma independiente.

La tercera pregunta de investigación se refiere al contrato de pruebas. ¿Una separación mecanografiada de las fuentes, observaciones, afirmaciones de ontología, axiomas de dominio y premisas de conocimiento del mundo mejora la transferencia y la auditabilidad en comparación con un gráfico de conocimiento plano o una cadena de pensamiento de forma libre? La hipótesis es que los contratos tipificados reducen la introducción silenciosa de premisas y hacen que la reutilización de dominios cruzados sea más predecible. El costo probable es la creación de gastos generales y más carreras detenidas.

El punto de referencia debe incluir la misma regla aplicada a través de documentos de varios dominios, con variación controlada en el vocabulario y la ontología. Por ejemplo, un requisito general de que cada reclamo aceptado tenga evidencia rastreable puede ser probado en documentos de investigación, documentos de política, especificaciones técnicas y planes. Las observaciones estructurales pueden transferirse directamente; la entidad específica del dominio y los productores de relaciones no. La evaluación registra qué comprobaciones de compatibilidad se detienen, si la razón de parada es correcta y cuánto nuevo material semántico se requiere para reanudar la ejecución. La transferencia exitosa no es simplemente obtener una respuesta; es reconocer con precisión lo que puede y no puede ser reutilizado.

La cuarta pregunta de investigación se refiere a la elección del formalismo. ¿El enrutamiento a una cartera de motores especializados supera a un lenguaje intermedio universal bajo la supervisión semántica igual? Beiser y Penz muestran que la elección del lenguaje intermedio puede cambiar sustancialmente la precisión general y de ejecución en las tareas de razonamiento neurosimbólico [Beiser and Penz 2025]. La hipótesis aquí es más específica: un router tipografiado que seleccione el formalismo adecuado más simple mejorará el éxito de la compilación, la calidad de la explicación y la eficiencia sin sacrificar la consistencia de la tarea cruzada.

Una comparación significativa no debería permitir que el sistema de cartera tenga más información oculta. Cada tarea recibe la misma evidencia de origen y ontología. La línea de base de formalismo único podría usar lógica de primer orden, ASP, SMT o un lenguaje de consulta de gráfico general. El sistema de cartera puede elegir consultas relacionales, Datalog, tablas de decisión, SMT, lógica temporal o ejecución simbólica. Las métricas incluyen la validez de la formalización, la equivalencia semántica, el éxito del solucionador, el tiempo de ejecución, la calidad de los testigos y el costo de mantenimiento. Los casos de formalismo cruzado deben requerir dos motores y probar si los símbolos compartidos siguen siendo consistentes.

El experimento también debería buscar la contraevidencia. Una representación universal puede superar a la cartera en dominios estrechos porque el enrutamiento introduce errores y múltiples compiladores aumentan la base de confianza. La conclusión esperada no es que las carteras sean siempre superiores. Es que la pluralidad del formalismo solo se justifica cuando la distribución de tareas contiene genuinamente diferentes estructuras computacionales y cuando las interfaces conservan la semántica.

La quinta pregunta de investigación se refiere a la capa de lenguaje final. ¿La realización con pruebas restringidas reduce las afirmaciones no admitidas o semánticamente reforzadas mientras preserva la legibilidad aceptable? La hipótesis es que un resultado canónico más la generación con licencia de reclamo reduce sustancialmente la alucinación y la deriva de la modalidad en relación con la generación directa de respuestas o la generación de respuestas a partir de una traza de solucionador en bruto. El costo puede ser menos prosa natural y más regeneración.

El punto de referencia debe incluir resultados canónicos con incertidumbre sutil, evidencia conflictiva, condiciones anidadas, cantidades y fechas. Los evaluadores humanos juzgan la fidelidad fáctica, la integridad de las limitaciones materiales, la legibilidad y la utilidad. Los verificadores deterministas miden la preservación de la entidad, número, fecha, polaridad y modalidad. Un conjunto adversario debería tentar al renderizador a producir una conclusión más fuerte y simple que la que el resultado apoya. La métrica central no es la preferencia estilística, sino la velocidad a la que la prosa excede la autoridad del producto simbólico.

La sexta pregunta de investigación se refiere a la arquitectura federativa en sí. La revisión sistemática de 2026 de NLP neurosimbólico informa ganancias prometedoras para los sistemas federativos, pero advierte que la evidencia se confunde por las diferencias de tareas [Chatzikyriakidis and Lappin 2026]. Un experimento directo debe comparar una canalización federativa con una alternativa inyectiva o aprendida de extremo a extremo en la misma escala de tareas, datos y modelo. Los beneficios relevantes pueden parecer menos en la precisión en bruto que en la auditabilidad, abstención, localización de errores y reparación.

Por consiguiente, el estudio debería informar de varias dimensiones. La precisión de la tarea cruda sigue siendo necesaria. Debe ir acompañado de precisión de forma semántica, precisión acondicionada por el solucionador, calibración desconocida, reproducibilidad en versiones del modelo, fidelidad de explicación, tiempo de revisión humana y la fracción de errores atribuibles a la extracción, conexión a tierra, compilación, ejecución o realización. Un sistema modular que coincida en lugar de superar la precisión de extremo a extremo todavía puede ser preferible para el trabajo de alta seguridad si convierte los errores silenciosos en artefactos diagnosticables. Por el contrario, la modularidad no debe ser defendida como una virtud intrínseca si simplemente agrega burocracia sin mejorar la evidencia.

Una séptima pregunta de investigación se refiere a la interpretación abstracta sobre las alternativas semánticas. ¿Se puede definir un dominio abstracto útil que sea conservador en relación con un conjunto de interpretación de referencia y más escalable que la enumeración explícita? Los primeros experimentos pueden utilizar conjuntos finitos como referencia concreta. Los dominios abstractos para identidad, intervalos temporales, modalidad, cantidades y cobertura se implementan por separado y luego se combinan. La solidez se comprueba mecánicamente contra la enumeración en casos pequeños. La precisión se mide por la tasa de hallazgos de maya que pueden ser eliminados por la búsqueda de testigos o el refinamiento.

Este programa debe tener cuidado con la palabra solidez. Si la anotación experta define un conjunto de interpretación finita, el intérprete abstracto puede ser sólido en relación con ese conjunto. Por lo tanto, no puede probar que el conjunto agota el significado humano. La contribución es una semántica condicional ejecutable y evidencia sobre su utilidad. Un artículo teórico posterior puede definir la relación de admisibilidad, las funciones de abstracción y concretización, las condiciones de transferencia y los operadores de refinamiento. El sistema empírico no debe esperar a una semántica completa del lenguaje, sino que debe hacer explícita la naturaleza condicional de sus afirmaciones.

Una octava pregunta de investigación se refiere a la ejecución simbólica sobre los programas semánticos. ¿La construcción de testigos sensibles a la trayectoria mejora la comprensión del revisor y el descubrimiento de defectos en relación con los resultados de las reglas estáticas? En un flujo de trabajo o dominio de política, el tiempo de ejecución puede comparar un simple evaluador de reglas con la exploración de ruta simbólica. El evaluador dice que una violación es posible; el ejecutor devuelve una secuencia concreta de eventos y

asignaciones. Los revisores humanos juzgan si el testigo es inteligible y útil. Los defectos de la verdad sobre el terreno y los mutantes sembrados determinan si el testigo corresponde a un caso real.

La explosión de la trayectoria debe ser parte de la evaluación en lugar de ser tratada como un inconveniente de la implementación. El estudio debe variar la ramificación, los límites de bucle, las alternativas de identidad y las elecciones temporales. La guía LLM puede priorizar las rutas, como sugiere el trabajo reciente de ejecución simbólica guiada por LLM, pero el motor debe distinguir la cobertura heurística de la integridad formal. Las métricas de cobertura deben informar estados explorados, estados podados, tiempos de espera de solucionador y comportamiento externo sin soporte. Un testigo es una fuerte evidencia de la existencia; la falta de encontrar uno no es evidencia de ausencia a menos que la exploración vinculada y el modelo justifiquen esa conclusión.

La arquitectura de referencia debe estar en capas. La capa fuente prueba la ingestión y el anclaje. La capa de observación prueba la extracción y el estado. La capa de conexión a tierra prueba la identidad, el tiempo, la modalidad, las cantidades y la autoridad. La capa de formalización prueba la equivalencia semántica y la validez del idioma objetivo. La capa de ejecución prueba las salidas y los certificados del solucionador. La capa de resultado prueba la composición del veredicto y la incertidumbre. La capa de realización prueba la fidelidad de las respuestas del lenguaje natural. Cada ejemplo debe identificar la capa más temprana en la que se debe detectar una falla inyectada.

Los casos de referencia deben ser lo suficientemente pequeños como para validar manualmente y componibles en documentos más largos. Pequeños ejemplos proporcionan una verdad fundamental confiable para la lógica sutil. Los compuestos largos prueban la recuperación, la indexación, la continuidad de la identidad y la escalabilidad. La misma regla subyacente puede aparecer en varias formas lingüísticas y estructuras de documentos. Las familias de parafraseo prueban la invariancia; los conjuntos de contraste cambian una característica semántica y la sensibilidad de la prueba. El punto de referencia debe evitar el fallo común en el que un sistema funciona bien memorizando los patrones de superficie mientras ignora la distinción prevista.

Las pruebas de mutación deben ser centrales. Las mutaciones pueden cambiar un cuantificador, negar una condición, mover una excepción, alterar un límite de fecha, fusionar identidades, eliminar evidencia, cambiar la fuerza modal, intercambiar actor y objeto, duplicar un requisito o modificar un umbral de circuito. Una suite robusta debe detectar el cambio esperado y dejar los resultados no relacionados estables. La puntuación de mutación no es una medida de calidad completa, pero revela pruebas que simplemente confirman un camino feliz. Para nllAgent, las mutaciones ejecutables abordarían directamente una de las brechas reconocidas del repositorio [AssistOS-AI 2026].

La evaluación debe incluir la inyección inmediata y adversarios semánticos. Los documentos pueden contener instrucciones dirigidas al modelo, encabezados engañosos, texto de política citado, Unicode contradictorio, contenido oculto o declaraciones que se asemejan a declaraciones de ontología. El analizador de origen debe preservarlos como datos. Los productores semánticos deben operar bajo la autoridad del sistema fijo. El punto de referencia debe probar si el texto del documento puede alterar el circuito, suprimir los hallazgos o causar llamadas de herramientas privilegiadas. Los resultados de seguridad deben registrarse por separado de la precisión semántica.

La evaluación del modelo en vivo debe ser lo suficientemente reproducible como para apoyar las afirmaciones científicas. Cada ejecución debe fijar el proveedor, el identificador del modelo, el esquema de solicitud, la temperatura, la semilla cuando se admite, la construcción del contexto y la política de reintento. Las entradas y salidas deben ser capturadas sujetas a restricciones de privacidad. Los ensayos repetidos estiman la varianza. Los dobles deterministas siguen siendo útiles para las pruebas de software, pero no pueden establecer la calidad de la traducción. Al menos dos familias modelo deben ser probadas cuando sea posible para distinguir los efectos arquitectónicos del comportamiento de un proveedor.

La evaluación humana es necesaria, pero debe ser estrechamente diseñada. A los expertos no se les debe preguntar si una respuesta “se ve bien”. Deben juzgar obligaciones específicas: si la formalización preserva la fuente, si las identidades están justificadas, si un testigo es factible, si la explicación establece todas las suposiciones materiales y si una descripción desconocida se devuelve adecuadamente. El desacuerdo entre factores de valoración es en sí mismo una evidencia de ambigüedad semántica y no debe ser forzado a una sola etiqueta de oro sin análisis.

Las métricas cuantitativas básicas deben incluir la precisión de la fuente-ancla; la precisión de la observación y el recuerdo por estado; ontología y validez de tipo; precisión, recuerdo y calibración de enlace de identidad; precisión de la relación temporal y modal; cobertura del conjunto de candidatos; may/must solidez en relación con las interpretaciones anotadas; sintaxis de formalización y equivalencia semántica; éxito del solucionador; validez del testigo; éxito de prueba de prueba; tasa de seguridad falsa; Una sola puntuación agregada ocultaría el mecanismo y debería ser secundaria.

La falsa seguridad merece especial énfasis. Mide los casos en los que el sistema presenta una conclusión verificada determinada, aunque la fuente, la base o el conjunto de interpretación no lo justifican. En aplicaciones de alta seguridad, reducir la falsa garantía puede importar más que maximizar las preguntas respondidas. El sistema debe recibir crédito por un estado de parada precisa cuando un LLM directo inventa con confianza un puente. Lo desconocido apropiado es un resultado positivo.

Las líneas de base deben incluir respuesta directa LLM, indicación de razonamiento de cadena de pensamiento o equivalente, ejecución de LLM aumentada por herramientas sin artefactos formales persistentes, traducción de un solo formalismo, traducción de candidatos sin interpretación abstracta, ejecución simbólica sin refinamiento semántico y la arquitectura completa. Para el estudio de caso nllAgent, la versión actual de la fundación limitada puede servir como una línea de base implementada, mientras que la identidad propuesta, la semántica abstracta y las extensiones de multiformalismo se evalúan gradualmente. Esto evita comparar una visión de investigación completa solo con sistemas externos más débiles.

El programa de publicación puede organizarse en torno a cinco contribuciones en lugar de una reclamación de gran tamaño. El primer documento debe ser un documento de posición y arquitectura que defina el lenguaje natural ejecutable, el límite de confianza, la brecha de formalización, la envolvente de solidez y el diseño multiformalismo. Su contribución es la claridad conceptual y una arquitectura de referencia, apoyada por un estudio sistemática y un pequeño demostrador de trabajo.

El segundo documento debe formalizar el pivote semántico que lleva la procedencia y la solidez condicional. Define objetos de origen, observaciones escritas, interpretaciones candidatas, semántica de may/imprescindible y compatibilidad. Demuestra la solidez de la

computación abstracta en relación con un conjunto de interpretación admisible finito e informa experimentos sobre identidad, tiempo, modalidad y cobertura. Este es el núcleo de programación-lenguajes y formal-semántica.

El tercer documento debería introducir un perfeccionamiento semántico guiado por contraejemplo. Define los mapeos de los conflictos de los solucionadores a los compromisos semánticos vinculados a la fuente y compara el refinamiento con la autorreflexión y la retroalimentación de ejecución genérica. El resultado más fuerte sería una menor falsa seguridad, reparaciones más pequeñas, una mejor preservación de la regresión y un esfuerzo humano reducido. Incluso un resultado negativo sería valioso si identifica qué retroalimentación del solucionador es demasiado bajo para guiar la reparación semántica.

El cuarto documento debe presentar la cartera de formalismo y los contratos de motores mecanografiados. Evalúa los adaptadores de ejecución relacional, Datalog, de mesa de decisión, SMT, temporales y simbólicos en un modelo de artefacto común. El documento debe hacer hincapié en la preservación semántica y los certificados en lugar del número de integraciones. Un pequeño conjunto de motores genuinamente diferentes es más persuasivo que muchos envoltorios poco profundos.

El quinto documento debe ser el sistema nllAgent y el documento de evaluación. Debería distinguir claramente la versión implementada del tiempo de ejecución más amplio. Puede documentar revisiones de fuente inmutables, LongTextJS, CircuitJS, observaciones impulsadas por la demanda, compatibilidad, cobertura, ejecución restringida, gobierno de aprendizaje y publicación, verificación de repetición, auditoría canónica y productos de planes, y el proceso de problemas graves. La evaluación debe incluir la mutación ejecutable, los registros de modelos vivos, los casos de identidad, la gobernanza del paquete del mundo actual, los puntos de referencia de planificación y un verificador independiente o implementado por separado cuando sea posible.

Un sexto artículo posterior podría estudiar la realización restringida a pruebas. Compararía la generación directa, el renderizado de plantillas, el renderizado restringido LLM y la verificación post-hoc. La contribución central sería una representación de licencia de reclamo y evidencia de que se puede generar un lenguaje legible sin cambiar la modalidad, la incertidumbre o el contenido fáctico. Este trabajo conecta la ejecución formal con la necesidad práctica de una comunicación humana accesible.

El proyecto debe definir criterios explícitos de fracaso. El enfoque de conjunto de candidatos debe reconsiderarse si produce tantas alternativas que pueden/deben ser resultados no informativos y el refinamiento no puede recuperar la precisión. La cartera de formalismo debe reducirse si los errores de enrutamiento y formalismo cruzado exceden los beneficios de la especialización. La formalización asistida por LLM debería restringirse a la creación de apoyo si la equivalencia semántica sigue siendo deficiente en los documentos realistas. La realización con prueba restringida debería recurrir a las plantillas si la prosa generada por el modelo excede repetidamente su licencia. El razonamiento mundial actual debe permanecer fuera del núcleo de confianza si no se puede hacer cumplir la fuente, el tiempo, la jurisdicción y los controles de conflicto.

La honestidad científica también requiere la comparación con alternativas más simples. Muchas comprobaciones de documentos se pueden resolver mediante análisis determinísticos, expresiones regulares, validación de esquemas, consultas de base de datos o reglas de autoría humana. Un LLM se justifica cuando la variación del lenguaje, el mapeo ontológico o la

propuesta semántica reduce materialmente el costo de la creación o aumenta la cobertura. Un núcleo simbólico se justifica cuando el estado exacto, la repetición, los testigos o el asunto de verificación independiente. Un trabajo de investigación debe informar cuando un sistema más simple funciona igual de bien. El lenguaje natural ejecutable no es una razón para formalizar cada preferencia editorial o insertar un solucionador en tareas que solo necesitan un analizador confiable.

El resultado práctico probable no es una máquina que ejecute todo el lenguaje natural. Es una familia de compiladores semánticos gobernados para dominios de documentos y agentes, compartiendo la infraestructura de fuente, conexión a tierra, ejecución y certificado. Cada compilador tiene una envolvente semántica declarada. LLMs hace que esos compiladores sean más fáciles de escribir, adaptar y explicar. Los métodos formales hacen que sus resultados aceptados sean reproducibles y controvertibles. La combinación es útil precisamente porque a ninguno de los componentes se le pide que proporcione lo que no puede proporcionar de manera confiable solo.

Por lo tanto, el programa de investigación es más modesto y más consecuente que el lema “el lenguaje natural como código”. El lenguaje natural sigue siendo un medio abierto, social y dependiente del contexto. El tiempo de ejecución crea proyecciones ejecutables de la misma para las preguntas declaradas. Esas proyecciones pueden ser lo suficientemente rigurosas como para soportar auditorías reales, planes, protocolos y controles científicos, siempre que el sistema registre la distancia entre la fuente y el objeto formal en lugar de pretender que la distancia ha desaparecido.

Conclusión: de la forma probabilística a la consecuencia simbólica

El reciente renacimiento del neuro-simbólico AI ha producido un patrón arquitectónico reconocible. Se utilizan modelos de lenguaje grandes donde el espacio de búsqueda es lingüístico, heterogéneo o difícil de enumerar. Los sistemas formales se utilizan donde el estado exacto, la consistencia, la búsqueda exhaustiva y limitada, los testigos o las pruebas importan. La pregunta interesante ya no es si los métodos neuronales y simbólicos se pueden combinar. Claramente pueden. La pregunta importante es dónde se encuentra la autoridad y qué se verifica de forma independiente.

La encuesta muestra que los sistemas más creíbles no otorgan el modelo de lenguaje a la autoridad final sobre el resultado. Los lenguajes controlados traducen una superficie restringida a la semántica formal. Los analizadores semánticos generan consultas o programas ejecutables. PAL delega computación en Python. SatLM delega restricciones a un solucionador. Logic-LM y LINC delegan la deducción a los motores lógicos. NL2TL y nl2spec traducen los requisitos a especificaciones temporales. Los sistemas de planificación utilizan modelos formales y herramientas de verificación para mantener la consistencia de horizonte largo. SymGPT compila reglas de lenguaje natural ERC en restricciones y utiliza la ejecución simbólica para construir rutas de contrato concretas. SAIL utiliza LLMs para buscar transformadores abstractos, pero admite solo candidatos que cumplen con las obligaciones mecánicas. Estos sistemas difieren mucho, pero convergen en el principio que propone la generación y decide un mecanismo independiente.

Esta convergencia no debe ser exagerada. El uso de solucionadores no verifica la comprensión. La brecha de formalización sigue siendo el problema central. Un probador de teoremas puede derivar una conclusión válida de predicados que tergiversan el texto. Un programa puede devolver la respuesta esperada a través de un camino espurio. Un modelo SMT puede satisfacer restricciones que omiten el requisito decisivo del usuario. Un ejecutor simbólico puede exponer una ruta de software real para una regla mal traducida. Los sistemas actuales más fuertes reducen el error deductivo y computacional al tiempo que reubican el riesgo de análisis semántico, la conexión a tierra, la ontología y el alcance.

La respuesta adecuada no es abandonar las herramientas formales porque la traducción es imperfecta. Es para hacer de la traducción un artefacto persistente, mecanografiado y revisable. La fuente debe permanecer disponible. Toda observación semántica debe identificar a su productor, evidencia, estado y ontología. La identidad, el tiempo, la modalidad, las cantidades y la autoridad deben estar representados en lugar de inferirse de nuevo durante cada respuesta. Las interpretaciones alternativas deben seguir siendo alternativas cuando la distinción afecte al resultado. Un motor simbólico debe ejecutar solo objetos fundamentados que cumplan con contratos explícitos.

La interpretación abstracta proporciona una teoría útil para el lado global de esta arquitectura. Se pregunta qué se puede concluir de manera conservadora de un gran conjunto de posibles estados. Aplicado al lenguaje natural, los estados concretos no son “significados” indefinidos, sino estructuras semánticas admisibles bajo una gramática declarada, ontología, revisión de fuente y política de base. Un estado abstracto puede entonces representar lo que debe sostener, puede sostener, no sostener, entrar en conflicto o permanecer desconocido. La

garantía está condicionada al conjunto de interpretación, pero la solidez condicional sigue siendo significativa cuando las condiciones son explícitas y probadas.

La ejecución simbólica proporciona el complemento local. Cuando un análisis abstracto informa de una posible violación, la ejecución simbólica u otro motor de restricción puede buscar una interpretación y un camino concretos. Una búsqueda exitosa devuelve un testigo que un revisor puede inspeccionar. Una búsqueda fallida expone una abstracción demasiado tosca o una elección semántica incompatible. El mapeo de que el fracaso de vuelta a la fuente transcurre crea un refinamiento semántico guiado por contraejemplos: el modelo o humano revisa el compromiso formal más pequeño relevante y se repite el análisis. Este es un uso más disciplinado de la retroalimentación de LLM que la autocrítica genérica porque la señal de error está vinculada a una dependencia formal.

Es probable que ningún idioma de destino único admita todas las formas útiles de la prosa ejecutable. La literatura ya demuestra la dependencia de la elección del lenguaje intermedio, y los sistemas contemporáneos tienen éxito mediante el uso de Python, SQL, lógica de primer orden, lógica temporal, PDDL, lenguajes de restricción o gramáticas específicas de dominio de acuerdo con la tarea. Por lo tanto, un tiempo de ejecución práctico debe utilizar un pivote semántico que convalezca la procedencia y una cartera de motores formales. La evaluación relacional y Datalog maneja las consultas y el cierre de documentos; las tablas de decisión manejan políticas finitas; SAT/SMT/CSP manejan restricciones de interacción; la verificación de modelos maneja comportamientos temporales; la ejecución simbólica maneja los testigos de la ruta; los patrones ordenados de manejo automático; la lógica natural maneja las implicaciones seleccionadas; los asistentes de prueba manejan reclamos formales de alto valor.

La pluralidad introduce sus propios riesgos de ingeniería. Cada compilador puede distorsionar la semántica, cada motor expande la base de confianza y el enrutamiento puede fallar. La cartera solo se justifica si se escriben interfaces, los símbolos compartidos conservan la identidad y la procedencia, las garantías del motor son explícitas y los resultados se normalizan en un producto canónico con certificado. Una lógica universal puede seguir siendo preferible en un dominio estrecho. La afirmación arquitectónica no es que la pluralidad siempre sea mejor; es que el lenguaje ejecutable amplio requiere diferentes estructuras computacionales y no debe disfrazarlas detrás de un vago DSL.

El retorno de los símbolos al lenguaje es parte del límite de confianza. No se debe permitir que un LLM final convierta un resultado condicional en una recomendación incondicional, una posible violación en una violación confirmada o evidencia faltante en evidencia de ausencia. El resultado canónico debe definir un conjunto finito de reclamos con licencia con modalidad, cantidades, entidades, suposiciones y procedencia. El renderizador puede mejorar la legibilidad, pero debe permanecer subordinado a esa autoridad. La realización con pruebas restringidas es la finalización natural de la arquitectura.

La implementación pública nllAgent es significativa porque ya encarna varias de estas disciplinas. LongTextJS conserva un programa de documentos basado en la fuente; CircuitJS representa teorías reutilizables; la demanda de observación se deriva de circuitos; la compatibilidad y la cobertura son explícitas; los módulos restringidos y las extensiones revisadas limitan la ejecución oculta; el aprendizaje se lleva a cabo en la puesta en escena desechable; la publicación es manual e inmutable; los controles de verificación de reproducción de los hallazgos canónicos; y el conocimiento faltante no se trata como cumplimiento. Estas

decisiones hacen del proyecto una plataforma creíble para la investigación, aunque no establecen la comprensión semántica general.

Los mismos materiales públicos identifican el importante trabajo inconcluso: la materialización general de la identidad, los contratos primitivos de tipo completo, la mutación semántica ejecutable, la evaluación independiente de la sombra, la evaluación comparativa del modelo en vivo, los puntos de referencia de planificación y realización, y los paquetes de conocimiento del mundo actual gobernados. Para esta edición no se localizó ninguna evaluación independiente revisada por pares de nllAgent. La sentencia ofrecida aquí es, por lo tanto, arquitectónica: la implementación ha elegido límites inusualmente sólidos, pero sus afirmaciones defendibles más fuertes siguen siendo un análisis estructural y de lenguaje controlado. Su valor aumentará si la amplitud semántica se expande solo a través de productores explícitos, contratos, puntos de referencia y evidencia.

El siguiente paso propuesto no es reemplazar LongTextJS o CircuitJS con un lenguaje universal más grande. LongTextJS debería madurar como un mundo semántico finito, cuestionable y que lleva la procedencia. CircuitJS debe seguir siendo una capa de orquestación y contrato de evidencia restringida. Los motores registrados especializados deben realizar una ejecución relacional, restrictiva, temporal, de flujo de datos, de automatización o de ruta. La semántica de mayo/debe entrar en funcionamiento. Los contraejemplos deben impulsar el refinamiento semántico enfocado. Los productos canónicos CNL deben controlar la salida final del idioma.

Un riguroso programa de investigación puede probar cada parte. La semántica de conjunto de candidatos se puede comparar con los sistemas de traducción única. El refinamiento derivado del solucionador se puede comparar con la autorreflexión. Los contratos de evidencia mecanografiados pueden ser probados para la transferencia y la falsa garantía. Una cartera de formalismo se puede comparar con un lenguaje intermedio universal. Los dominios abstractos se pueden comparar en conjuntos de interpretación explícita. Los testigos simbólicos pueden ser evaluados para el descubrimiento de defectos y la utilidad del revisor. La realización con la prueba restringida se puede probar para la fidelidad. Las arquitecturas modulares y de extremo a extremo se pueden comparar en las mismas tareas en lugar de en diferentes familias de referencia.

La métrica decisiva debe ser la falsa seguridad, no simplemente la tasa de respuesta. Un sistema que regresa “desconocido porque la identidad no está resuelta” puede ser más valioso que uno que produce un veredicto de confianza al fusionar silenciosamente a dos personas. Un sistema que se detiene porque la evidencia del mundo actual es obsoleta es más confiable que uno que recupera un hecho sin soporte. Un sistema que expone tres interpretaciones es mejor que una que elige a los más fluidos. La abstención apropiada es parte de la semántica de ejecución.

Por lo tanto, el lenguaje natural ejecutable debe entenderse como una relación de compilación gobernada, no como una propiedad intrínseca de la prosa. El texto fuente sigue siendo más rico que cualquier proyección ejecutable. El sistema selecciona una tarea, materializa la semántica relevante, preserva la incertidumbre, fundamenta los símbolos, compila a un formalismo apropiado, ejecuta bajo límites explícitos, verifica el resultado y devuelve una explicación restringida. Lo que se hace ejecutable no es el lenguaje en su totalidad, sino una proyección semántica declarada cuyos supuestos y límites permanecen visibles.

Esta concepción más estrecha es suficiente para un impacto práctico sustancial. Las afirmaciones científicas se pueden verificar en cuanto a procedencia y consistencia. Los requisitos pueden estar relacionados con pruebas y estados del ciclo de vida. Las políticas se pueden compilar en estructuras de decisión. Los planes se pueden verificar antes de la realización. Las reglas del protocolo se pueden asignar a análisis de estado y ruta. Los documentos largos pueden ser revestidos por teorías reutilizables sin pedir un modelo para recordar cada detalle. Los agentes pueden usar el lenguaje natural como una capa de creación y coordinación mientras delegan las consecuencias exactas en los tiempos de ejecución simbólicos.

El principio central se puede establecer sin simbolismo formal: deje que el LLM descubra la forma posible; deje que el núcleo de conexión a tierra decida qué se permite denotar los símbolos; deje que los motores formales determinen lo que sigue; y deje que el LLM final explique solo lo que el registro simbólico licencia. Este principio no resuelve todos los problemas del lenguaje. Crea un límite defendible entre la interpretación y la consecuencia, que es la base necesaria para que el lenguaje natural participe de manera segura en los sistemas ejecutables.

Referencias

- Abzianidze, Lasha, Rik van Noord, Hessel Haagsma y Johan Bos. 2019. «La primera tarea compartida sobre análisis y evaluación de la estructura de representación del discurso». Actas de la tarea compartida IWCS sobre análisis semántico. Asociación para la Lingüística Computacional. <https://aclanthology.org/W19-1201/>
- Abzianidze, Lasha. 2017. «LangPro: Teorema del Lenguaje Natural Prover». Actas de la Conferencia 2017 sobre Métodos Empíricos en el Procesamiento del Lenguaje Natural: Demostraciones del Sistema. Asociación para la Lingüística Computacional. <https://aclanthology.org/D17-2022/>
- Angeli, Gabor y Christopher D. Manning. 2014. «NaturalLI: Inferencia lógica natural para el razonamiento de sentido común». Actas de EMNLP 2014. Asociación para la Lingüística Computacional. <https://aclanthology.org/D14-1209/>
- AssistOS-AI. 2026. «NaturalLanguageLinterAgent (nllAgent): LongTextJS–CircuitJS Arquitectura, Documentación y Cuestiones Serias». Repositorio público y documentación generada, inspeccionado 2 August 2026. <https://github.com/assistos-ai/nllAgent>
- Badreddine, Samy, Artur d'Ávila Garcez, Luciano Serafini y Michael Spranger. 2022. «Redes de Tensor Lógico». Inteligencia Artificial 303: 103649. <https://doi.org/10.1016/j.artint.2021.103649>
- Baldoni, Roberto, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu y Irene Finocchi. 2018. «Un estudio de técnicas de ejecución simbólicas». ACM Encuestas de computación 51(3): 1-39. <https://doi.org/10.1145/3182657>
- Beg, Arshad, Diarmuid P. O'Donoghue y Rosemary Monahan. 2025. «Una breve encuesta sobre la formalización de los requisitos de software utilizando modelos de lenguaje grandes». arXiv:2506.11874. <https://doi.org/10.48550/arXiv.2506.11874>
- Beiser, Alexander y David Penz. 2025. «¿Hacer LLMs Razón? El problema del lenguaje intermedio en los enfoques neurosimbólicos». arXiv:2502.17216. <https://arxiv.org/abs/2502.17216>
- Bos, Johan. 2008. «Análisis semántico de cobertura amplia con Boxer». Actas de la Conferencia 2008 sobre Semántica en el Procesamiento de Texto. <https://aclanthology.org/W08-2222/>
- Chatzikyriakidis, Stergios y Shalom Lappin. 2026. «Neuro-Simbólico NLP: Taxonomía, Evaluación y Direcciones». Fronteras en Inteligencia Artificial 9. <https://doi.org/10.3389/frai.2026.1797587>
- Chen, Lingfeng, Tao Xiao, Masanari Kondo y Yasutaka Kamei. 2026. «Ejecución simbólica dirigida para el descubrimiento de vulnerabilidades: un enfoque guiado por LLM en KLEE». arXiv:2607.21676. <https://arxiv.org/abs/2607.21676>
- Chen, Yongchao, Rujul Gandhi, Yang Zhang y Chuchu Fan. 2023. «NL2TL: Transformando los lenguajes naturales a la lógica temporal utilizando modelos de lenguaje grandes». Actas de la Conferencia 2023 sobre Métodos Empíricos en el Procesamiento del Lenguaje Natural. Asociación para la Lingüística Computacional. <https://aclanthology.org/2023.emnlp-main.985/>
- Clarke, Edmund, Orna Grumberg, Somesh Jha, Yuan Lu y Helmut Veith. 2000. «Refinamiento de abstracción guiado por contraejemplo». Verificación asistida por computadora, LNCS 1855, 154–169. https://doi.org/10.1007/10722167_15
- Cosler, Matthias, Christopher Hahn, Daniel Mendoza, Frederik Schmitt y Caroline Trippel. 2023. «nl2spec: Traducir interactivamente el lenguaje natural no estructurado a la lógica temporal con modelos de lenguaje grandes». Verificación asistida por computadora, LNCS 13964, 383–396. https://doi.org/10.1007/978-3-031-37703-7_18
- Cousot, Patrick y Radhia Cousot. 1977. «Interpretación: Un modelo de celosía unificada para el análisis estático de programas por construcción o aproximación de puntos de arreglo». Actas de POPL 1977, 238–252. <https://doi.org/10.1145/512950.512973>
- Delvecchio, Giovanni Pio, Lorenzo Molfetta y Gianluca Moro. 2025. «Inteligencia artificial neurosímbola: un estudio dirigida a tareas en la era de los modelos de caja negra». Actas de IJCAI 2025, Survey Track, 10418–10426. <https://doi.org/10.24963/ijcai.2025/1157>
- Dong, Wenkai y Yifan Wang. 2026. «De la interpretación a la compilación: Ejecución basada en compilaciones de operadores semánticos». arXiv:2607.13407. <https://doi.org/10.48550/arXiv.2607.13407>

- Ferrari, Alessio y Paola Spoletini. 2025. «Ingeniería de requisitos formales y modelos de lenguaje grande: una hoja de ruta bidireccional». *Tecnología de la información y software* 181: 107697. <https://doi.org/10.1016/j.infsof.2025.107697>
- Fuchs, Norbert E. y Rolf Schwitter. 1996. «Intento Inglés Controlado (ACE)». *Actas del Primer Taller Internacional sobre Aplicaciones de Lenguaje Controlado*. <https://attempto.ifi.uzh.ch/site/pubs/papers/ace96.pdf>
- Fuchs, Norbert E., Kaarel Kaljurand y Tobias Kuhn. 2008. «Intento Inglés Controlado para la Representación del Conocimiento». *Razonamiento Web, LNCS 5224*, 104–124. https://doi.org/10.1007/978-3-540-85658-0_3
- Gao, Luyu, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan y Graham Neubig. 2022. «PAL: Modelos De Lenguaje Asistidos Por Programas». *arXiv:2211.10435*. <https://arxiv.org/abs/2211.10435>
- Goldman, Omer, Veronica Latcinnik, Udi Naveh, Amir Globerson y Jonathan Berant. 2018. «Análisis semántico débilmente supervisado con ejemplos abstractos». *Actas de ACL 2018. Asociación para la Lingüística Computacional*. <https://aclanthology.org/P18-1168/>
- Grimaldi, Pasquale. 2026. «pamaldi en SemEval-2026 Tarea 11: Razonamiento silogístico neurosimbólico a través de la extracción de estructura guiada LLM y validación determinista». *Actas del XX Taller Internacional de Evaluación Semántica. Asociación para la Lingüística Computacional*. <https://doi.org/10.18653/v1/2026.semeval-1.186>
- Gu, Qiuhan, Avaljot Singh y Gagandeep Singh. 2026. «SAIL: Intérpretes abstractos de sonido con LLMs». *Actas de la ACM sobre Lenguajes de Programación 10 (PLDI): 1534–1559*. <https://doi.org/10.1145/3808308>
- Guu, Kelvin, Panupong Pasupat, Evan Liu y Percy Liang. 2017. «Del lenguaje a los programas: acercamiento al aprendizaje de refuerzo y la máxima probabilidad marginal». *Actas de ACL 2017. Asociación para la Lingüística Computacional*. <https://aclanthology.org/P17-1016/>
- Hamilton, Kyle, Aparna Nayak, Bojan Božić y Luca Longo. 2024. «¿Está cumpliendo Neuro-Simbólico AI sus promesas en el procesamiento del lenguaje natural? Una revisión estructurada». *Web Semántica* 15(4): 1265–1306. <https://doi.org/10.3233/SW-223228>
- Han, Simeng, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alex Wardle-Solano, Hannah Szabó, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander Fabbri, Wojciech Kryściński, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying y Dragomir Radev. 2024. «FOLIO: Razonamiento del lenguaje natural con lógica de primer orden». *Actas de EMNLP 2024. Asociación para la Lingüística Computacional*. <https://aclanthology.org/2024.emnlp-main.1229/>
- Hao, Yilun, Yongchao Chen, Yang Zhang y Chuchu Fan. 2025. «Los modelos de lenguaje grande pueden resolver la planificación del mundo real rigurosamente con herramientas de verificación formal». *Actas de NAACL 2025*, 3434–3483. <https://aclanthology.org/2025.naacl-long.176/>
- Hu, Ruikang, Shaoyu Lin, Yeliang Xiu y Yongmei Liu. 2025. «LTRAG: Mejora de la autoformalización y el auto-refinamiento para el razonamiento lógico con RAG guiado por el pensamiento». *Hallazgos de ACL 2025*, 2483–2493. <https://aclanthology.org/2025.findings-acl.126/>
- Huang, Pei, Jiayu Li, Yuhao Zhang, Changliu Liu y Shih-hsi Alex Liu. 2026. «Interpretación abstracta parametrizada para la verificación de transformadores». *Actas de AAAI 2026*. <https://doi.org/10.1609/aaai.v40i42.40860>
- Jia, Robin, Aditi Raghunathan, Kerem Göksel y Percy Liang. 2019. «Robustez certificada a las sustituciones de palabras adversas». *Actas de EMNLP-IJCNLP 2019. Asociación para la Lingüística Computacional*. <https://aclanthology.org/D19-1423/>
- Karthikeyan, T, Om Dehlan, Mausam y Manish Gupta. 2025. «LRPLAN: Una colaboración multi-agente de grandes modelos de lenguaje y razonamiento para la planificación con restricciones implícitas y explícitas». *Hallazgos de EMNLP 2025*, 8280–8310. <https://doi.org/10.18653/v1/2025.findings-emnlp.440>
- Kartáč, Ivan, Kristýna Onderková, Jan Bronec, Zdeněk Kasner, Mateusz Lango y Ondřej Dušek. 2026. «UFAL-CUNI en SemEval-2026 Tarea 11: Un método neuro-simbólico modular eficiente para el razonamiento silogístico». *Actas del XX Taller Internacional de Evaluación Semántica. Asociación para la Lingüística Computacional*. <https://aclanthology.org/2026.semeval-1.418/>
- King, James C. 1976. «Ejecución simbólica y pruebas de programa». *Comunicaciones de la ACM* 19(7): 385–394. <https://doi.org/10.1145/360248.360252>

- Kirtania, Shashank, Priyanshu Gupta y Arjun Radhakrishna. 2024. «LOGIC-LM++: Refinamiento multipaso para formulaciones simbólicas». *Actas del 2o Taller sobre Razonamiento y Explicaciones Estructuradas del Lenguaje Natural*, 56–63. <https://aclanthology.org/2024.nlse-1.6/>
- Kuhn, Tobias. 2014. «Encuesta y clasificación de los lenguajes naturales controlados». *Lingüística Computacional* 40(1): 121–170. https://doi.org/10.1162/COLI_a_00168
- Lalwani, Vivek, et al. 2025. «Autoformalización del lenguaje natural a la lógica de primer orden: un estudio de caso en la detección de falacias lógicas». *Hallazgos de IJCNLP-AACL 2025*, 132–147. <https://aclanthology.org/2025.findings-ijcnlp.8/>
- Lee, Kang-il, Segwang Kim y Kyomin Jung. 2023. «Análisis semántico débilmente supervisado con filtrado de programas espurios basado en ejecución». *Actas de EMNLP 2023*, 6884–6894. <https://aclanthology.org/2023.emnlp-main.425/>
- Li, Yihe, Ruijie Meng y Gregory J. Duck. 2025. «Modelo de lenguaje grande impulsado por ejecución simbólica». *Actas de la ACM en Lenguajes de Programación 9 (OOPSLA2)*, artículo 385, 3148–3176. <https://doi.org/10.1145/3763163>
- Liang, Chen, Jonathan Berant, Quoc Le, Kenneth D. Forbus y Ni Lao. 2017. «Máquinas simbólicas neuronales: aprendiendo analizadores semánticos en Freebase con una supervisión débil». *Actas de ACL 2017. Asociación para la Lingüística Computacional*. <https://aclanthology.org/P17-1003/>
- Liu, Jiangming, Shay B. Cohen y Mirella Lapata. 2018. «Análisis de la estructura de representación del discurso». *Actas de ACL 2018. Asociación para la Lingüística Computacional*. <https://aclanthology.org/P18-1040/>
- Locascio, Nicholas, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman y Regina Barzilay. 2018. «Generación neuronal de expresiones regulares del lenguaje natural con un conocimiento mínimo del dominio». *Actas de EMNLP 2018. Asociación para la Lingüística Computacional*. <https://aclanthology.org/D18-1436/>
- MacCartney, Bill y Christopher D. Manning. 2007. «Lógica natural para la inferencia textual». *Actas del Taller ACL-PASCAL sobre la implicación textual y el parafraseo*. <https://aclanthology.org/W07-1401/>
- MacCartney, Bill y Christopher D. Manning. 2009. «Un modelo extendido de lógica natural». *Actas de IWCS 2009*. <https://aclanthology.org/W09-3714/>
- Manhaeve, Robin, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester y Luc De Raedt. 2018. «DeepProbLog: Programación Lógica Probabilística Neuronal». *Avances en los Sistemas de Procesamiento de Información Neuronal* 31. <https://proceedings.neurips.cc/paper/2018/hash/dc5d637ed5e62c36ecb73b654b05ba2a-Abstract.html>
- Mao, Ziyu, Jingyi Wang, Jun Sun, Shengchao Qin y Jiawen Xiong. 2025. «Modelado automático asistido por LLM para la verificación del protocolo de seguridad». *Actas de la 47a Conferencia Internacional IEEE/ACM sobre Ingeniería de Software, 2025*. https://ink.library.smu.edu.sg/sis_research/10297/
- Mitchell, Jacqueline L., Brian Hyeongseok Kim, Chenyu Zhou y Chao Wang. 2025. «¿Puede LLMs Razonar Formalmente como Intérpretes Abstractos para el Análisis de Programas?». *arXiv:2503.12686*. <https://arxiv.org/abs/2503.12686>
- Mou, Lili, Zhengdong Lu, Hang Li y Zhi Jin. 2017. «Acoplamiento de ejecuciones distribuidas y simbólicas para consultas de lenguaje natural». *Actas de ICML 2017*. <https://proceedings.mlr.press/v70/mou17a.html>
- Olausson, Theo X., Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum y Roger Levy. 2023. «LINC: Un enfoque neurosimbólico para el razonamiento lógico mediante la combinación de modelos de lenguaje con probadores lógicos de primer orden». *Actas de EMNLP 2023. Asociación para la Lingüística Computacional*. <https://aclanthology.org/2023.emnlp-main.313/>
- Pan, Liangming, Alon Albalak, Xinyi Wang y William Yang Wang. 2023. «Logic-LM: Empoderando modelos de lenguaje grandes con solucionadores simbólicos para un razonamiento lógico fiel». *Resultados de EMNLP 2023. Asociación para la Lingüística Computacional*. <https://aclanthology.org/2023.findings-emnlp.248/>
- Pasupat, Panupong y Percy Liang. 2016. «Inferir formularios lógicos de las denotaciones». *Actas de ACL 2016. Asociación para la Lingüística Computacional*. <https://aclanthology.org/P16-1004/>
- Pei, Yu, Yongping Du y Xingnan Jin. 2025. «FoVer: Verificación lógica de primer orden para el razonamiento del lenguaje natural». *Transacciones de la Asociación de Lingüística Computacional* 13: 1340–1359. <https://aclanthology.org/2025.tacl-1.61/>

- Tafjord, Oyvind, Bhavana Dalvi Mishra y Peter Clark. 2021. «ProofWriter: Generación de implicaciones, pruebas y declaraciones abductivas sobre el lenguaje natural». Resultados de ACL-IJCNLP 2021. Asociación para la Lingüística Computacional. <https://aclanthology.org/2021.findings-acl.317/>
- Tantakoun, Nat, Christian Muise y Yilun Zhu. 2025. «Modelos de lenguaje grandes como formalizadores de planificación: un estudio». Resultados de ACL 2025. <https://aclanthology.org/2025.findings-acl.1291/>
- Tziafas, Georgios y Hamidreza Kasaei. 2026. «Planificación de tareas neurosimbólicas y manipulación de objetos con modelos de lenguaje grandes». Robots Autónomos 50. <https://doi.org/10.1007/s10514-026-10230-6>
- Wang, Bailin, Richard Shin, Xiaodong Liu, Oleksandr Polozov y Matthew Richardson. 2018. «Decodificación de programas neuronales guiada por ejecución». Actas del primer taller sobre síntesis de programas aprendidos. <https://arxiv.org/abs/1807.03100>
- Wang, Boxin, Chejian Xu, Xiangyu Liu, Yu Cheng y Bo Li. 2023. «La incrustación de palabras consciente de la robustez mejora la robustez certificada a las sustituciones adversarias de palabras». Resultados de ACL 2023. Asociación para la Lingüística Computacional. <https://aclanthology.org/2023.findings-acl.620/>
- Wang, Jiayimei, Tao Ni, Wei-Bin Lee y Qingchuan Zhao. 2025. «Un estudio contemporánea del análisis de programas asistidos por modelos de lenguaje grande». Tendencias en Inteligencia Artificial. <https://arxiv.org/abs/2502.18474>
- Wang, Wenguan, Yi Yang y Fei Wu. 2025. «Hacia la información basada en datos y conocimientos AI: un estudio sobre computación neurosimbólica». Transacciones IEEE en análisis de patrones e inteligencia de máquinas 47(2): 878–899. <https://doi.org/10.1109/TPAMI.2024.3483273>
- Wang, Yu, You Zhang, Hao Zhang y Dan Xu. 2026. «YNU-NLP en SemEval-2026 Tarea 11: Un enfoque neuro-simbólico con mecanismo de reflexión que desenreda contenido y razonamiento formal en modelos de lenguaje». Actas del XX Taller Internacional de Evaluación Semántica. Asociación para la Lingüística Computacional. <https://doi.org/10.18653/v1/2026.semeval-1.126>
- Weng, Ke, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu y Tiancheng Zhang. 2025. «Autoformalización en la era de los modelos de lenguaje grande: un estudio». arXiv:2505.23486. <https://arxiv.org/abs/2505.23486>
- Xia, Shihao, Mengting He, Shuai Shao, Tingting Yu, Yiyang Zhang, Nobuko Yoshida y Linhai Song. 2026. «SymGPT: Auditoría de contratos inteligentes mediante la combinación de ejecución simbólica con modelos de lenguaje grandes». Actas de la ACM sobre Lenguajes de Programación 10 (OOPSLA1), artículo 109. <https://doi.org/10.1145/3798217>
- Yang, Zonglin, et al. 2024. «Autoformalización de problemas de razonamiento del lenguaje natural a la lógica de primer orden». Actas de EMNLP 2024. Asociación para la Lingüística Computacional. <https://aclanthology.org/2024.emnlp-main.1132/>
- Ye, Mao, Chengyue Gong y Qiang Liu. 2020. «SAFER: Un enfoque sin estructura para la robustez certificada a las sustituciones de palabras adversarias». Actas de ACL 2020. Asociación para la Lingüística Computacional. <https://aclanthology.org/2020.acl-main.317/>
- Ye, Xi, Qiaochu Chen, Isil Dillig y Greg Durrett. 2023. «SatLM: modelos de lenguaje asistidos por satisfacibilidad mediante prompting declarativo». Avances en los Sistemas de Procesamiento de Información Neuronal 36. <https://arxiv.org/abs/2305.09656>
- Zheng, Zihan, Tianle Cui, Chuwen Xie, Jiahui Pan, Qianglong Chen y Lewei He. 2025. «PlanningArena: Un punto de referencia modular para la evaluación multidimensional de la planificación y el aprendizaje de herramientas». Actas de ACL 2025, 31047–31086. <https://doi.org/10.18653/v1/2025.acl-long.1499>